

EX.NO: 1

DATE:

CREATION OF LARGE DATA SET

AIM:

To create large datasets and store it in different data formats such as .csv, .json,.xml format.

CODE:

1. Create a 100 record employee dataset with the following attributes and save it as .csv

Attribute	Description
<u>Emp id</u>	Unique integer number
Name	Should have proper suffix like Dr, <u>Mr.Ms</u>
Age	25 to 65
Salary	30,000 to 10000000 Provide appropriate salary lab
Designation	Doctor, Nurse, Radiologist, Pharmacist, Manager, Lab Technician, Paramedic, System Administrator
Gender	Male/Female
Experience	1-25
City	Chennai, Bangalore, Hyderabad, Pune
Year of Joining	2000 to 2026

```
import pandas as pd
from faker import Faker
import numpy as np
```

```
fake = Faker()
no = 100
```

1. Define specific salary ranges (Annual in INR) for each role

```
salary_ranges = {
    "Doctor": (1800000, 4500000),    # 18L - 45L
    "Manager": (1200000, 2500000),    # 12L - 25L
    "Radiologist": (1000000, 2400000), # 10L - 24L
    "System Administrator": (600000, 1500000), # 6L - 15L
    "Pharmacist": (400000, 900000),    # 4L - 9L
    "Nurse": (350000, 850000),        # 3.5L - 8.5L
    "Lab Technician": (300000, 700000), # 3L - 7L
    "Paramedic": (250000, 600000)     # 2.5L - 6L
}
```

```

designations = list(salary_ranges.keys())
# 2. Generate baseline arrays
designation_list = np.random.choice(designations, size=no)
gender_list = np.random.choice(["Male", "Female"], size=no)

# 3. Build names and salaries based on role logic
names = []
salaries = []

for desig, gender in zip(designation_list, gender_list):
    # Name generation logic
    base_name = f'{fake.first_name()} {fake.last_name()}'
    if desig == "Doctor":
        prefix = "Dr."
    elif gender == "Male":
        prefix = "Mr."
    else:
        prefix = np.random.choice(["Mrs.", "Miss"])
    names.append(f'{prefix} {base_name}')

    # Salary generation logic based on the range mapping
    min_sal, max_sal = salary_ranges[desig]
    salaries.append(np.random.randint(min_sal, max_sal))

# 4. Assemble the dataframe
data = {
    "ID": np.arange(1, no + 1),
    "Name": names,
    "Age": np.random.randint(21, 65, size=no),
    "Salary": salaries,
    "Designation": designation_list,
    "Rating": np.round(np.random.uniform(1.0, 5.0, size=no), 1),
    "Gender": gender_list,
    "Experience": np.random.randint(1, 25, size=no),
    "City": np.random.choice(["Hyderabad", "Chennai", "Bangalore", "Pune"], size=no),
    "YOJ": np.random.randint(2000, 2025, size=no)
}

df = pd.DataFrame(data)

# Sort by designation just to verify the ranges easily in the console
print(df.head(10))

df.to_csv("emp.csv", index=False)

```

OUTPUT:

ID	Name	Age	Salary	Designation	Rating	Gender	\
87 88	Dr. Maria Stein	39	2489373	Doctor	2.3	Male	
78 79	Dr. Kathleen Morrison	58	3031369	Doctor	1.9	Female	
77 78	Dr. Kayla Pace	39	1978807	Doctor	1.0	Female	
29 30	Dr. Brenda Ray	55	3873689	Doctor	2.4	Male	
76 77	Dr. Angela Deleon	25	2324293	Doctor	3.3	Male	
25 26	Dr. Amanda Scott	51	3456938	Doctor	2.6	Female	
73 74	Dr. Lee Brooks	49	2526573	Doctor	3.6	Female	
85 86	Dr. Paul Chavez	54	4418320	Doctor	2.8	Female	
42 43	Dr. Greg Stark	56	2567819	Doctor	2.9	Female	
71 72	Dr. Riley Davis	34	2349992	Doctor	5.0	Female	

Experience	City	Y0J
87 13	Pune	2002
78 12	Chennai	2009
77 16	Chennai	2023
29 13	Pune	2004
76 20	Pune	2014
25 14	Hyderabad	2005
73 18	Pune	2023
85 19	Bangalore	2006
42 24	Pune	2014
71 20	Hyderabad	2003

CSV file 'emp.csv' created successfully.

2. Create a 100 record social media dataset with the following attributes and save it as .json

Attribute	Data Type	Description	Possible Values / Range
article_id	String	Unique identifier for each article	ART001, ART002, ..., ART100
title	String	Headline of the news article	Any meaningful news headline
author	String	Name of the journalist/author	Generated names (e.g., "Priya Sharma", "Rahul Kumar")
publisher	String	News publishing organization	Daily News, Global Times, Tech Today, Sports Weekly, Health Chronicle, Business World
category	String	News category	Politics, Technology, Sports, Health, Business, Entertainment, Education, Science
published_date	Date	Date the article was published	Any date within the last 2 years

content	Text	Main content of the news article	100–500 characters of article text
views	Integer	Number of times the article was viewed	100 – 100,000
likes	Integer	Number of likes received	10 – 10,000
comments	Integer	Number of user comments	0 – 2,000
shares	Integer	Number of times the article was shared	0 – 5,000
reading_time_minutes	Integer	Estimated reading time	2 – 15 minutes

```
import pandas as pd
from faker import Faker
import numpy as np
```

```
# Initialize Faker with Indian locale
fake = Faker('en_IN')
no = 100
```

```
# Configuration lists from requirements
publishers = ["Daily News", "Global Times", "Tech Today", "Sports Weekly", "Health Chronicle",
"Business World"]
categories = ["Politics", "Technology", "Sports", "Health", "Business", "Entertainment", "Education",
"Science"]
```

```
# 1. Attribute Generation (12 Columns)
```

```
article_ids = [f'ART{str(i).zfill(3)}' for i in range(1, no + 1)]
categories_list = np.random.choice(categories, size=no)
publishers_list = np.random.choice(publishers, size=no)
authors_list = [f'{fake.first_name()} {fake.last_name()}' for _ in range(no)]
titles_list = [f'Trends in {cat}' for cat in categories_list]
dates_list = [fake.date_between(start_date="-2y", end_date="today").strftime('%Y-%m-%d') for _ in
range(no)]
contents_list = [fake.text(max_nb_chars=140).replace('\n', ' ') for _ in range(no)]
```

```
views_list = np.random.randint(100, 100001, size=no)
likes_list = np.random.randint(10, 10001, size=no)
comments_list = np.random.randint(0, 2001, size=no)
shares_list = np.random.randint(0, 5001, size=no)
reading_time_list = np.random.randint(2, 16, size=no)
```

```
# 2. Assemble Dataframe in precise sequence
data = {
```

```

"article_id": article_ids,
"title": titles_list,
"author": authors_list,
"publisher": publishers_list,
"category": categories_list,
"published_date": dates_list,
"content": contents_list,
"views": views_list,
"likes": likes_list,
"comments": comments_list,
"shares": shares_list,
"reading_time_minutes": reading_time_list
}

```

```
df = pd.DataFrame(data)
```

```
# 3. Print the data with explicit layout column spacing widths
```

```
# This ensures that headers and content line up exactly row by row
```

```
headers = list(data.keys())
```

```
col_widths = [11, 22, 18, 18, 14, 15, 20, 8, 7, 9, 7, 10]
```

```
# Generate the header row string
```

```
header_row = "".join([f'{headers[i]:<{col_widths[i]}}' for i in range(len(headers))])
```

```
print("\n" + "=" * sum(col_widths))
```

```
print(header_row)
```

```
print("=" * sum(col_widths))
```

```
# Loop over the first 10 rows and print with structured column padding widths
```

```
for index, row in df.head(10).iterrows():
```

```
    row_str = ""
```

```
    for idx, col in enumerate(df.columns):
```

```
        val = str(row[col])
```

```
        # If the string value exceeds the allocated column size, truncate it nicely with '..'
```

```
        if len(val) > col_widths[idx] - 2:
```

```
            val = val[:col_widths[idx] - 4] + ".."
```

```
        row_str += f'{val:<{col_widths[idx]}}'
```

```
    print(row_str)
```

```
print("=" * sum(col_widths) + "\n")
```

```
# 4. Save to files
```

```
df.to_csv("news_articles_extended.csv", index=False)
```

```
df.to_json("news_articles.json", orient="records", indent=4)
```

OUTPUT:

article_id	title	author	publisher	category	published_date	content	views	likes	comments	shares	reading_time_minutes
ART001	Trends in Entertai..	Chanakya Chatt..	Global Times	Entertainm..	2024-07-01	Autem sapiente e..	32175	1023	1638	1678	9
ART002	Trends in Health	Neel Sathe	Business World	Health	2024-08-06	Asperiores quis ..	43260	3963	1726	4151	4
ART003	Trends in Health	Yashvi Bhandari	Tech Today	Health	2025-01-28	Nihil magnam max..	58750	7007	661	3142	11
ART004	Trends in Business	Chandresh Ravi	Business World	Business	2024-09-09	Minus sapiente i..	95075	1475	1788	1710	11
ART005	Trends in Science	Saksham Bala	Health Chronicle	Science	2026-06-13	Natus optio occa..	5090	1132	667	4143	11
ART006	Trends in Education	Turvi Karnik	Tech Today	Education	2025-09-02	Quas blanditiis ..	20660	3550	1654	2398	6
ART007	Trends in Technology	Girindra Bhasin	Global Times	Technology	2025-04-19	Nesciunt beatae ..	13415	9661	886	1667	10
ART008	Trends in Health	Praneel Mahajan	Daily News	Health	2025-09-02	Et reiciendis in..	32893	4410	1011	284	2
ART009	Trends in Business	Chakradev Tailor	Health Chronicle	Business	2026-03-17	Magnam dicta seq..	79429	4263	1431	905	8
ART010	Trends in Politics	Frederick Dey	Tech Today	Politics	2024-09-25	Nulla ullam aper..	67308	8301	1754	223	3

3. Create a 100 record purchase dataset with the following attributes and save it as .xml.

Attribute
Deviceid
OwnerName
IPAddress
OS
OSversion
PurchaseDate
WarrantyStatus
BatteryHealth
IsOnline

```
import pandas as pd
from faker import Faker
import numpy as np

# Initialize Faker with Indian locale for owner names
fake = Faker('en_IN')
no = 100

# 1. Configuration maps for realistic OS and version pairings
os_options = ["Windows", "macOS", "Linux", "Android", "iOS"]
```

```

os_versions = {
    "Windows": ["10 Home", "11 Pro", "11 Home"],
    "macOS": ["Ventura 13.5", "Sonoma 14.2", "Sequoia 15.0"],
    "Linux": ["Ubuntu 22.04", "Fedora 40", "Debian 12"],
    "Android": ["Android 13", "Android 14", "Android 15"],
    "iOS": ["iOS 16.5", "iOS 17.2", "iOS 18.0"]
}

# 2. Attribute Generation lists
device_ids = [f"DEV{str(i).zfill(3)}" for i in range(1, no + 1)]
owner_names = [f"{fake.first_name()} {fake.last_name()}" for _ in range(no)]
ip_addresses = [fake.ipv4() for _ in range(no)]

# Dynamically link OS and its corresponding versions
os_list = np.random.choice(os_options, size=no)
os_version_list = [np.random.choice(os_versions[os_item]) for os_item in os_list]

# Date, warranty, battery health, and connection status rules
dates_list = [fake.date_between(start_date="-3y", end_date="today").strftime('%Y-%m-%d') for _ in range(no)]
warranty_list = np.random.choice(["Active", "Expired"], size=no, p=[0.6, 0.4])
battery_list = [f"{np.random.randint(50, 101)}%" for _ in range(no)] # 50% to 100%
is_online_list = np.random.choice([True, False], size=no, p=[0.7, 0.3])

# 3. Assemble Dataset into strict column mapping order
data = {
    "Deviceid": device_ids,
    "OwnerName": owner_names,
    "IPAddress": ip_addresses,
    "OS": os_list,
    "OSversion": os_version_list,
    "PurchaseDate": dates_list,
    "WarrantyStatus": warranty_list,
    "BatteryHealth": battery_list,
    "IsOnline": is_online_list
}

df = pd.DataFrame(data)

# 4. Save to XML format (using standard element structure)
df.to_xml("purchase_dataset.xml", index=False, row_name="PurchaseRecord")

print("Successfully generated 'purchase_dataset.xml' with 100 records!")

# 5. Clean preview layout configuration for console confirmation
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

```

```
print("\nPREVIEW OF FIRST 5 XML RECORDS:")
print(df.head(5).to_string(index=False))
```

OUTPUT:

Successfully generated 'purchase_dataset.xml' with 100 records!

PREVIEW OF FIRST 5 XML RECORDS:

Deviceid	OwnerName	IPAddress	OS	OSversion	PurchaseDate	WarrantyStatus	BatteryHealth	IsOnline
DEV001	Vasatika Kapoor	116.171.97.182	macOS	Ventura 13.5	2025-10-16	Expired	89%	False
DEV002	Tamanna Khurana	38.111.177.77	Android	Android 14	2024-02-23	Active	53%	True
DEV003	Praneel Ravi	23.122.93.205	Linux	Ubuntu 22.04	2024-08-31	Active	60%	True
DEV004	Ekani Mody	118.199.59.248	Android	Android 15	2026-03-14	Active	55%	True
DEV005	Sathvik Shroff	54.37.107.243	iOS	iOS 18.0	2023-12-10	Active	81%	True

RESULT:

Thus, the creation of large datasets and store it in different data formats such as csv,json,xml has been implemented successfully.

EX.NO: 2

DATE:

DATA PREPROCESSING

AIM:

To perform various data preprocessing techniques on sample datasets using python and essential libraries.

QUESTIONS:

1. Import and Export the dataset.

i) Import the necessary python libraries.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

ii) Load Bank Marketing dataset into a pandas data frame.

```
[2]: data = pd.read_csv('Bank_Marketing_Synthetic_dataset.csv')
df = pd.DataFrame(data)
print(df)
```

	age	job	marital	education	default	balance	housing	loan	\
0	41	entrepreneur	single	tertiary	yes	22318	yes	no	
1	43	blue-collar	married	unknown	yes	34677	no	no	
2	38	services	single	primary	no	11552	no	no	
3	41	entrepreneur	single	tertiary	yes	22318	yes	no	
4	43	blue-collar	married	unknown	yes	34677	no	no	
..	...	...	...	...	...	...	...	...	
495	20	technician	married	secondary	no	20534	no	no	
496	35	admin.	married	tertiary	no	34123	no	yes	
497	57	blue-collar	single	secondary	yes	45122	no	no	
498	68	retired	divorced	secondary	no	-4	no	no	
499	43	technician	married	tertiary	yes	22720	no	yes	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	unknown	16	oct	204	5	-1	1	unknown	no
1	telephone	4	jul	1471	5	-1	2	failure	yes
2	cellular	16	jan	403	8	201	2	other	yes
3	unknown	16	oct	204	5	-1	1	unknown	no
4	telephone	4	jul	1471	5	-1	2	failure	yes
..	...	...	...	...	...	...	...	...	...
495	unknown	5	aug	1220	3	-1	9	unknown	no
496	telephone	15	aug	1012	3	177	6	failure	yes
497	unknown	4	aug	1153	2	-1	3	success	yes
498	unknown	29	jun	609	1	-1	3	failure	yes
499	unknown	10	aug	1356	5	65	0	other	no

iii) Display the first and last five records.

3]: `df.head(5)`

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	41	entrepreneur	single	tertiary	yes	22318	yes	no	unknown	16	oct	204	5	-1	1	unknown	no
1	43	blue-collar	married	unknown	yes	34677	no	no	telephone	4	jul	1471	5	-1	2	failure	yes
2	38	services	single	primary	no	11552	no	no	cellular	16	jan	403	8	201	2	other	yes
3	41	entrepreneur	single	tertiary	yes	22318	yes	no	unknown	16	oct	204	5	-1	1	unknown	no
4	43	blue-collar	married	unknown	yes	34677	no	no	telephone	4	jul	1471	5	-1	2	failure	yes

4]: `df.tail(5)`

	age	job	marital	education	default	balance	housing	loan	contact	day	month	duration	campaign	pdays	previous	poutcome	y
495	20	technician	married	secondary	no	20534	no	no	unknown	5	aug	1220	3	-1	9	unknown	no
496	35	admin.	married	tertiary	no	34123	no	yes	telephone	15	aug	1012	3	177	6	failure	yes
497	57	blue-collar	single	secondary	yes	45122	no	no	unknown	4	aug	1153	2	-1	3	success	yes
498	68	retired	divorced	secondary	no	-4	no	no	unknown	29	jun	609	1	-1	3	failure	yes
499	43	technician	married	tertiary	yes	22720	no	yes	unknown	10	aug	1356	5	65	0	other	no

iv) Determine the number of rows and columns.

```
[7]: rows, columns = df.shape
      print(f"Rows: {rows}, Columns: {columns}")
```

Rows: 500, Columns: 17

v) Display the datatypes of all attributes.

```
[8]: print(df.dtypes)
```

```
age          int64
job          object
marital      object
education    object
default      object
balance      int64
housing      object
loan         object
contact      object
day          int64
month        object
duration     int64
campaign     int64
pdays       int64
previous     int64
poutcome     object
y           object
dtype: object
```

vi) Generate summary statistics for numerical features.

```
[6]: df.describe()
```

```
[6]:
```

	age	balance	day	duration	campaign	pdays	previous
count	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000	500.000000
mean	43.956000	24633.386000	15.720000	990.282000	5.540000	73.724000	4.948000
std	14.897716	14834.895597	8.692567	554.674006	2.801445	94.083917	3.171342
min	18.000000	-907.000000	1.000000	35.000000	1.000000	-1.000000	0.000000
25%	31.000000	11915.000000	8.000000	544.000000	3.000000	-1.000000	2.000000
50%	43.000000	24689.000000	16.000000	967.500000	6.000000	3.500000	5.000000
75%	57.000000	36703.500000	23.000000	1451.500000	8.000000	149.250000	8.000000
max	70.000000	49991.000000	31.000000	1988.000000	10.000000	300.000000	10.000000

2. Identify and Handle missing values.

i) Identify missing values using appropriate Pandas functions.

```
[10]: placeholder=["unknown"]
value=df.isin(placeholder)
print(value.sum())
```

```
age          0
job          0
marital      0
education    144
default      0
balance      0
housing      0
loan         0
contact     158
day          0
month        0
duration     0
campaign     0
pdays       0
previous     0
poutcome    128
y            0
dtype: int64
```

ii) Treat the value "unknown" as a missing value.

```
[11]: df.replace("unknown", np.nan, inplace=True)
print("Replaced 'unknown' values with NaN objects.")
print(df)
```

Replaced 'unknown' values with NaN objects.

	age	job	marital	education	default	balance	housing	loan	\
0	41	entrepreneur	single	tertiary	yes	22318	yes	no	
1	43	blue-collar	married	NaN	yes	34677	no	no	
2	38	services	single	primary	no	11552	no	no	
3	41	entrepreneur	single	tertiary	yes	22318	yes	no	
4	43	blue-collar	married	NaN	yes	34677	no	no	
..	...	...	...	...	...	...	...	...	
495	20	technician	married	secondary	no	20534	no	no	
496	35	admin.	married	tertiary	no	34123	no	yes	
497	57	blue-collar	single	secondary	yes	45122	no	no	
498	68	retired	divorced	secondary	no	-4	no	no	
499	43	technician	married	tertiary	yes	22720	no	yes	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	NaN	16	oct	204	5	-1	1	NaN	no
1	telephone	4	jul	1471	5	-1	2	failure	yes
2	cellular	16	jan	403	8	201	2	other	yes
3	NaN	16	oct	204	5	-1	1	NaN	no
4	telephone	4	jul	1471	5	-1	2	failure	yes
..	...	...	...	...	...	...	...	...	...
495	NaN	5	aug	1220	3	-1	9	NaN	no
496	telephone	15	aug	1012	3	177	6	failure	yes
497	NaN	4	aug	1153	2	-1	3	success	yes
498	NaN	29	jun	609	1	-1	3	failure	yes
499	NaN	10	aug	1356	5	65	0	other	no

[500 rows x 17 columns]

iii) Calculate the percentage of missing values in each column.

```
[12]: missing_percentage = (df.isnull().sum() / len(df)) * 100
print("Percentage of missing values per column:")
print(missing_percentage)
```

Percentage of missing values per column:

age	0.0
job	0.0
marital	0.0
education	28.8
default	0.0
balance	0.0
housing	0.0
loan	0.0
contact	31.6
day	0.0
month	0.0
duration	0.0
campaign	0.0
pdays	0.0
previous	0.0
poutcome	25.6
y	0.0
dtype:	float64

iv) Handle missing values using suitable imputation techniques.

```
[13]: for col in df.columns:
        if df[col].isnull().any():
            mode_val = df[col].mode()[0]
            df[col].fillna(mode_val, inplace=True)
            print(f"Imputed missing entries in '{col}' with Mode: {mode_val}")
```

Imputed missing entries in 'education' with Mode: tertiary
 Imputed missing entries in 'contact' with Mode: telephone
 Imputed missing entries in 'poutcome' with Mode: success

v) Verify that missing values have been addressed

```
[14]: total_remaining_missing = df.isnull().sum().sum()
        print(f"Total remaining missing values in the entire dataset: {total_remaining_missing}")
```

Total remaining missing values in the entire dataset: 0

3. Detect and remove duplicate records.

i) Find duplicate records in the dataset.

```
[15]: duplicates = df[df.duplicated()]
        print("\nDuplicate Records:")
        print(duplicates)
```

Duplicate Records:

	age	job	marital	education	default	balance	housing	loan	\
3	41	entrepreneur	single	tertiary	yes	22318	yes	no	
4	43	blue-collar	married	tertiary	yes	34677	no	no	
11	41	entrepreneur	single	tertiary	yes	22318	yes	no	
12	43	blue-collar	married	tertiary	yes	34677	no	no	
25	47	management	married	primary	yes	15570	no	yes	
26	23	management	married	tertiary	no	17738	yes	yes	
113	47	management	married	primary	yes	15570	no	yes	
114	23	management	married	tertiary	no	17738	yes	yes	
186	43	services	single	tertiary	yes	36168	yes	yes	
187	52	student	single	secondary	no	31906	yes	no	
188	42	student	married	primary	yes	918	yes	no	
263	61	technician	single	tertiary	yes	40627	no	yes	
264	33	technician	married	tertiary	yes	82	no	no	
265	31	technician	divorced	tertiary	yes	30242	yes	yes	
266	67	admin.	married	tertiary	yes	10549	no	yes	
267	22	entrepreneur	married	secondary	yes	5874	yes	no	
268	22	technician	single	tertiary	no	18964	no	yes	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
3	telephone	16	oct	204	5	-1	1	success	no
4	telephone	4	jul	1471	5	-1	2	failure	yes

ii) Count the total number of duplicate rows.

```
[17]: duplicate_count = df.duplicated().sum()
        print("\nTotal Number of Duplicate Rows:", duplicate_count)
```

Total Number of Duplicate Rows: 17

iii) Remove duplicate entries.

```
[18]: df_clean = df.drop_duplicates()
      print(df_clean)
```

	age	job	marital	education	default	balance	housing	loan	\
0	41	entrepreneur	single	tertiary	yes	22318	yes	no	
1	43	blue-collar	married	tertiary	yes	34677	no	no	
2	38	services	single	primary	no	11552	no	no	
5	41	technician	divorced	primary	yes	16703	no	no	
6	38	blue-collar	married	secondary	no	18972	no	no	
..	...	...	...	...	...	...	...	...	
495	20	technician	married	secondary	no	20534	no	no	
496	35	admin.	married	tertiary	no	34123	no	yes	
497	57	blue-collar	single	secondary	yes	45122	no	no	
498	68	retired	divorced	secondary	no	-4	no	no	
499	43	technician	married	tertiary	yes	22720	no	yes	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	telephone	16	oct	204	5	-1	1	success	no
1	telephone	4	jul	1471	5	-1	2	failure	yes
2	cellular	16	jan	403	8	201	2	other	yes
5	telephone	8	oct	1933	7	-1	8	success	yes
6	telephone	21	mar	1207	7	78	7	success	no
..	...	...	...	...	...	...	...	...	...
495	telephone	5	aug	1220	3	-1	9	success	no
496	telephone	15	aug	1012	3	177	6	failure	yes
497	telephone	4	aug	1153	2	-1	3	success	yes
498	telephone	29	jun	609	1	-1	3	failure	yes
499	telephone	10	aug	1356	5	65	0	other	no

[483 rows x 17 columns]

iv) Compare the dataset dimensions before and after duplicate removal.

```
print("\nDataset dimensions before removing duplicates:", df.shape)
print("\nDataset dimensions after removing duplicates:", df_clean.shape)
```

Dataset dimensions before removing duplicates: (500, 17)

Dataset dimensions after removing duplicates: (493, 17)

4. Perform Categorical data encoding.

i) Identify categorical attributes.

```
categorical_attributes = df.select_dtypes(include=['object']).columns.tolist()
print("Identified Categorical Attributes:\n", categorical_attributes)
```

Identified Categorical Attributes:

['job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'poutcome', 'y']

ii) Apply Label Encoding to binary categorical variables.

```
from sklearn.preprocessing import LabelEncoder,MinMaxScaler,StandardScaler
binary_categorical_cols = ['default', 'housing', 'loan', 'y']
le = LabelEncoder()
for col in binary_categorical_cols:
    df[col] = le.fit_transform(df[col])
print("Label Encoding:", binary_categorical_cols)
```

```
Label Encoding: ['default', 'housing', 'loan', 'y']
```

iii) Apply One-Hot Encoding to multi-category variables.

```
import pandas as pd
df.columns = df.columns.str.strip()
multi_category_cols = ['job', 'marital', 'education']
existing_cols = [col for col in multi_category_cols if col in df.columns]
missing_cols = set(multi_category_cols) - set(existing_cols)
if missing_cols:
    print(f"Missing columns ignored: {missing_cols}")
if existing_cols:
    df = pd.get_dummies(
        df,
        columns=existing_cols,
        drop_first=True,
        dtype=int
    )
else:
    print("None of the target columns exist in the DataFrame!")
print(df.columns.tolist())
```

```
['age', 'default', 'balance', 'housing', 'loan', 'contact', 'day', 'month', 'duration', 'campaign', 'pdays', 'previous', 'r', 'job_entrepreneur', 'job_management', 'job_retired', 'job_services', 'job_student', 'job_technician', 'marital_married', 'marital_secondary', 'education_tertiary']
```

iv) Compare the dataset before and after encoding.

```
print("\nDataset Before Encoding:")
print(df.head())
```

Dataset Before Encoding:

	age	job	marital	education	default	balance	housing	loan	\
0	41	entrepreneur	single	tertiary	1	22318	1	0	
1	43	blue-collar	married	unknown	1	34677	0	0	
2	38	services	single	primary	0	11552	0	0	
3	49	entrepreneur	single	primary	0	15696	0	1	
4	31	student	single	unknown	1	5204	0	1	

	contact	day	month	duration	campaign	pdays	previous	poutcome	y
0	unknown	16	oct	204	5	-1	1	unknown	0
1	telephone	4	jul	1471	5	-1	2	failure	1
2	cellular	16	jan	403	8	201	2	other	1
3	unknown	7	jan	1335	2	-1	10	unknown	0
4	unknown	20	feb	99	2	40	6	other	1

```
print("Dataset after Encoding:\n")
print(df.head())
```

Dataset after Encoding:

	age	default	balance	housing	loan	contact	day	month	duration	\
0	41	1	22318	1	0	unknown	16	oct	204	
1	43	1	34677	0	0	telephone	4	jul	1471	
2	38	0	11552	0	0	cellular	16	jan	403	
3	49	0	15696	0	1	unknown	7	jan	1335	
4	31	1	5204	0	1	unknown	20	feb	99	

	campaign	...	job_management	job_retired	job_services	job_student	\
0	5	...	0	0	0	0	
1	5	...	0	0	0	0	
2	8	...	0	0	1	0	
3	2	...	0	0	0	0	
4	2	...	0	0	0	1	

	job_technician	marital_married	marital_single	education_secondary	\
0	0	0	1	0	
1	0	1	0	0	
2	0	0	1	0	
3	0	0	1	0	
4	0	0	1	0	

5. Normalize and Standardize numerical features.

i) Identify numerical features.

```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns.tolist()
print("Numerical Features:")
print(numerical_cols)
```

Numerical Features:

['age', 'default', 'balance', 'housing', 'loan', 'day', 'duration', 'campaign', 'pdays',

ii) Apply Min-Max Normalization.

```
minmax_scaler = MinMaxScaler()
df_minmax = df.copy()
df_minmax[numerical_cols] = minmax_scaler.fit_transform(df[numerical_cols])
print("\nDataset After Min-Max Normalization:")
print(df_minmax[numerical_cols].head())
```

Dataset After Min-Max Normalization:

	age	default	balance	housing	loan	day	duration	campaign
0	0.442308	1.0	0.456305	1.0	0.0	0.500000	0.086534	0.444444
1	0.480769	1.0	0.699124	0.0	0.0	0.100000	0.735279	0.444444
2	0.384615	0.0	0.244784	0.0	0.0	0.500000	0.188428	0.777778
3	0.596154	0.0	0.326201	0.0	1.0	0.200000	0.665643	0.111111
4	0.250000	1.0	0.120064	0.0	1.0	0.633333	0.032770	0.111111

iii) Apply Standardization (Z-score normalization).

```
standard_scaler = StandardScaler()
df_standard = df.copy()
df_standard[numerical_cols] = standard_scaler.fit_transform(df[numerical_cols])
print("\nDataset After Standardization (Z-score):")
print(df_standard[numerical_cols].head())
```

Dataset After Standardization (Z-score):

	age	default	balance	housing	loan	day	duration	\
0	-0.208696	0.922958	-0.173050	0.956927	-1.008032	0.024688	-1.449724	
1	-0.073091	0.922958	0.659126	-1.045012	-1.008032	-1.359660	0.840967	
2	-0.412104	-1.083473	-0.897964	-1.045012	-1.008032	0.024688	-1.089939	
3	0.333725	-1.083473	-0.618933	-1.045012	0.992032	-1.013573	0.595084	
4	-0.886723	0.922958	-1.325398	-1.045012	0.992032	0.486137	-1.639560	

iv) Compare the transformed values with the original dataset.

```
[29]: print("Comparison of first record values:")
print("Original numerical features (from non-transformed view):\n", df_minmax[numerical_features].head(1))
print("Standardized Z-Score view:\n", df_standardized[numerical_features].head(1))
```

Comparison of first record values:

Original numerical features (from non-transformed view):

	age	balance	duration	campaign
0	0.442308	0.456305	0.086534	0.444444

Standardized Z-Score view:

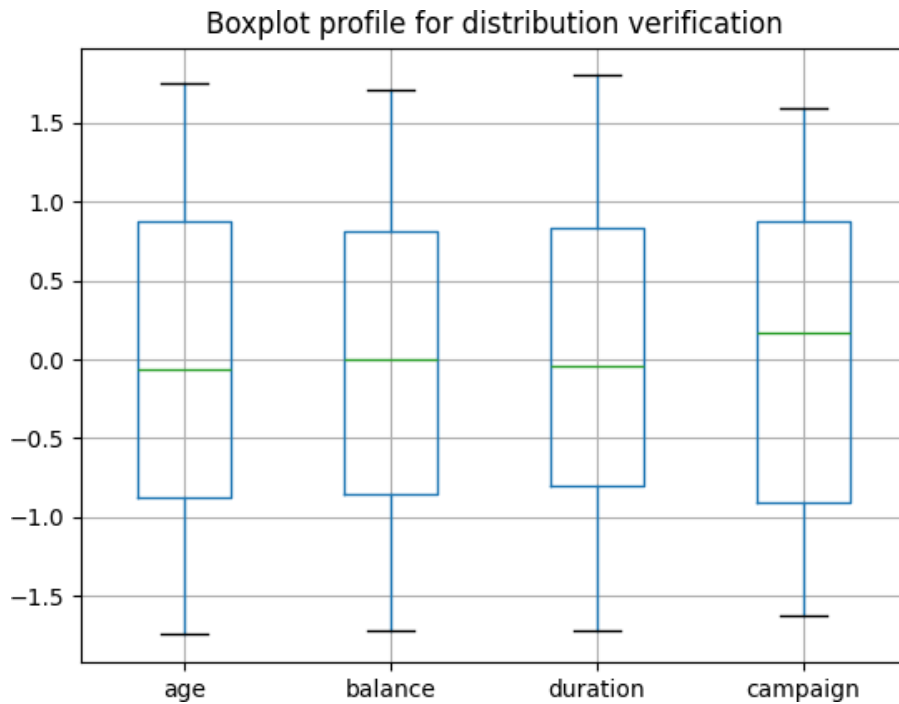
	age	balance	duration	campaign
0	-0.198618	-0.156233	-1.418977	-0.192951

6. Detect and handle outliers.

i) Plot boxplots for numerical attributes.

```
[ ]: pip install matplotlib
```

```
[ ]: import matplotlib.pyplot as plt
df[numerical_features].boxplot()
plt.title("Boxplot profile for distribution verification")
plt.savefig("outlier_boxplot.png")
plt.close()
```



ii) Detect outliers using the Interquartile Range (IQR) method.

```
[34]: outliers_dict = {}
      for col in numerical_features:
          Q1 = df[col].quantile(0.25)
          Q3 = df[col].quantile(0.75)
          IQR = Q3 - Q1

          lower_bound = Q1 - 1.5 * IQR
          upper_bound = Q3 + 1.5 * IQR

          outlier_count = ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()
          outliers_dict[col] = outlier_count

      print("Outlier counts detected per column using IQR method:")
      print(outliers_dict)
```

Outlier counts detected per column using IQR method:
{'age': np.int64(0), 'balance': np.int64(0), 'duration': np.int64(0), 'campaign': np.int64(0)}

iii) Remove or cap the outliers.

```
[35]: for col in numerical_features:
      Q1 = df[col].quantile(0.25)
      Q3 = df[col].quantile(0.75)
      IQR = Q3 - Q1

      lower_bound = Q1 - 1.5 * IQR
      upper_bound = Q3 + 1.5 * IQR

      df[col] = np.clip(df[col], lower_bound, upper_bound)
      print(df[col])
```

```
0    -0.192951
1    -0.192951
2     0.878998
3    -0.192951
4    -0.192951
...
495  -0.907583
496  -0.907583
497  -1.264899
498  -1.622215
499  -0.192951
Name: campaign, Length: 500, dtype: float64
```

iv) Compare the dataset before and after outlier treatment.

```
[36]: print(df[numerical_features].describe())
```

	age	balance	duration	campaign
count	5.000000e+02	5.000000e+02	5.000000e+02	500.000000
mean	-1.971756e-16	9.237056e-17	-8.260059e-17	0.000000
std	1.001002e+00	1.001002e+00	1.001002e+00	1.001002
min	-1.744025e+00	-1.723367e+00	-1.723965e+00	-1.622215
25%	-8.705345e-01	-8.581876e-01	-8.053901e-01	-0.907583
50%	-6.423518e-02	3.752618e-03	-4.111391e-02	0.164365
75%	8.764473e-01	8.144447e-01	8.323446e-01	0.878998
max	1.749938e+00	1.711034e+00	1.800548e+00	1.593630

7. Perform feature engineering.

i) Create age groups such as Young, Middle-aged, and Senior.

```
[37]: df_eng = pd.read_csv("Bank_Marketing_Synthetic_dataset.csv")
df_eng['age_group'] = pd.cut(df_eng['age'], bins=[0, 35, 60, 120], labels=['Young', 'Middle-aged', 'Senior'])
print(df_eng['age_group'])
```

```
0    Middle-aged
1    Middle-aged
2    Middle-aged
3    Middle-aged
4    Middle-aged
...
495    Young
496    Young
497    Middle-aged
498    Senior
499    Middle-aged
Name: age_group, Length: 500, dtype: category
Categories (3, object): ['Young' < 'Middle-aged' < 'Senior']
```

ii) Categorize account balances into Low, Medium, and High.

```
[38]: df_eng['acc_balance'] = pd.qcut(df_eng['balance'], q=3, labels=['Low', 'Medium', 'High'])
print(df_eng['acc_balance'])
```

```
0    Medium
1    High
2    Low
3    Medium
4    High
...
495    Medium
496    High
497    High
498    Low
499    Medium
Name: acc_balance, Length: 500, dtype: category
Categories (3, object): ['Low' < 'Medium' < 'High']
```

iii) Create a new feature indicating whether a customer has any loan.

```
[39]: df_eng['cus_loan'] = np.where((df_eng['housing'] == 'yes') | (df_eng['loan'] == 'yes'), 1, 0)
      print(df_eng['cus_loan'])
```

```
0      1
1      0
2      0
3      1
4      0
..
495    0
496    1
497    0
498    0
499    1
Name: cus_loan, Length: 500, dtype: int64
```

iv) Display the updated dataset.

```
[40]: print(df_eng[['age', 'age_group', 'balance', 'acc_balance', 'cus_loan']].head(5))
```

	age	age_group	balance	acc_balance	cus_loan
0	41	Middle-aged	22318	Medium	1
1	43	Middle-aged	34677	High	0
2	38	Middle-aged	11552	Low	0
3	41	Middle-aged	22318	Medium	1
4	43	Middle-aged	34677	High	0

8. Perform feature selection.

i) Compute the correlation matrix for numerical variables.

```
[32]: correlation_matrix = df[numerical_features].corr()
      print("Correlation Matrix details:")
      print(correlation_matrix)
```

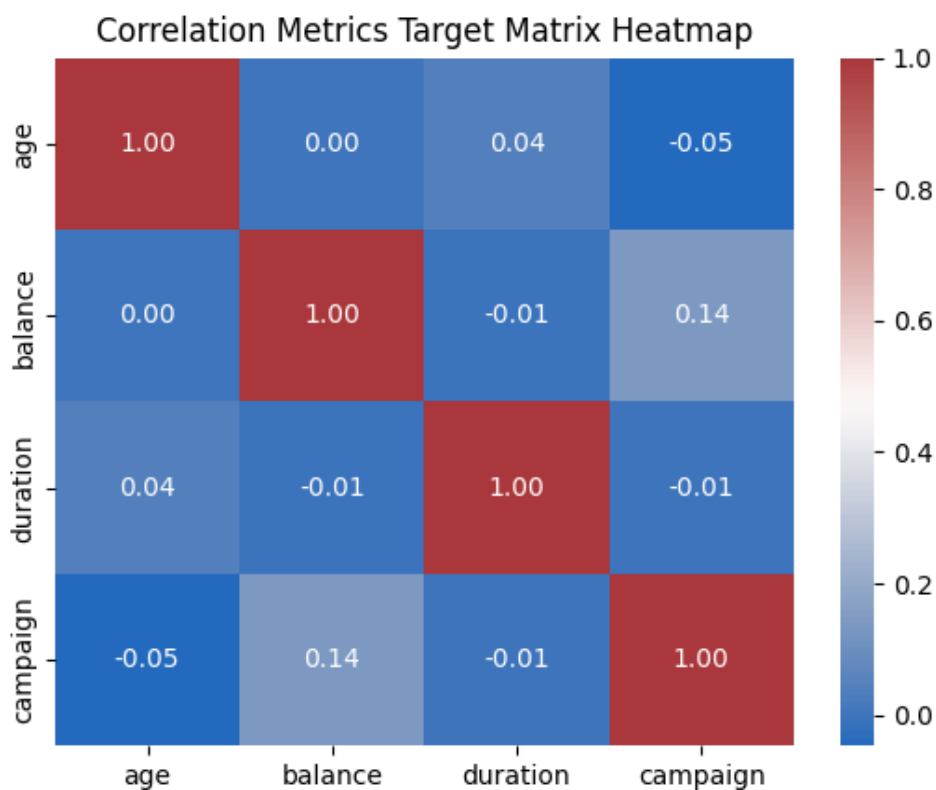
Correlation Matrix details:

	age	balance	duration	campaign
age	1.000000	0.000257	0.041997	-0.046679
balance	0.000257	1.000000	-0.007328	0.140741
duration	0.041997	-0.007328	1.000000	-0.005365
campaign	-0.046679	0.140741	-0.005365	1.000000

ii) Visualize correlations using a heatmap.

```
[ ]: pip install seaborn
```

```
[45]: import seaborn as sns
import matplotlib.pyplot as plt
sns.heatmap(correlation_matrix, annot=True, cmap="vlag", fmt=".2f")
plt.title("Correlation Metrics Target Matrix Heatmap")
plt.savefig("correlation_heatmap.png")
plt.close()
```



iii) Remove highly correlated features, if any.

```
print("\nHighly Correlated Feature Pairs (>|0.85|):")
high_corr_set = set()
for i in range(len(correlation_matrix.columns)):
    for j in range(i):
        corr_value = correlation_matrix.iloc[i, j]
        if abs(corr_value) > 0.85:
            col1 = correlation_matrix.columns[j]
            col2 = correlation_matrix.columns[i]
            print(f"{col1} <--> {col2} : {corr_value:.2f}")
            high_corr_set.add(col2)
print("\nColumns to be Removed:")
print(list(high_corr_set))
df.drop(columns=list(high_corr_set), inplace=True, errors='ignore')
print("\nDataset After Removing Highly Correlated Features:")
print(df.head())
```

Highly Correlated Feature Pairs ($>|0.85|$):

Columns to be Removed:

[]

Dataset After Removing Highly Correlated Features:

	age	default	balance	housing	loan	contact	day	month	duration
0	41	1	22318	1	0	unknown	16	oct	204
1	43	1	34677	0	0	telephone	4	jul	1471
2	38	0	11552	0	0	cellular	16	jan	403
3	49	0	15696	0	1	unknown	7	jan	1335
4	31	1	5204	0	1	unknown	20	feb	99

	campaign	...	job_management	job_retired	job_services	job_student
0	5	...	0	0	0	0
1	5	...	0	0	0	0
2	8	...	0	0	1	0
3	2	...	0	0	0	0
4	2	...	0	0	0	1

iv) Identify the most relevant features for predicting customer subscription.

```
correlation = df.corr(numeric_only=True)
target_corr = correlation['y'].sort_values(ascending=False)
print("Correlation of Features with Target Variable (y):")
print(target_corr)
important_features = target_corr[abs(target_corr) > 0.05]
print("\nMost Relevant Features:")
print(important_features)
important_features = important_features.drop('y')
print("\nSelected Features for Predicting Customer Subscription:")
print(important_features.index.tolist())
```

Correlation of Features with Target Variable (y):

y	1.000000
job_retired	0.125384
education_secondary	0.106668
pdays	0.084034
job_student	0.053948
duration	0.051897
day	0.025438
education_unknown	0.021889
marital_married	0.019411
previous	0.013285
job_services	-0.000576
housing	-0.004426

Most Relevant Features:

y	1.000000
job_retired	0.125384
education_secondary	0.106668
pdays	0.084034
job_student	0.053948
duration	0.051897
loan	-0.061108
campaign	-0.067569
default	-0.068562
job_management	-0.080303
education_tertiary	-0.084717

Name: y, dtype: float64

Selected Features for Predicting Customer Subscription:

['job_retired', 'education_secondary', 'pdays', 'job_student', 'duration']

9. Build a complete data preprocessing pipeline.

i) Load the dataset and Handle missing values.

```
import pandas as pd
ds = pd.read_csv("Bank_Marketing_Synthetic_dataset.csv")
print(ds.head())
```

	age	job	marital	education	default	balance	housing	loan	\
0	41	entrepreneur	single	tertiary	yes	22318	yes	no	
1	43	blue-collar	married	unknown	yes	34677	no	no	
2	38	services	single	primary	no	11552	no	no	
3	49	entrepreneur	single	primary	no	15696	no	yes	
4	31	student	single	unknown	yes	5204	no	yes	

	contact	day	month	duration	campaign	pdays	previous	outcome	y
0	unknown	16	oct	204	5	-1	1	unknown	no
1	telephone	4	jul	1471	5	-1	2	failure	yes
2	cellular	16	jan	403	8	201	2	other	yes
3	unknown	7	jan	1335	2	-1	10	unknown	no
4	unknown	20	feb	99	2	40	6	other	yes

```
import numpy as np
df.replace("unknown", np.nan, inplace=True)
numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
for col in numeric_cols:
    df[col].fillna(df[col].mean(), inplace=True)
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
print(df.isnull().sum())
```

age	0
default	0
balance	0
housing	0
loan	0
contact	0
day	0
month	0
duration	0
campaign	0
pdays	0
previous	0

ii) Remove duplicates.

```
print("Before Removing Duplicates:", df.shape)

df.drop_duplicates(inplace=True)

print("After Removing Duplicates:", df.shape)
```

```
Before Removing Duplicates: (500, 26)
After Removing Duplicates: (493, 26)
```

iii) Encode Categorical variables.

```
from sklearn.preprocessing import LabelEncoder
binary_cols = ['default', 'housing', 'loan', 'y']
le = LabelEncoder()
for col in binary_cols:
    df[col] = le.fit_transform(df[col])
print("Label Encoding Completed")
```

Label Encoding Completed

iv) Normalize numerical attributes.

```
from sklearn.preprocessing import MinMaxScaler
numerical_cols = df.select_dtypes(include=['int64', 'float64'])
                        .columns.tolist()
scaler = MinMaxScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
print(df.head())
```

	age	default	balance	housing	loan	contact	day	month
0	0.442308	1.0	0.456305	1.0	0.0	telephone	0.500000	oct
1	0.480769	1.0	0.699124	0.0	0.0	telephone	0.100000	jul
2	0.384615	0.0	0.244784	0.0	0.0	cellular	0.500000	jan
3	0.596154	0.0	0.326201	0.0	1.0	telephone	0.200000	jan
4	0.250000	1.0	0.120064	0.0	1.0	telephone	0.633333	feb

	duration	campaign	...	job_management	job_retired	job_services
0	0.086534	0.444444	...	0.0	0.0	0.0
1	0.735279	0.444444	...	0.0	0.0	0.0
2	0.188428	0.777778	...	0.0	0.0	1.0
3	0.665643	0.111111	...	0.0	0.0	0.0

v) Detect and treat outliers.

```
numerical_cols = df.select_dtypes(include=['int64', 'float64'])
                        .columns.tolist()
for col in numerical_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[col] = df[col].clip(lower, upper)
print("Outliers Treated Successfully")
```

Outliers Treated Successfully

vi) Select relevant features.

```
correlation = df.corr(numeric_only=True)
target_corr = correlation['y'].sort_values(ascending=False)
print(target_corr)
selected_features = target_corr[abs(target_corr) > 0.05]
print("\nSelected Features:")
print(selected_features)
```

```
y          1.000000
pdays     0.087689
duration    0.046603
day         0.022359
education_unknown 0.018468
```

Name: y, dtype: float64

Selected Features:

```
y          1.000000
pdays     0.087689
loan       -0.059725
default    -0.066375
campaign   -0.072188
Name: y, dtype: float64
```

vii) Split the dataset into training and testing sets.

```
from sklearn.model_selection import train_test_split
X = df.drop('y', axis=1)
y = df['y']
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

Training Data: (394, 25)

Testing Data: (99, 25)

viii) Save the preprocessed dataset as a new CSV file.

```
df.to_csv("preprocessed_dataset.csv", index=False)
```

age	default	balance	housing	loan	contact	day
0.4423076923076924	1.0	0.4563047663955362	1.0	0.0	telephone	0.5
0.48076923076923084	1.0	0.6991237376714212	0.0	0.0	telephone	0.1
0.3846153846153847	0.0	0.24478368501709302	0.0	0.0	cellular	0.5
0.5961538461538463	0.0	0.32620142245274864	0.0	1.0	telephone	0.2
0.25	1.0	0.12006365672521514	0.0	1.0	telephone	0.6333333333333333
0.4423076923076924	1.0	0.3459860898267123	0.0	0.0	telephone	0.23333333333333334
0.3846153846153847	0.0	0.39056544461471965	0.0	0.0	telephone	0.6666666666666666
0.5576923076923077	1.0	0.46316161735235173	0.0	0.0	cellular	0.1
0.5576923076923077	1.0	0.32372588313882666	0.0	1.0	cellular	0.5
0.0961538461538462	0.0	0.36632087704821403	1.0	1.0	cellular	0.5333333333333333

RESULT:

Hence the data preprocessing on a sample dataset using python has been executed successfully.

EX.NO:

DATE:

WEB SCRAPING AND VISUALIZATION

AIM:

To Scrape headlines from the given website and analyze the most frequent words.

Libraries Required:

- **requests** – Fetches webpage HTML content.
- **BeautifulSoup** – Parses and extracts data from HTML.
- **Counter** – Counts frequency of words.
- **matplotlib.pyplot** – Creates charts for visualization.
- **WordCloud** – Generates a word cloud image.
- **re** – Helps in text cleaning using regular expressions.

Question 1: Fetch HTML content from <https://www.bbc.com/news> and extract all headlines present in <h2> and <h3> tags. Print the first 5 headlines.

Explanation:

- requests.get(url) fetches the raw HTML.
- BeautifulSoup(..., 'html.parser') parses the HTML structure.
- .find_all(['h2', 'h3']) finds all headlines in <h2> and <h3> tags.
- The first 5 headlines are displayed as a sample.

PROGRAM :

```
import requests
from bs4 import BeautifulSoup
import re

url = "https://www.bbc.com/news"
response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(response.text, "html.parser")
headlines = soup.find_all(["h2", "h3"])
text = " ".join([h.get_text() for h in headlines]).lower()
text = re.sub(r"[^a-z\s]", "", text)
stopwords = {"the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "an", "by", "is", "are", "this", "that"}
filtered_words = [word for word in text.split() if word not in stopwords]
print(filtered_words[:20])
```

OUTPUT:

```
['trump', 'alleges', 'shocking', 'vulnerabilities', 'us', 'election',
'says', 'civilian', 'infrastructure', 'hit', 'philippines']
```

```
'election', 'security', 'ahead', 'midterms', 'us', 'launches', 'new', 'strikes', 'as', 'iran',
]
```

Question 2: From the extracted headlines:

- Convert text to lowercase.
- Remove numbers, punctuation, and special characters.
- Remove common stopwords:
- {'the','and','to','of','in','for','on','at','with','a','an','by','is','are','this','that'}
- Display the first 20 cleaned words.

PROGRAM :

```
import requests
from bs4 import BeautifulSoup
import re
from collections import Counter

url = "https://www.bbc.com/news"
response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(response.text, "html.parser")
headlines = soup.find_all(["h2", "h3"])
text = " ".join([h.get_text() for h in headlines]).lower()
text = re.sub(r"[^a-z\s]", "", text)
stopwords = {"the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "an", "by", "is", "are", "this", "that"}
filtered_words = [word for word in text.split() if word not in stopwords]
word_counts = Counter(filtered_words)
for word, count in word_counts.most_common(10):
    print(word, count)
```

OUTPUT:

```
['trump', 'alleges', 'shocking', 'vulnerabilities', 'us', 'election',
'says', 'civilian', 'infrastructure', 'hit', 'philippines']
'election', 'security', 'ahead', 'midterms', 'us', 'launches', 'new', 'strikes', 'as', 'iran',
]
```

Question 3: Count the frequency of all words in the cleaned text and display the Top 10 most frequent words along with their counts.**PROGRAM :**

```
import requests
from bs4 import BeautifulSoup
import re
from collections import Counter

url = "https://www.bbc.com/news"
response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
soup = BeautifulSoup(response.text, "html.parser")
headlines = soup.find_all(["h2", "h3"])
text = " ".join([h.get_text() for h in headlines]).lower()
text = re.sub(r"[^a-z\s]", "", text)
stopwords = {"the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "an", "by", "is", "are", "this", "that"}
filtered_words = [word for word in text.split() if word not in stopwords]
word_counts = Counter(filtered_words)
for word, count in word_counts.most_common(10):
    print(word, count)
```

OUTPUT:

```

as 10
us 7
strike 7
world 7
cup 7
trump 5
new 5
chinese 5
but 5
what 5

```

Question 4: Create a bar chart using matplotlib to show the Top 10 most frequent words.

X-axis: Words

Y-axis: Frequency

PROGRAM :

```

import requests
from bs4 import BeautifulSoup
import re
from collections import Counter
import matplotlib.pyplot as plt

url = "https://www.bbc.com/news"

response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})

soup = BeautifulSoup(response.text, "html.parser")

headlines = soup.find_all(["h2", "h3"])

text = " ".join([h.get_text() for h in headlines]).lower()
text = re.sub(r"[^a-z\s]", "", text)

stopwords = {"the", "and", "to", "of", "in", "for", "on", "at", "with", "a", "an", "by", "is", "are", "this", "that"}

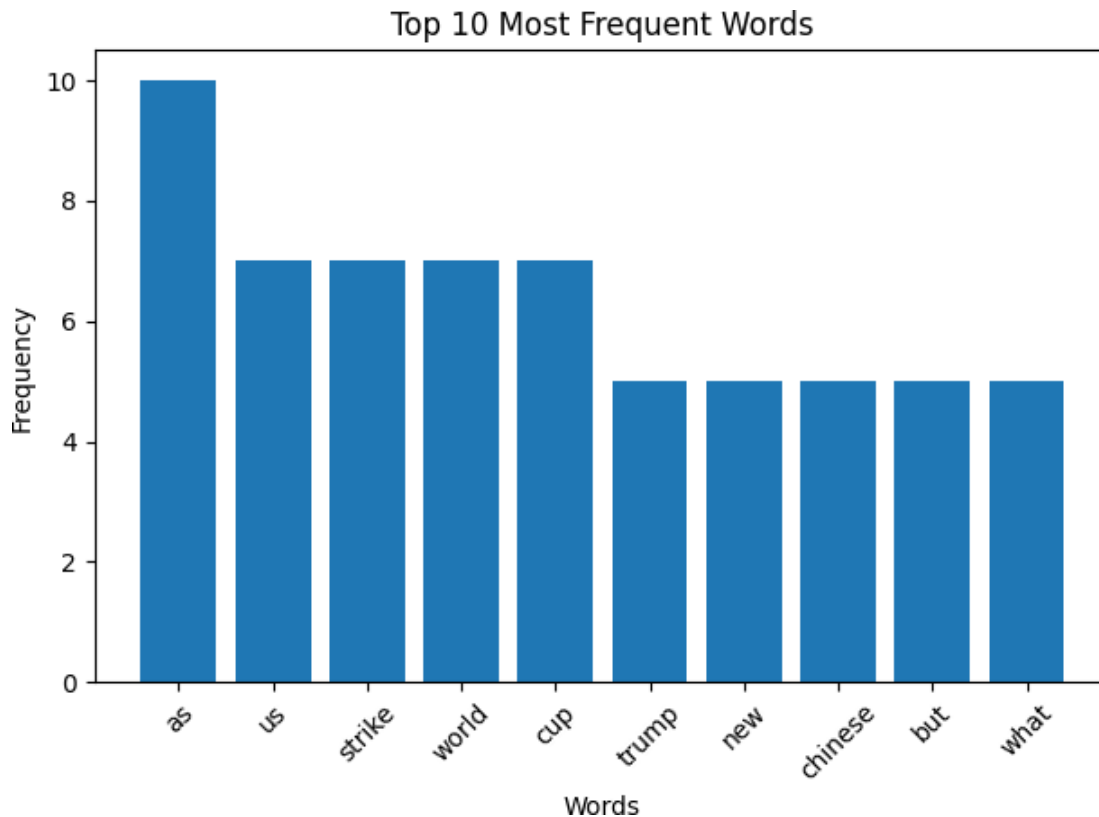
filtered_words = [word for word in text.split() if word not in stopwords]

word_counts = Counter(filtered_words)

words = [item[0] for item in word_counts.most_common(10)]
counts = [item[1] for item in word_counts.most_common(10)]

plt.bar(words, counts)
plt.xlabel("Words")
plt.ylabel("Frequency")
plt.title("Top 10 Most Frequent Words")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

```

OUTPUT:

Question 5: Generate and display a WordCloud of all cleaned words from the headlines.

Explanation:

- `''.join(filtered_words)` merges all words into a single text string.
- `WordCloud` creates the word cloud.
- `plt.imshow()` displays the image.

PROGRAM :

```
import requests
from bs4 import BeautifulSoup
import re
from wordcloud import WordCloud
import matplotlib.pyplot as plt

url = "https://www.bbc.com/news"

response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})

soup = BeautifulSoup(response.text, "html.parser")

headlines = soup.find_all(["h2", "h3"])

text = " ".join([h.get_text() for h in headlines]).lower()

text = re.sub(r"[^a-z\s]", "", text)
```

```
stopwords = {
    "the","and","to","of","in","for","on",
    "at","with","a","an","by","is","are",
    "this","that"
}

filtered_words = [word for word in text.split() if word not in stopwords]

print(filtered_words[:20])

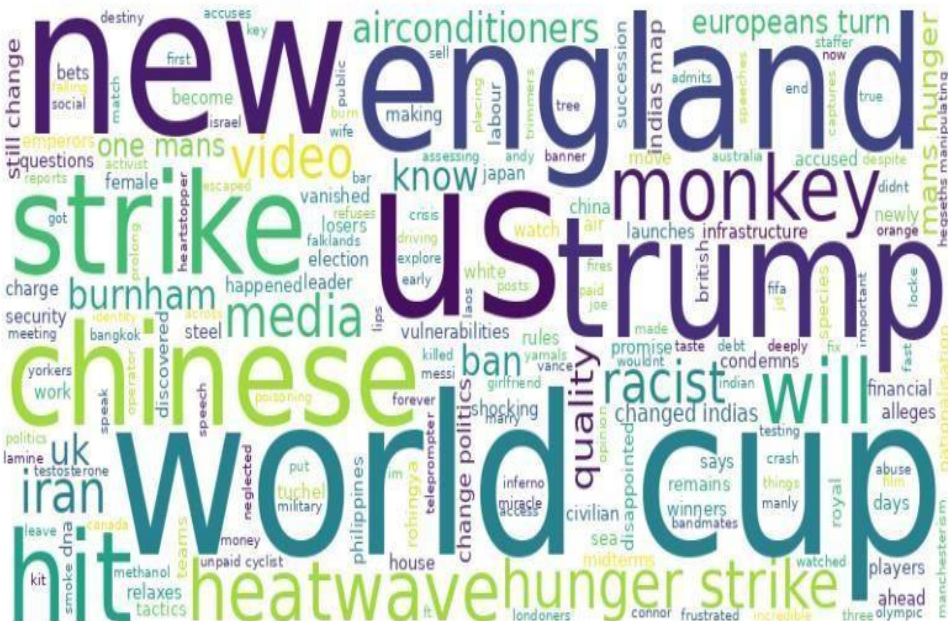
font_path = "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf"

wordcloud = WordCloud(
    font_path=font_path,
    width=800,
    height=400,
    background_color="white"
).generate(" ".join(filtered_words))

plt.figure(figsize=(10,5))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.show()
```

OUTPUT:

['trump', 'alleges', 'shocking', 'vulnerabilities', 'us', 'election', 'security', 'ahead', 'midterms', 'us', 'launches', 'new', 'strikes', 'as', 'iran', 'says', 'civilian', 'infrastructure', 'hit', 'philippines']



EX.NO: 4

DATE:

DATA VISUALIZATION IN DIFFERENT DIMENSION

AIM:

To perform 1D, 2D, and 3D data visualizations using different datasets (Iris, Titanic, Wine, and COVID-19) and analyze the patterns, relationships, distributions, and trends using Python visualization libraries.

Question 1: Iris Dataset – Classic Dataset for 1D, 2D, and 3D Visualization**1D Visualization –**

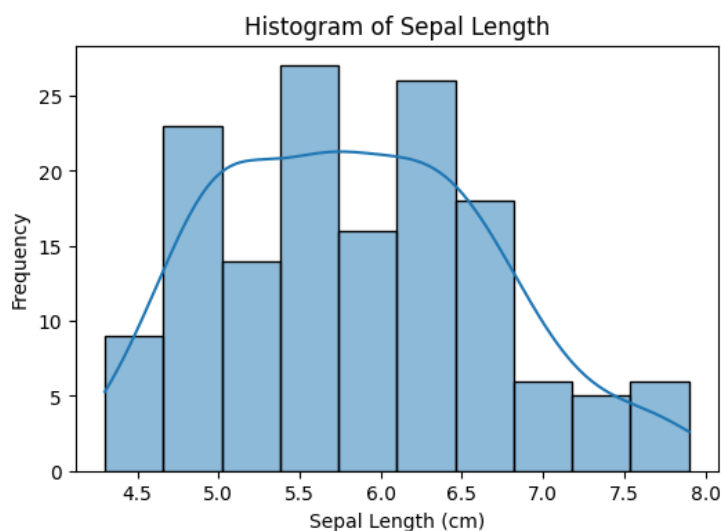
```
[28]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris(as_frame = True)
df = iris.frame
df["species"] = df["target"].map(dict(enumerate(iris.target_names)))
print(df)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

- Plot a histogram for sepal length. How is the distribution shaped?

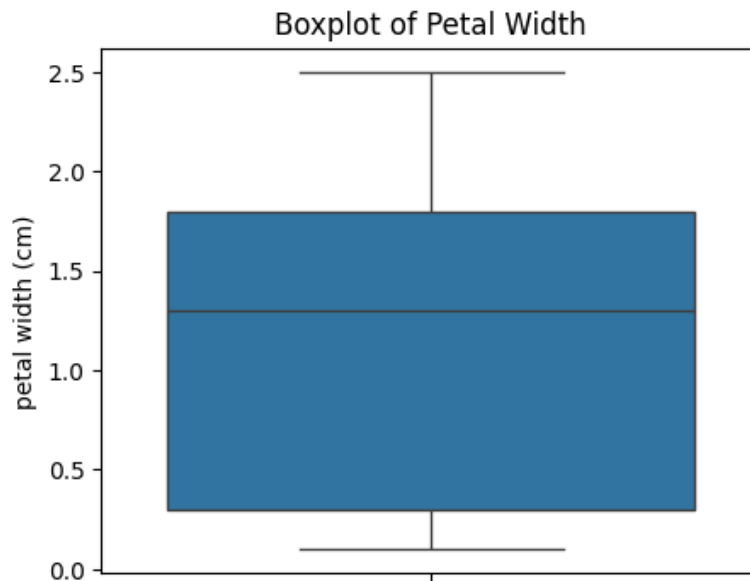
```
plt.figure(figsize=(6,4))
sns.histplot(df["sepal length (cm)"], bins=10, kde=True)

plt.title("Histogram of Sepal Length")
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Frequency")
plt.show()
```



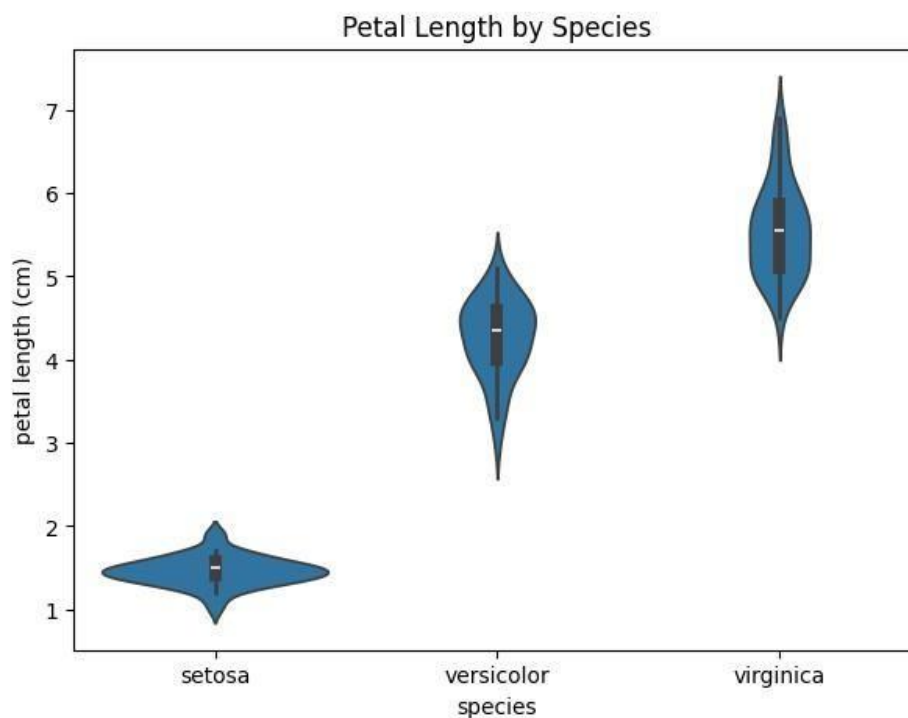
- Draw a boxplot for petal width. Identify outliers if any.

```
[26]: plt.figure(figsize=(5,4))
sns.boxplot(y=df["petal width (cm)"])
plt.title("Boxplot of Petal Width")
plt.show()
```



- Create a violin plot for petal length categorized by species. What insights can you gather?

```
[29]: plt.figure(figsize=(7,5))
sns.violinplot(x="species",y="petal length (cm)",data=df)
plt.title("Petal Length by Species")
plt.show()
```



Insights:

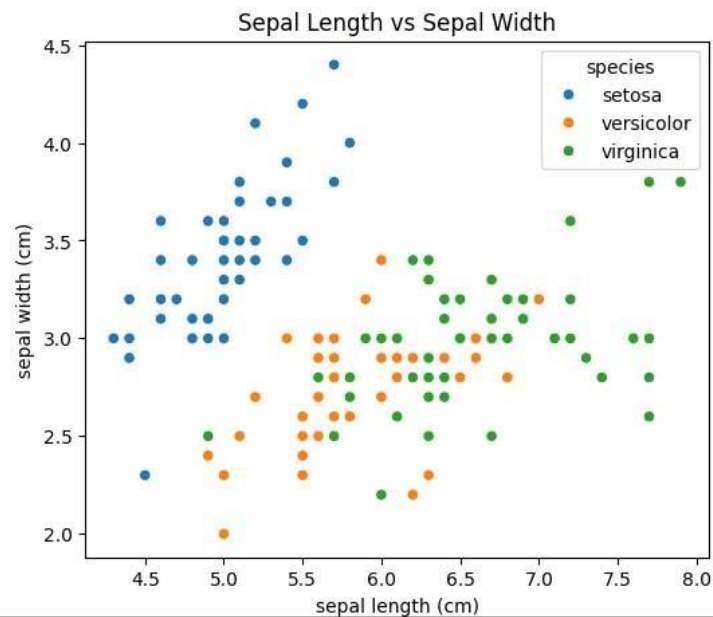
Setosa has the shortest petal length with low variation, while Virginica has the longest petal length with the highest variation.

Versicolor lies between them, and petal length clearly helps distinguish the three Iris species.

2D Visualization -

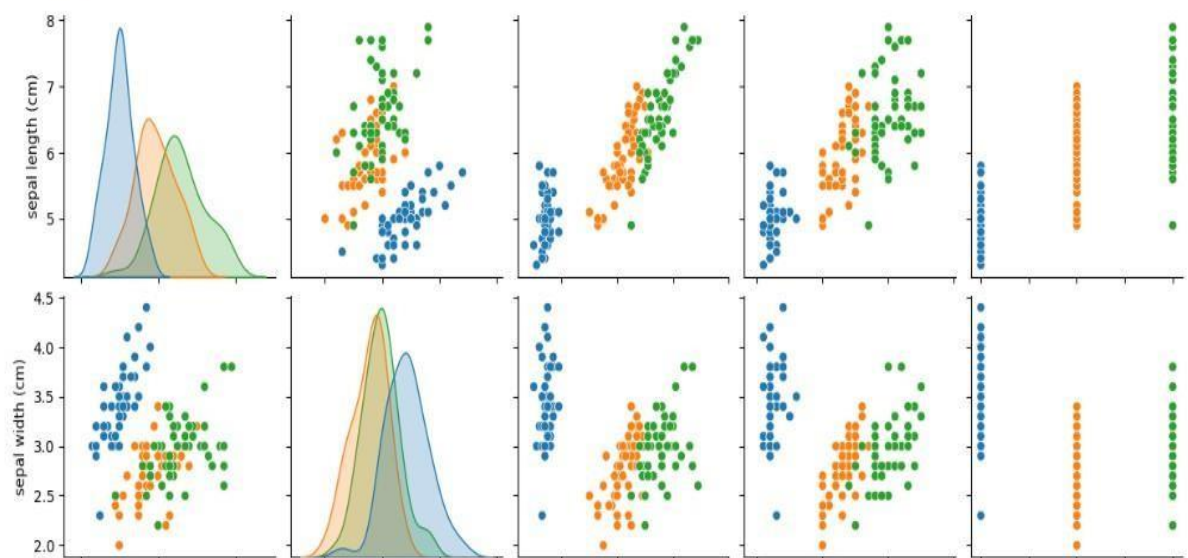
- Generate a scatter plot of sepal length vs sepal width, color-coded by species. What pattern emerges?

```
[30]: plt.figure(figsize=(6,5))
sns.scatterplot(
    x="sepal length (cm)",
    y="sepal width (cm)",
    hue="species",
    data=df
)
plt.title("Sepal Length vs Sepal Width")
plt.show()
```



- Create a pair plot (scatter matrix). How does this help in understanding relationships?

```
[31]: sns.pairplot(df, hue="species")
plt.show()
```



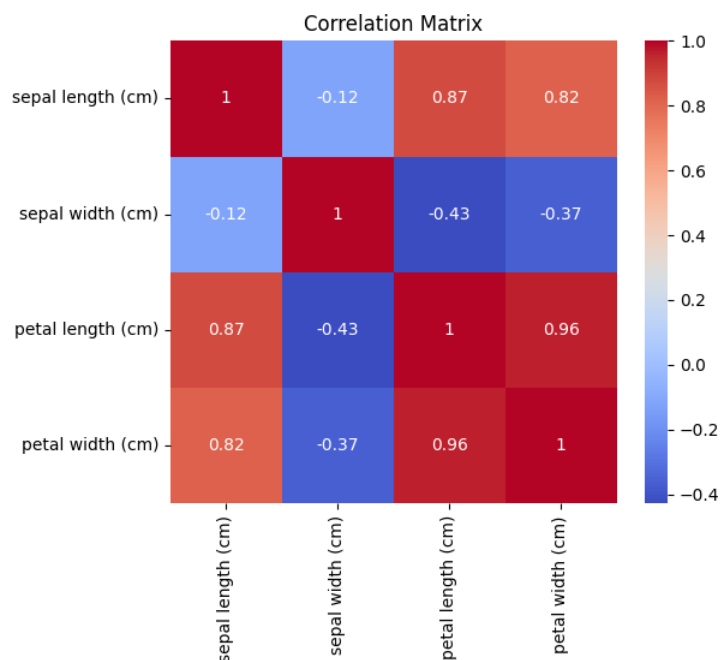
A pair plot (scatter matrix) shows the relationship between every pair of features and the distribution of each feature.

It helps identify correlations, clusters of species, and overlapping features, making it easier to compare all variables at once.

- Plot a heatmap of the correlation matrix. What features are most strongly correlated?

```
[32]: plt.figure(figsize=(6,5))
      corr = df.iloc[:,0:4].corr()
      sns.heatmap(
          corr,
          annot=True,
          cmap="coolwarm"
      )
      plt.title("Correlation Matrix")

      plt.show()
```

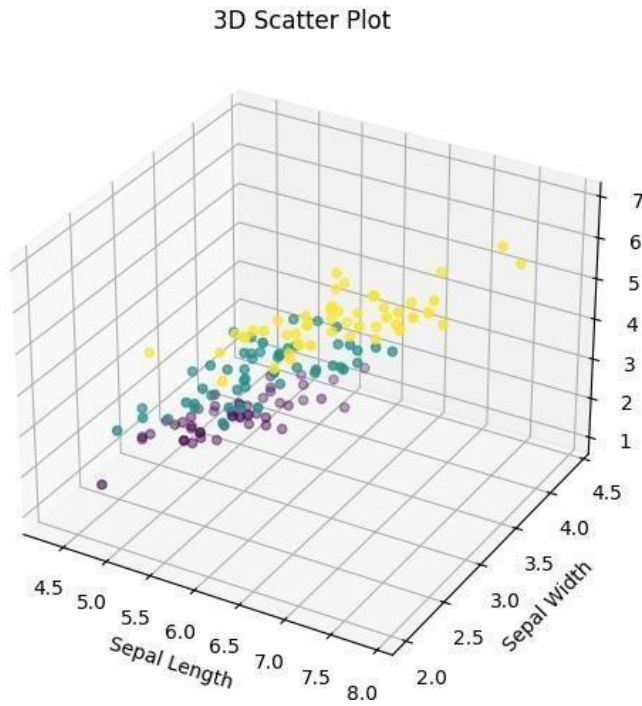


Petal length and petal width are the most strongly positively correlated features, indicating they increase together.

3D Visualization -

- Create a 3D scatter plot using sepal length, sepal width, and petal length. Use species for color.

```
[33]: from mpl_toolkits.mplot3d import Axes3D
      fig = plt.figure(figsize=(8,6))
      ax = fig.add_subplot(111, projection="3d")
      ax.scatter(
          df["sepal length (cm)"],
          df["sepal width (cm)"],
          df["petal length (cm)"],
          c=df["target"],
          cmap="viridis"
      )
      ax.set_xlabel("Sepal Length")
      ax.set_ylabel("Sepal Width")
      ax.set_zlabel("Petal Length")
      plt.title("3D Scatter Plot")
      plt.show()
```



Question 2: Titanic Dataset – Categorical vs Numerical Data

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D
df = sns.load_dataset("titanic")
df = df.dropna(subset=["age", "fare", "pclass", "survived"])

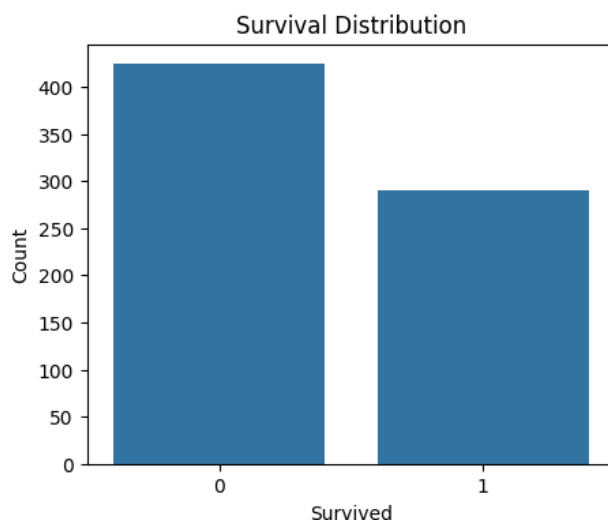
```

- Plot a count plot for the "Survived" column. What is the survival distribution?

```

[40]: plt.figure(figsize=(5,4))
sns.countplot(x="survived", data=df)
plt.title("Survival Distribution")
plt.xlabel("Survived")
plt.ylabel("Count")
plt.show()

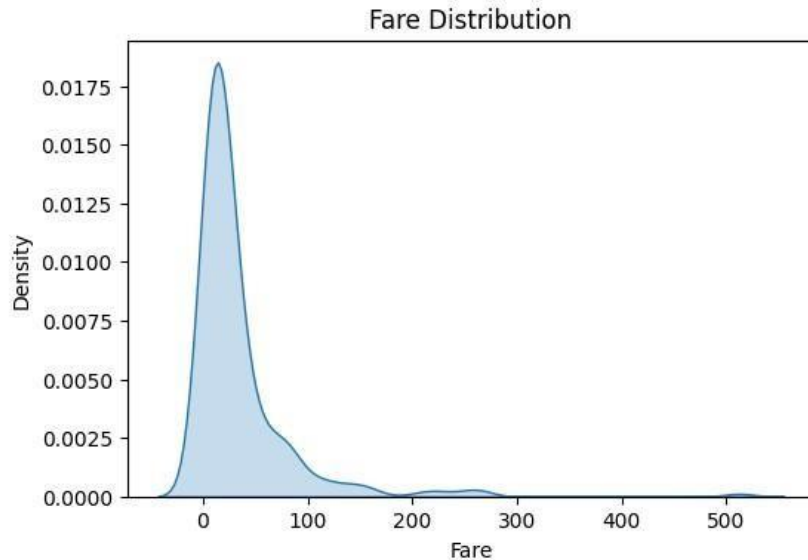
```



The count plot shows that more passengers did not survive (549) than survived (342), indicating a lower survival rate.

- Visualize fare distribution with a KDE plot. Is it skewed?

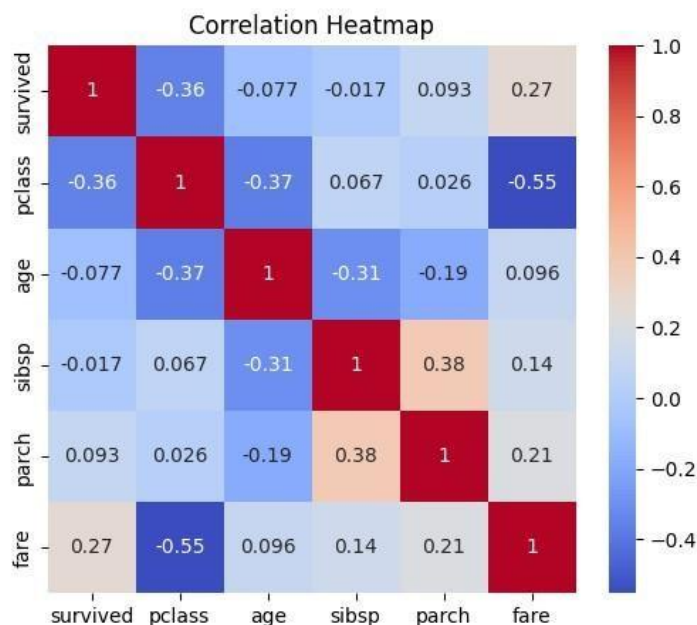
```
[35]: plt.figure(figsize=(6,4))
sns.kdeplot(df["fare"], fill=True)
plt.title("Fare Distribution")
plt.xlabel("Fare")
plt.show()
```



The Fare distribution is positively (right) skewed, with most passengers paying low fares and a few paying very high fares.

- Display a correlation heatmap of numerical features. What variables are correlated?

```
[36]: plt.figure(figsize=(6,5))
corr = df[["survived", "pclass", "age", "sibsp", "parch", "fare"]].corr()
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

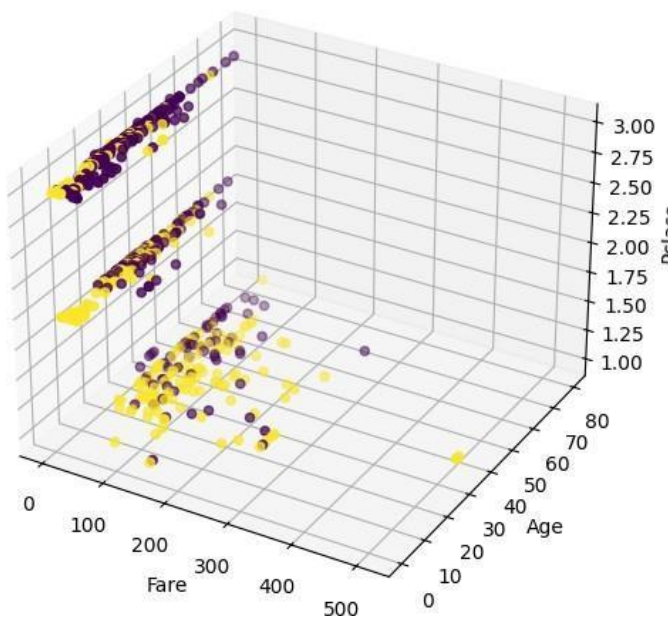


Fare is positively correlated with Survived, while Pclass is negatively correlated with both Fare and Survived.

- Use Fare, Age, and Pclass for a 3D scatter plot. Color code by "Survived".

```
[37]: fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(
    df["fare"],
    df["age"],
    df["pclass"],
    c=df["survived"],
    cmap="viridis"
)
ax.set_xlabel("Fare")
ax.set_ylabel("Age")
ax.set_zlabel("Pclass")
plt.title("3D Scatter Plot")
plt.show()
```

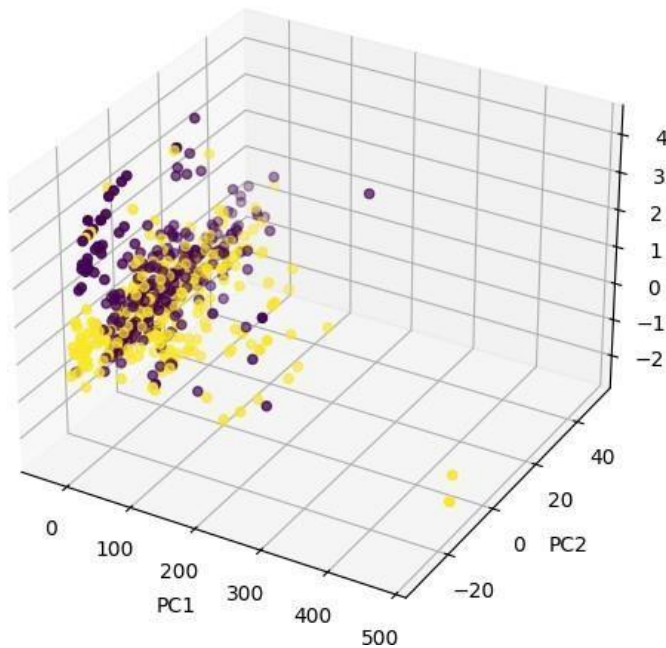
3D Scatter Plot



- Apply PCA and reduce to 3 dimensions. Plot the result in 3D.

```
[38]: features = df[["age", "fare", "pclass", "sibsp", "parch"]]
pca = PCA(n_components=3)
result = pca.fit_transform(features)
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(
    result[:,0],
    result[:,1],
    result[:,2],
    c=df["survived"],
    cmap="viridis"
)
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
plt.title("3D PCA Visualization")
plt.show()
```

3D PCA Visualization

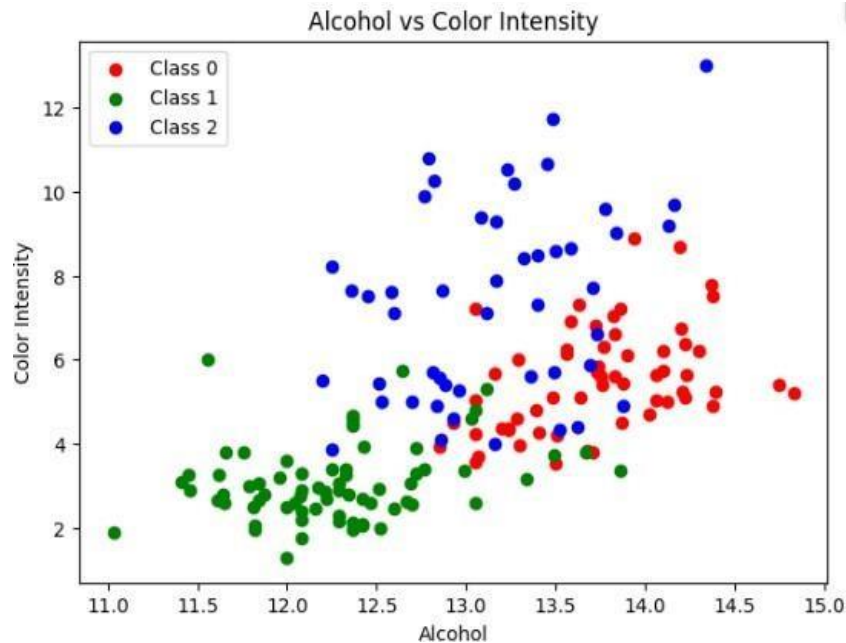


Question 3: Wine Dataset – Multidimensional Visualization

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from pandas.plotting import parallel_coordinates
import numpy as np
wine = load_wine()
df = pd.DataFrame(wine.data, columns=wine.feature_names)
df['Class'] = wine.target
```

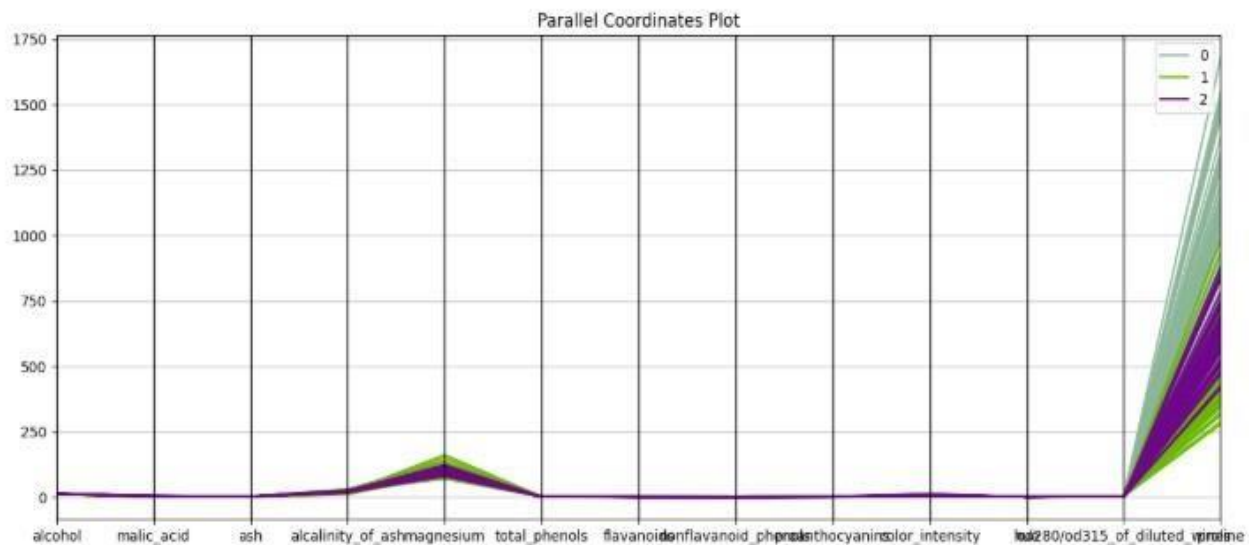
- Create a scatter plot of "Alcohol" vs "Color intensity" with hue as wine class.

```
plt.figure(figsize=(7,5))
colors = ['red', 'green', 'blue']
for i in range(3):
    temp = df[df['Class'] == i]
    plt.scatter(temp['alcohol'],
                temp['color_intensity'],
                color=colors[i],
                label=f'Class {i}')
plt.xlabel("Alcohol")
plt.ylabel("Color Intensity")
plt.title("Alcohol vs Color Intensity")
plt.legend()
plt.show()
```



- Create a parallel coordinates plot for all features colored by wine class.

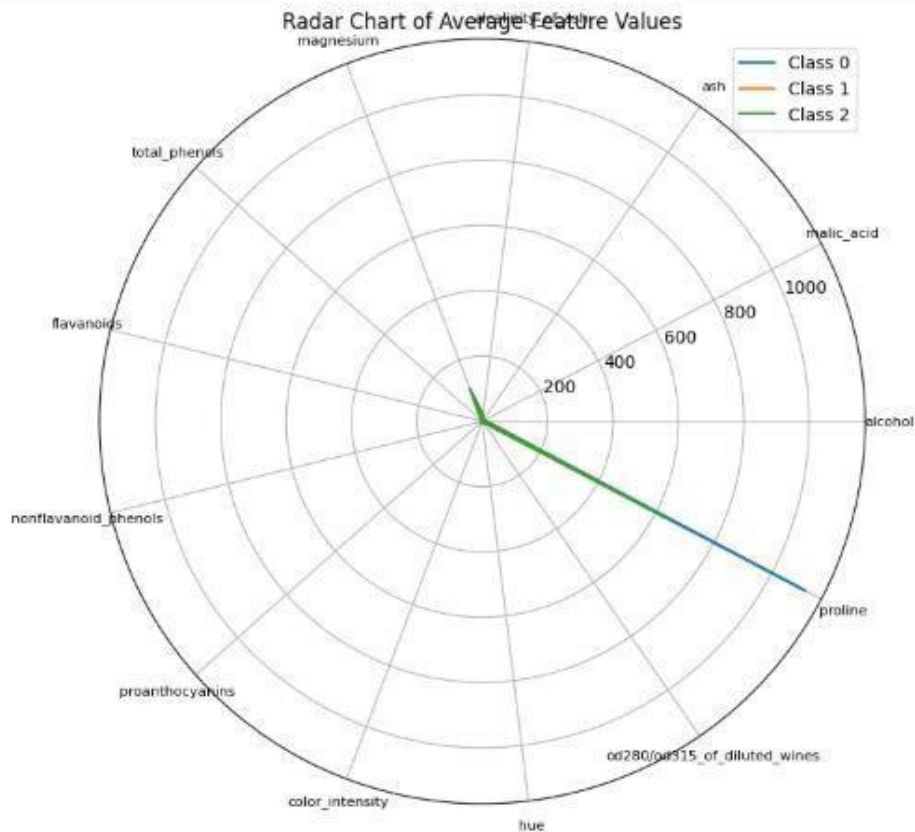
```
plt.figure(figsize=(15,6))
parallel_coordinates(df, 'Class')
plt.title("Parallel Coordinates Plot")
plt.show()
```



- Plot a radar chart for average feature values per class. What do you observe.

```
mean_df = df.groupby('Class').mean()
features = mean_df.columns
N = len(features)
angles = np.linspace(0, 2*np.pi, N, endpoint=False).tolist()
angles += angles[:1]
fig = plt.figure(figsize=(8,8))
ax = plt.subplot(111, polar=True)
for i in range(3):
    values = mean_df.iloc[i].tolist()
    values += values[:1]
    ax.plot(angles, values, label=f'Class {i}')
    ax.fill(angles, values, alpha=0.1)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(features, fontsize=8)
plt.title("Radar Chart of Average Feature Values")
plt.legend()
plt.show()
```



The radar chart shows that each wine class has a unique feature profile based on the average values.

Question 4: Covid Dataset – Map based Visualization

```
import pandas as pd
covid = {
    "Country": ["India", "USA", "Brazil", "UK", "Russia", "France", "Germany", "Italy"],
    "ISO_Code": ["IND", "USA", "BRA", "GBR", "RUS", "FRA", "DEU", "ITA"],
    "Confirmed": [45000000, 103000000, 37000000, 24500000, 23000000, 39000000, 38000000, 38000000],
    "Recovered": [44400000, 100000000, 36000000, 24000000, 22400000, 38500000, 37500000, 37500000],
    "Deaths": [530000, 1120000, 700000, 220000, 400000, 170000, 175000, 190000]
}
df = pd.DataFrame(covid)
print(df.head())
```

	Country	ISO_Code	Confirmed	Recovered	Deaths
0	India	IND	45000000	44400000	530000
1	USA	USA	103000000	100000000	1120000
2	Brazil	BRA	37000000	36000000	700000

- Using a dataset containing COVID-19 cases and ISO country codes, create a choropleth map to visualize the number of confirmed cases globally.

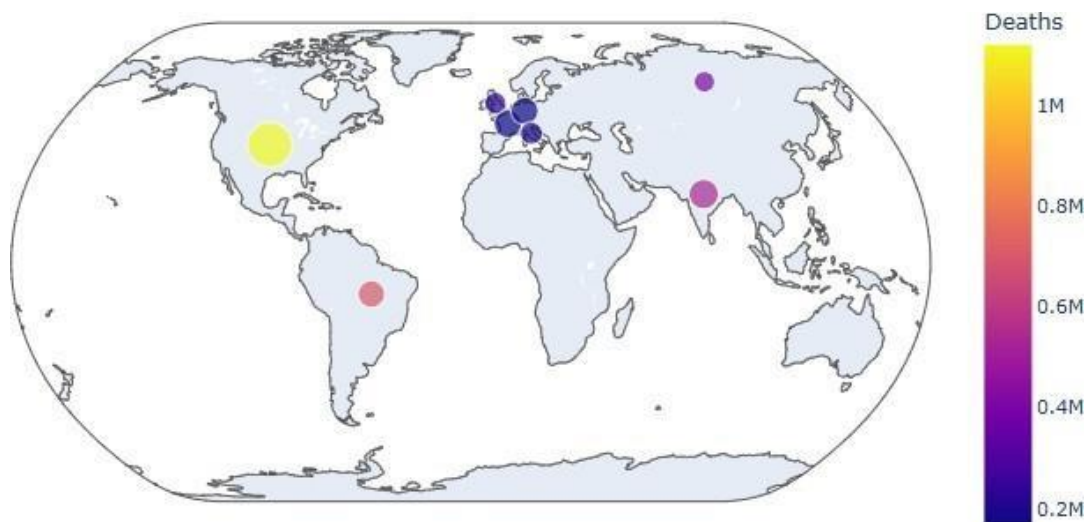
```
import plotly.express as px
fig = px.choropleth(
    df,
    locations="ISO_Code",
    color="Confirmed",
    hover_name="Country",
    color_continuous_scale="Reds",
    title="Global COVID-19 Confirmed Cases"
)
fig.show()
```

Global COVID-19 Confirmed Cases



- Build an interactive map using Plotly to visualize confirmed, recovered, and death counts for each country. What regions show higher mortality rates?

```
fig = px.scatter_geo(
    df,
    locations="ISO_Code",
    hover_name="Country",
    size="Confirmed",
    color="Deaths",
    projection="natural earth",
    title="COVID-19 Confirmed, Recovered and Death Cases"
)
fig.show()
```



The interactive map shows that countries with higher death counts have higher mortality, with the USA and Brazil showing comparatively higher mortality rates.

RESULT:

Thus the required data visualizations were successfully created, and the datasets were effectively analyzed using different visualization techniques.

EX.NO: 5

DATE:

EXPLORING AND MANAGING HADOOP USING CLUDERA CLI AND WEBUI

AIM:

To explore the cludera manager WebUI and to view, monitor and analyze the status of datanode in.

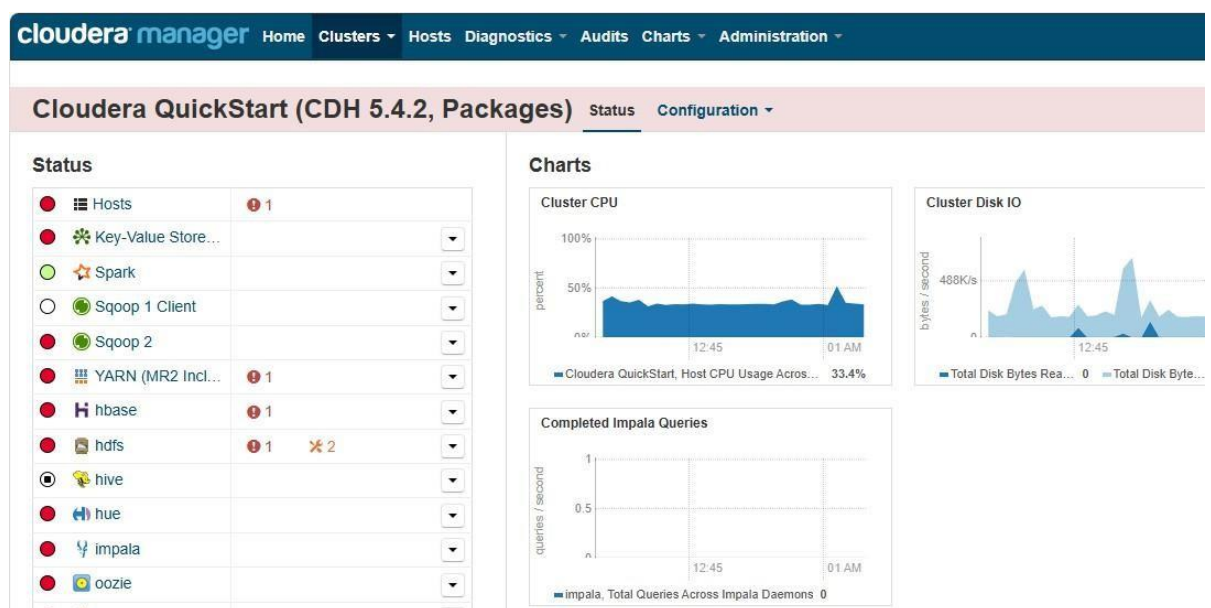
QUESTIONS:

1. Ensure Required Services Are Running.

i) On the Cludera Manager home page, you will see a section titled "Clusters".



ii) Click on your cluster (e.g., QuickStart Cluster).



iii) You'll see a list of services such as: HDFS,YARN (MapReduce2),Hue,Hive

cloudera manager Home Clusters Hosts

Cloudera QuickStart (CDH 5.4.2, P

Status

Hosts	1
Key-Value Store...	
Spark	
Sqoop 1 Client	
Sqoop 2	
YARN (MR2 Incl...	1
hbase	1
hdfs	1 2
hive	
hue	
impala	
oozie	

iv) Make sure these services are running.

Status

Hosts	1
Key-Value Store...	
Spark	
Sqoop 1 Client	
Sqoop 2	
YARN (MR2 Incl...	1
hbase	1
hdfs	1 2
hive	
hue	
impala	
oozie	
solr	
zookeeper	

hue Actions

- Start
- Stop
- Restart
- Rename
- Delete

2. Monitoring Services

i) Click on HDFS Service from the cluster dashboard.

cloudera manager Home Clusters Hosts Diagnostics Audits Charts Administration

Cloudera QuickStart »

hdfs Status Instances Configuration Commands Audits Charts

Role Instances Migrate Role

1 validation warning.
Service hdfs has 1 DataNode. Cloudera suggests at least 3 DataNodes for HDFS.

Filters

SEARCH

STATUS

All 4

None 1

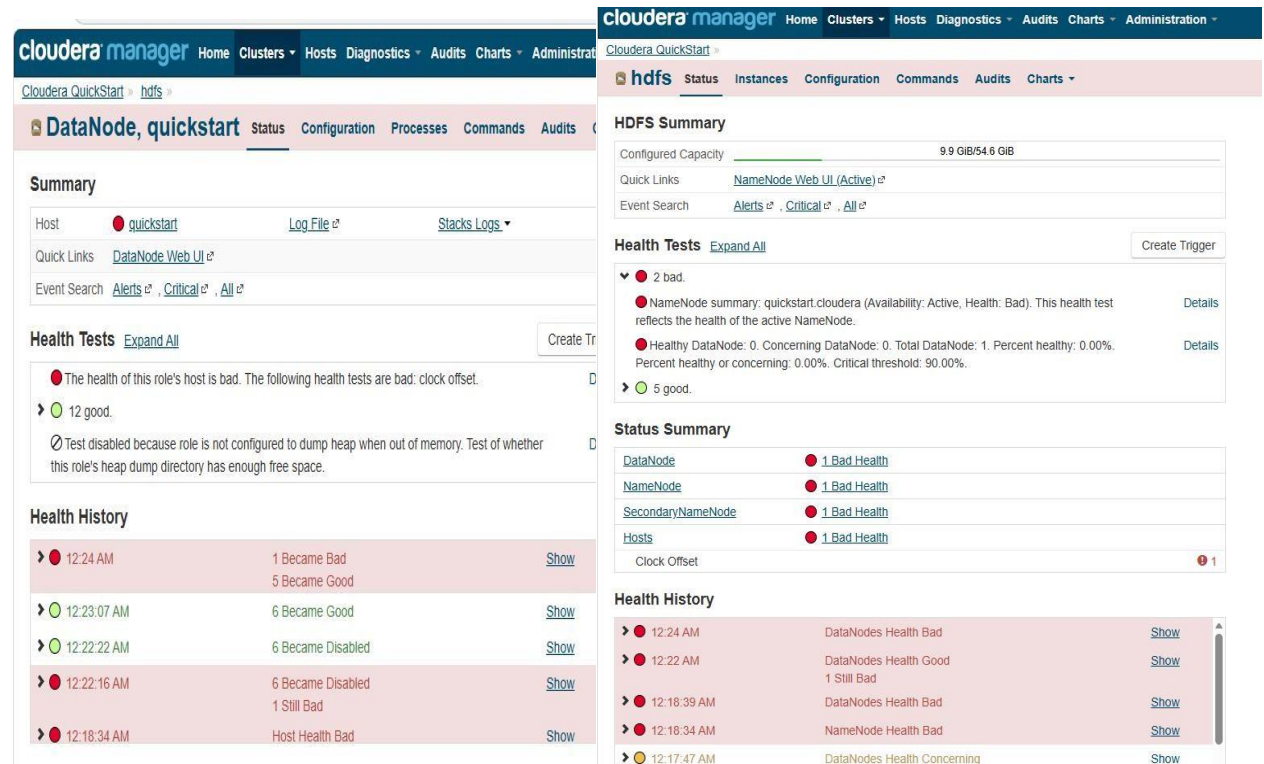
Bad Health 3

DECOMMISSIONED

Actions for Selected

☐	Role Type	State	Host
☐	Balancer	N/A	quickstart.cloudera
☐	DataNode	Started	quickstart.cloudera
☐	NameNode (Active)	Started	quickstart.cloudera
☐	SecondaryNameNode	Started	quickstart.cloudera

ii) Explore the Instances tab: Identify the NameNode and DataNodes.



iii) Go to the Charts tab: Monitor HDFS capacity, IO activity, and file operations.

Charts



3. View Data Nodes.

i) Click on the Instances tab.

cloudera manager Home Clusters Hosts Diagnostics Audits Charts Administration

Cloudera QuickStart »

hdfs Status **Instances** Configuration Commands Audits Charts

Role Instances Migrate Roles

1 validation warning.
Service hdfs has 1 DataNode. Cloudera suggests at least 3 DataNodes for HDFS.

Filters Actions for Selected

SEARCH

ii) You will see the NameNode and a list of DataNodes.

☐	Role Type	State	Host	Role Group
☐	Balancer	N/A	quickstart.cloudera	Balancer Default Group
☐	DataNode	Started	quickstart.cloudera	DataNode Default Group
☐	NameNode (Active)	Started	quickstart.cloudera	NameNode Default Group
☐	SecondaryNameNode	Started	quickstart.cloudera	SecondaryNameNode Default G

iii) Each DataNode entry shows: Host name, Role type (DataNode), Health status (Good / Bad / Concerning) and Uptime

cloudera manager Home Clusters Hosts Diagnostics Audits Charts Administration

Cloudera QuickStart » hdfs »

DataNode, quickstart Status Configuration Processes Commands Audits Ch

Summary

Host quickstart [Log File](#) [Stacks Logs](#)

Quick Links [DataNode Web UI](#)

Event Search [Alerts](#), [Critical](#), [All](#)

Health Tests [Expand All](#) Create Trigg

● The health of this role's host is bad. The following health tests are bad: clock offset. Det

● 12 good.

○ Test disabled because role is not configured to dump heap when out of memory. Test of whether this role's heap dump directory has enough free space. Det

Health History

● 12:24 AM	1 Became Bad 5 Became Good	Show
● 12:23:07 AM	6 Became Good	Show
● 12:22:22 AM	6 Became Disabled	Show
● 12:22:16 AM	6 Became Disabled 1 Still Bad	Show
● 12:18:34 AM	Host Health Bad	Show

iv) Click on a specific DataNode role \(\rightarrow\) You can check:

- CPU/Memory usage
- Disk space used/free
- Logs (helpful for debugging)

Actions for Selected ▾ Columns: 9 Selected ▾

Status	Name	IP	Roles	Last Heartbeat	Load Average	Disk Usage	Physical Memory	Swap Space
	quickstart.cloudera	192.168.10.5	29 Role(s)	14.63s ago	0.21 0.56 0.80	10 GiB / 74 GiB	6.5 GiB / 18.9 GiB	0 B / 8 GiB

Display 25 ▾ Entries

cloudera manager Home Clusters ▾ Hosts Diagnostics ▾ Audits Charts ▾ Administration ▾

Logs

error| [Additional Settings](#)

▸ Select Sources Hosts Minimum Log Level

Host	Log Level	Time	Source	Message
quickstart.cloudera	WARN	July 29, 2026 1:17 AM	DfsClientShmManager	EndpointShmManager(DatanodeInfoWithStorage[192.168.10.5:50010,DS-dcf35ebd-2f86-4fe6-8001-67f8db5e89d4,DISK], parent=ShortCircuitIOException calling shutdown(SHUT_RDWR) java.nio.channels.ClosedChannelException at org.apache.hadoop.util.CloseableReferenceCount.reference(CloseableReferenceCount.java:57) at org.apache.hadoop.net.unix.DomainSocket.shutdown(DomainSocket.java:387) at org.apache.hadoop.hdfs.shortcircuit.DfsClientShmManager\$EndpointShmManager.shutdown(DfsClientShmManager.java:379) at org.apache.hadoop.hdfs.shortcircuit.ShortCircuitCache\$SlotReleaser.run(ShortCircuitCache.java:223) at java.util.concurrent.Executors\$RunnableAdapter.call(Executors.java:471) at java.util.concurrent.FutureTask.run(FutureTask.java:262) at java.util.concurrent.ScheduledThreadPoolExecutor\$ScheduledFutureTask.access\$201(ScheduledThreadPoolExecutor.java:17) at java.util.concurrent.ScheduledThreadPoolExecutor\$ScheduledFutureTask.run(ScheduledThreadPoolExecutor.java:292) at java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1145) at java.util.concurrent.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:615) at java.lang.Thread.run(Thread.java:745) View Log File
quickstart.cloudera	ERROR	July 29, 2026 1:17 AM	ShortCircuitCache	ShortCircuitCache(0x1ffc4054): failed to release short-circuit shared memory slot Slot(slotIdx=1, shm=DfsClientShm(2ccc14e28d2 ReleaseShortCircuitAccessRequestProto to /var/run/hdfs-sockets/dn. Closing shared memory segment. java.io.IOException: ERROR_INVALID: there is no shared memory segment registered with shmId 2ccc14e28d2a3c4465d4646221e14f44 at org.apache.hadoop.hdfs.shortcircuit.ShortCircuitCache\$SlotReleaser.run(ShortCircuitCache.java:208) at java.util.concurrent.Executors\$RunnableAdapter.call(Executors.java:471) at java.util.concurrent.FutureTask.run(FutureTask.java:262)

4. Access Hue to Use File Browser and Submit Jobs

i) <http://192.168.18.152:8888>



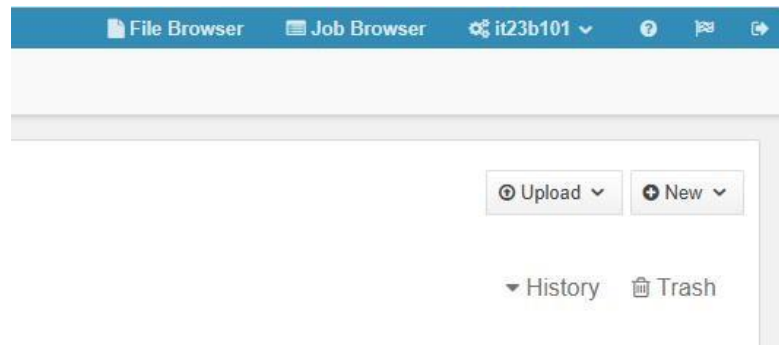
Sign in to continue to Hue

ii) Login, Pwd

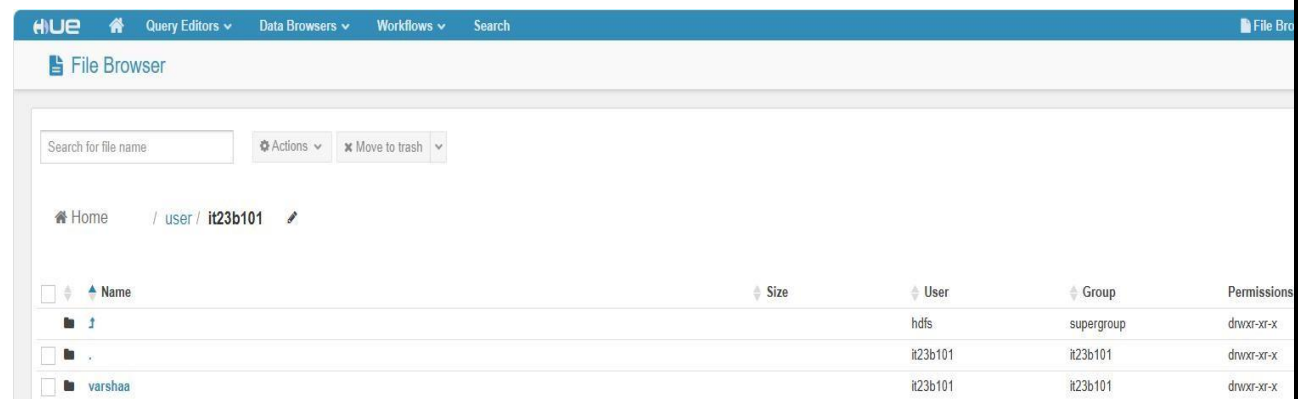
Login

☐ Remember me on this computer.

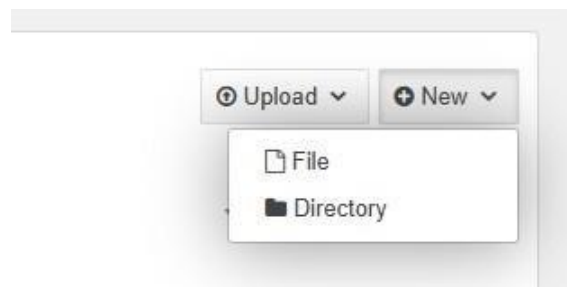
iii) File browser – Upload a file to HDFS



iv) Browse HDFS folders like /user/cloudera



v) Create new directories



5. Cluster Health & Logs

i) Click Clusters (\(\rightarrow\) All Hosts.

- o Note CPU, Memory, and Disk usage for each host.

Actions for Selected									
Columns: 9 Selected									
	Status	Name	IP	Roles	Last Heartbeat	Load Average	Disk Usage	Physical Memory	Swap Space
		quickstart.cloudera	192.168.10.5	29 Role(s)	14.63s ago	0.21 0.66 0.80	10 GiB / 74 GiB	6.5 GiB / 18.9 GiB	0 B / 8 GiB
Display 25 Entries									

ii) Open Diagnostics (\(\rightarrow\) Log Files.

- o Search for "error" in HDFS logs.

The screenshot shows the Cloudera Administration console. The top navigation bar includes Home, Clusters, Hosts, Diagnostics, Audits, Charts, and Administration. The Diagnostics dropdown menu is open, showing Events, Logs, and Server Log. Below the navigation bar, there are sections for 'Start)' and 'Logs'. The 'Logs' section has a search bar with the text 'error', a 'Q Search' button, and a link to 'Additional Settings'. Below the search bar, there are tabs for 'Select Sources' and 'Hosts', and a 'Minimum Log Level' dropdown set to 'INFO'. The log entry table shows a warning message from 'DfsClientShmManager' on 'July 29, 2026 1:17 AM'. The message details a 'java.nio.channels.ClosedChannelException' and lists the stack trace.

Host	Log Level	Time	Source	Message
quickstart.cloudera	WARN	July 29, 2026 1:17 AM	DfsClientShmManager	EndpointShmManager(DatanodeInfoWithStorage[192.168.10.5:50010,DS-dcf35ebd-2f86-4fe6-8001-67f8db5e89d4,DISK], parent=ShortCircuitIOException calling shutdown(SHUT_RDWR) java.nio.channels.ClosedChannelException at org.apache.hadoop.util.CloseableReferenceCount.reference(CloseableReferenceCount.java:57) at org.apache.hadoop.net.unix.DomainSocket.shutdown(DomainSocket.java:387) at org.apache.hadoop.hdfs.shortcircuit.DfsClientShmManager\$EndpointShmManager.shutdown(DfsClientShmManager.java:379) at org.apache.hadoop.hdfs.shortcircuit.ShortCircuitCache\$SlotReleaser.run(ShortCircuitCache.java:223) at java.util.concurrent.Executors\$RunnableAdapter.call(Executors.java:471) at java.util.concurrent.FutureTask.run(FutureTask.java:262) at java.util.concurrent.ScheduledThreadPoolExecutor\$ScheduledFutureTask.access\$201(ScheduledThreadPoolExecutor.java:171) at java.util.concurrent.ScheduledThreadPoolExecutor\$ScheduledFutureTask.run(ScheduledThreadPoolExecutor.java:292) at java.util.concurrent.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:1145) at java.util.concurrent.ThreadPoolExecutor\$Worker.run(ThreadPoolExecutor.java:615) at java.lang.Thread.run(Thread.java:745)

6. Run HDFS Commands in cloudera CLI

i) hdfs dfs -ls

```
[it23106@quickstart ~]$ hdfs dfs -ls
Found 1 items
drwxr-xr-x  - it23106 it23106          0 2026-07-29 01:24 yatheeshram
[it23106@quickstart ~]$
```

ii) hdfs dfs -mkdir t1

```
[it23106@quickstart ~]$ cat dfs -vecdir 21
[it23106@quickstart ~]$ cat fd
chmod: dfs: Is a hdfs file or directory
```

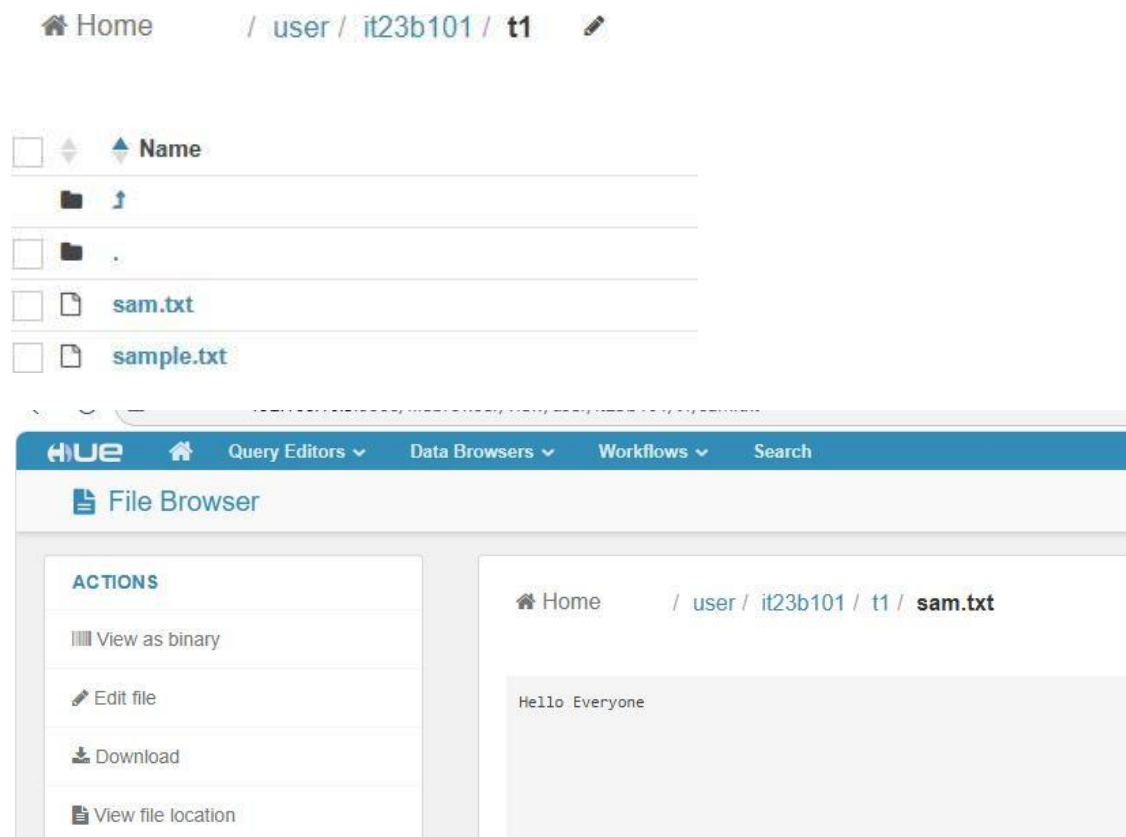
iii) Create a file named sample.txt using cat > sample.txt. Type content and Press cntrl c to save and quit

```
[it23106@quickstart ~]$ cat > sample.txt
Hello !! Good Evening
^C
[it23106@quickstart ~]$
```

iv) `hdfs dfs -put sample.txt t1/`

```
[it23106@quickstart ~]$ hdfs dfs -put sample.txt t1/
[it23106@quickstart ~]$ cd t1
-bash: cd: t1: No such file or directory
[it23106@quickstart ~]$ cat > sam.txt
Hello Everyone
^C
[it23106@quickstart ~]$
[it23106@quickstart ~]$ hdfs dfs -put sam.txt t1/
[it23106@quickstart ~]$
```

v) Refresh the huewebui to see the created contents in file browser.



RESULT:

Thus, the cloudera manager WebUI has been explored successfully.

EX.NO: 6**DATE:**

**DEMONSTRATE THE USE OF HADOOP MAP
REDUCE USING JAVA**

AIM:

To develop a word count map reduce program using java and run the job in hadoop.

PROGRAM:**WordCountDriver -**

```
package wordcount;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCountDriver {

    public static void main(String[] args) throws Exception
    {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Word Count");
        job.setJarByClass(WordCountDriver.class);
        job.setMapperClass(WordCountMapper.class);
        job.setReducerClass(WordCountReducer.class);
        job.setCombinerClass(WordCountReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

WordCountReducer -

```
package wordcount;

import java.io.IOException;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
```

```

import org.apache.hadoop.mapreduce.Reducer;
public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable>
{
    private IntWritable result = new IntWritable();
    public void reduce(Text key, Iterable<IntWritable> values,
        Context context) throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable value : values)
            { sum += value.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}

```

WordCountMapper -

```

package wordcount;
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.mapreduce.Mapper;
public class WordCountMapper extends Mapper<LongWritable, Text, Text, IntWritable>
{
    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        StringTokenizer tokenizer = new StringTokenizer(value.toString());
        while (tokenizer.hasMoreTokens()) {
            word.set(tokenizer.nextToken());
            context.write(word, one);
        }
    }
}

```

OUTPUT:

```
[it23106@quickstart ~]$ mkdir wordcount
[it23106@quickstart ~]$ cd wordcount
[it23106@quickstart wordcount]$ vi WordCountMapper.java
[it23106@quickstart wordcount]$ vi WordCountReducer.java
[it23106@quickstart wordcount]$ vi WordCountDriver.java
[it23106@quickstart wordcount]$ mkdir -p classes
```

```
[it23106@quickstart wordcount]$ javac -classpath ".:lib/*" -d classes WordCountMapper.java WordCountReducer.java
[it23106@quickstart wordcount]$ jar -cvf wordcount.jar -C classes/ .
added manifest
adding: wordcount/(in = 0) (out= 0)(stored 0%)
adding: wordcount/WordCountDriver.class(in = 1489) (out= 711)(deflated 48%)
adding: wordcount/WordCountReducer.class(in = 1710) (out= 723)(deflated 57%)
adding: wordcount/WordCountMapper.class(in = 1411) (out= 759)(deflated 46%)
[it23106@quickstart wordcount]$ hdfs dfs -mkdir -p /user/it23106/input
[it23106@quickstart wordcount]$ echo "hello hadoop hello mapreduce" > input.txt
[it23106@quickstart wordcount]$ hdfs dfs -put input.txt /user/it23106/input
```

```
[it23b101@quickstart wordcount]$ hadoop jar wordcount.jar wordcount.WordCountDriver /user/it23b101/input /user/it23b101/output
26/07/29 02:16:05 INFO client.RMProxy: Connecting to ResourceManager at quickstart.cloudera/192.168.10.5:8032
26/07/29 02:16:06 WARN mapreduce.JobSubmitter: Hadoop command-line option parsing not performed. Implement the Tool interface a
26/07/29 02:16:06 INFO input.FileInputFormat: Total input paths to process : 1
26/07/29 02:16:06 INFO mapreduce.JobSubmitter: number of splits:1
26/07/29 02:16:06 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1785309432360_0013
26/07/29 02:16:06 INFO impl.YarnClientImpl: Submitted application application_1785309432360_0013
26/07/29 02:16:06 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1785309432360
26/07/29 02:16:06 INFO mapreduce.Job: Running job: job_1785309432360_0013
26/07/29 02:16:12 INFO mapreduce.Job: Job job_1785309432360_0013 running in uber mode : false
26/07/29 02:16:12 INFO mapreduce.Job: map 0% reduce 0%
26/07/29 02:16:17 INFO mapreduce.Job: map 100% reduce 0%
26/07/29 02:16:23 INFO mapreduce.Job: map 100% reduce 100%
26/07/29 02:16:23 INFO mapreduce.Job: Job job_1785309432360_0013 completed successfully
26/07/29 02:16:24 INFO mapreduce.Job: Counters: 49
File System Counters
  FILE: Number of bytes read=61
  FILE: Number of bytes written=227731
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=155
  HDFS: Number of bytes written=29
  HDFS: Number of read operations=6
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
```

```
[it23106@quickstart wordcount]$ cat output.txt
Total megabyte seconds taken by all reduce tasks 555366
Map-Reduce Framework
  Map input records=1
  Map output records=4
  Map output bytes=45
  Map output materialized bytes=57
  Input split bytes=126
  Combine input records=4
  Combine output records=3
  Reduce input groups=3
  Reduce shuffle bytes=57
  Reduce input records=3
  Reduce output records=3
  Spilled Records=6
  Shuffled Maps =1
  Failed Shuffles=0
  Merged Map outputs=1
  GC time elapsed (ms)=90
  CPU time spent (ms)=1040
  Physical memory (bytes) snapshot=267575296
  Virtual memory (bytes) snapshot=1460502528
  Total committed heap usage (bytes)=95944704
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=29
File Output Format Counters
  Bytes Written=29
[it23106@quickstart wordcount]$ ls
classes input.txt WordCountDriver.java wordcount.jar WordCountMapper.java WordCountReducer.java
[it23106@quickstart wordcount]$
```

File Browser Query data Manage data

Search for file name Actions Move to trash Upload

Home / user / it23b101

Name	Size	User	Group	Permissions	Date
.		hdfs	supergroup	drwxr-xr-x	July 29, 2026 12:29 AM
.		it23b101	it23b101	drwxr-xr-x	July 29, 2026 02:16 AM
.staging		it23b101	it23b101	drwxr-xr-x	July 29, 2026 02:16 AM
input		it23b101	it23b101	drwxr-xr-x	July 29, 2026 02:12 AM
output		it23b101	it23b101	drwxr-xr-x	July 29, 2026 02:16 AM

File Browser

Search for file name Actions Move to trash

Home / user / it23b101 / output

Name	Size	User	Group	Permissions
.		it23b101	it23b101	drwxr-xr-x
._SUCCESS	0 bytes	it23b101	it23b101	-rw-r--r--
part-r-00000	29 bytes	it23b101	it23b101	-rw-r--r--

HUE Query Editors Data Browsers Workflows Search

File Browser

Home / user / it23b101 / output / part-r-00000

ACTIONS

- View as binary
- Edit file
- Download
- View file location
- Refresh

INFO

Last modified: July 29, 2026 2:16 a.m.

User: it23b101

Group: it23b101

Size: 29 bytes

```

hadoop 1
hello 2
mapreduce 1

```

RESULT:

Thus, the word count map reduce program using java has been developed and successfully run the job in hadoop.

EX.NO:7**DATE:**

IMPLEMENT CRUD OPERATIONS IN A BIG DATA ENVIRONMENT

AIM:

To implement and demonstrate create, read and append operations on files in HDFS using the Hadoop Java API.

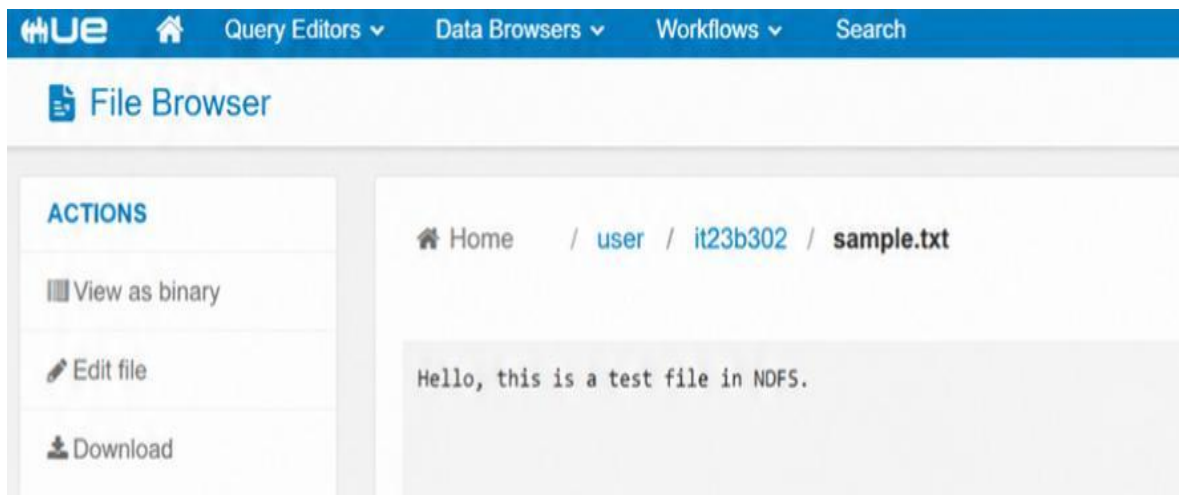
PROGRAM:**Create File:**

```
import java.io.OutputStream;
import java.io.IOException;
import java.net.URI;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.fs.FSDataOutputStream;
public class CreateHDFSFile {
    public static void main(String[] args)
    { try {
        String hdfsUri = "hdfs://quickstart.cloudera:8020";
        String filePath = "/user/it23b095/sample.txt";
        Configuration conf = new Configuration();
        conf.addResource(new Path("/etc/hadoop/conf/core-site.xml"));
        conf.addResource(new Path("/etc/hadoop/conf/hdfs-site.xml"));
        FileSystem fs = FileSystem.get(new URI(hdfsUri), conf);
        Path path = new Path(filePath);
        FSDataOutputStream os = fs.create(path, true);
        String content = "Hello, this is a test file in HDFS.\n";
        os.writeBytes(content);
        os.close();
        System.out.println(" File created successfully in HDFS at: " + filePath);
        fs.close();
    } catch (IOException e)
    { e.printStackTrace();
    } catch (Exception e) {
```

```
        e.printStackTrace();
    }
}
}
```

OUTPUT:

```
[it23b302@quickstart ~]$ vi CreateHDFSFile.java
[it23b302@quickstart ~]$ javac -classpath ``hadoop classpath`` CreateHDFSFile.java
[it23b302@quickstart ~]$ java -cp $(hadoop classpath):. CreateHDFSFile
26/08/05 00:24:28 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
File created succesfully in HDFS at: /user/it23b302/sample.txt
[it23b302@quickstart ~]$
```



Append File:

```

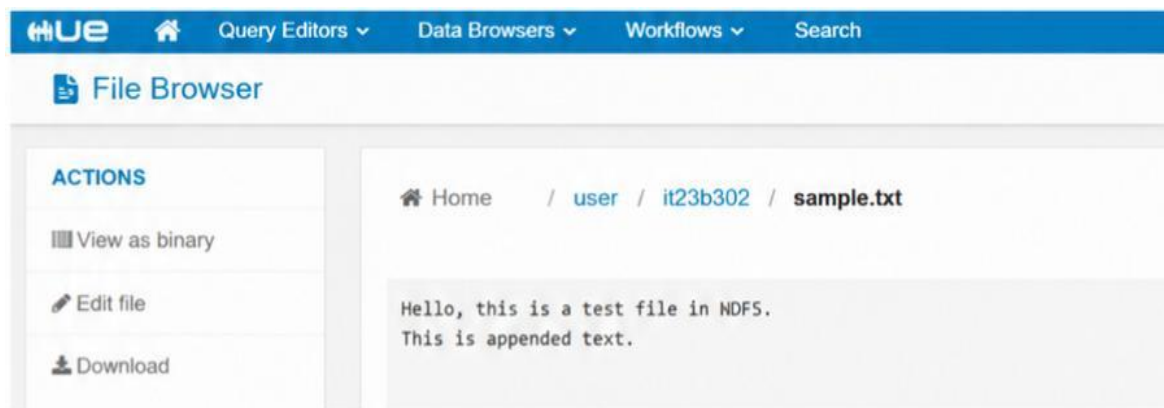
import java.io.OutputStreamWriter;
import java.io.BufferedWriter;
import java.net.URI;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FSDataOutputStream;
import org.apache.hadoop.fs.Path;
public class AppendHDFSFile {
    public static void main(String[] args)
    { try {
        Configuration conf = new Configuration();
        conf.addResource(new Path("/etc/hadoop/conf/core-site.xml"));
        conf.addResource(new Path("/etc/hadoop/conf/hdfs-site.xml"));
        FileSystem fs = FileSystem.get(conf);
        String filePath = "/user/it23b095/sample.txt";
        Path path = new Path(filePath);
        if (fs.exists(path)){

FSDataOutputStream out = fs.append(path);
        BufferedWriter writer = new BufferedWriter(new OutputStreamWriter(out));
        writer.write("This is appended text.\n");
        writer.close();
        System.out.println("Successfully appended to file: " + filePath);
        } else {
        System.out.println("File does not exist: " + filePath);
        }
        fs.close();
    } catch (Exception e)
    { e.printStackTrace();
    }
}
}

```

OUTPUT:

```
[it23b302@quickstart ~]$ vi AppendHDFSFile.java
[it23b302@quickstart ~]$ javac -classpath ``hadoop classpath` AppendHDFSFile.java
[it23b302@quickstart ~]$ java -cp $(hadoop classpath):. AppendHDFSFile
26/08/05 00:40:22 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
Successfully appended to file: /user/it23b302/sample.txt
[it23b302@quickstart ~]$
```

**Read File:**

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URI;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FSDataInputStream;
import org.apache.hadoop.fs.Path;
```

```

public class ReadHDFSFile {
    public static void main(String[] args)
    { try {
        Configuration conf = new Configuration();
        conf.addResource(new Path("/etc/hadoop/conf/core-site.xml"));
        conf.addResource(new Path("/etc/hadoop/conf/hdfs-site.xml"));
        FileSystem fs = FileSystem.get(conf);
        String filePath = "/user/it23b095/sample.txt";
        Path path = new Path(filePath);
        FSDataInputStream inputStream = fs.open(path);
        BufferedReader reader = new BufferedReader(new InputStreamReader(inputStream));
        String line;
        System.out.println("Contents of HDFS file " + filePath + ":");
        while ((line = reader.readLine()) != null) {
            System.out.println(line);
        }
        reader.close();
        fs.close();
    } catch (Exception e)
    { e.printStackTrace();
    }
}

```

OUTPUT:

```

[it23b302@quickstart ~]$ vi ReadHDFSFile.java
[it23b302@quickstart ~]$ javac -classpath `hadoop classpath` ReadHDFSFile.java
[it23b302@quickstart ~]$ java -cp $(hadoop classpath):. ReadHDFSFile
26/08/05 01:06:54 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.
Contents of HDFS file /user/it23b302/sample.txt:
Hello, this is a test file in HDFS.
This is appended text.
[it23b302@quickstart ~]$ █

```

RESULT:

Thus, the CRUD operation were performed on files in HDFS using Hadoop Java API.

EX NO:**DATE:****USE OF JAVA API'S TO INTERACT WITH HADOOP****AIM:**

To use the Hadoop Java API to interact with HDFS and display key cluster and file system information, including Name Node URI, DFS Capacity used and remaining space.

PROGRAM:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.hdfs.DistributedFileSystem;
import org.apache.hadoop.hdfs.protocol.DatanodeInfo;
import java.net.URI;
public class HDFSNodeInfo {
    public static void main(String[] args)
    { try {
        Configuration conf = new Configuration();
        conf.addResource(new org.apache.hadoop.fs.Path("/etc/hadoop/conf/core-site.xml"));
        conf.addResource(new org.apache.hadoop.fs.Path("/etc/hadoop/conf/hdfs-site.xml"));
        FileSystem fs = FileSystem.get(conf);
        if (fs instanceof DistributedFileSystem)
        { DistributedFileSystem dfs = (DistributedFileSystem) fs;
            System.out.println("NameNode URI: " + dfs.getUri());
            System.out.println("DFS Capacity: " + fs.getStatus().getCapacity());
            System.out.println("DFS Used: " + fs.getStatus().getUsed());
            System.out.println("DFS Remaining: " + fs.getStatus().getRemaining());
        }
        fs.close();
    } catch (Exception e)
    { e.printStackTrace();
    }
}}
```

OUTPUT:

```
[it23b302@quickstart ~]$ vi HDFSNodeInfo.java
[it23b302@quickstart ~]$ javac -classpath `hadoop classpath` HDFSNodeInfo.java
[it23b302@quickstart ~]$ java -cp $(hadoop classpath):.. HDFSNodeInfo
26/08/05 00:35:37 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform.. using built-in-java classes
where applicable
NameNode URI: hdfs://quickstart.cloudera:8020
DFS Capacity: 58613309440
DFS Used: 397176835
DFS Remaining: 47107325952
```

2 / 2

RESULT:

Thus, Java API was used to interact with Hadoop.

EX.NO:

DATE:

IMPLEMENTATION OF DATA MINING PROCESS USING WEKA TOOL

AIM:

To perform data mining process using WEKA tool.

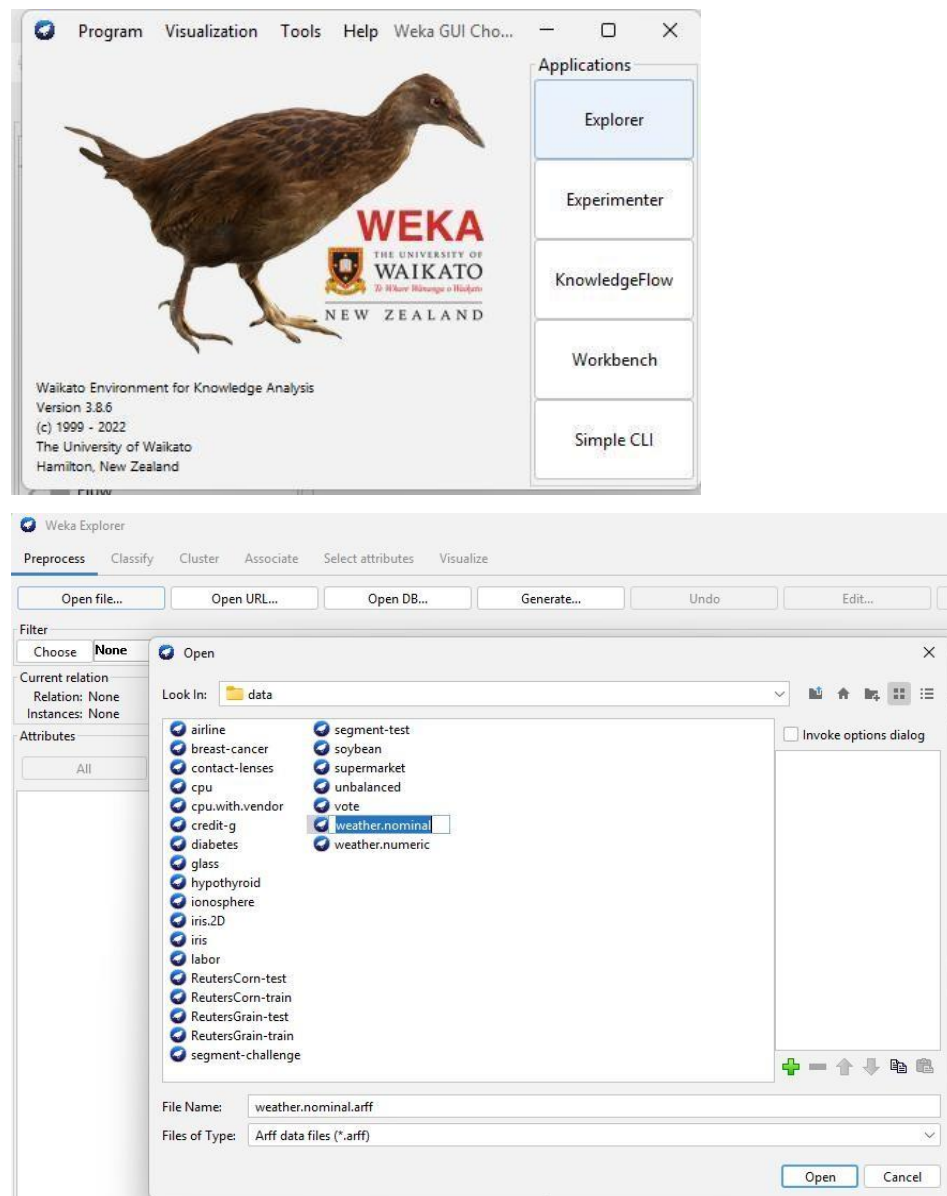
QUESTIONS:

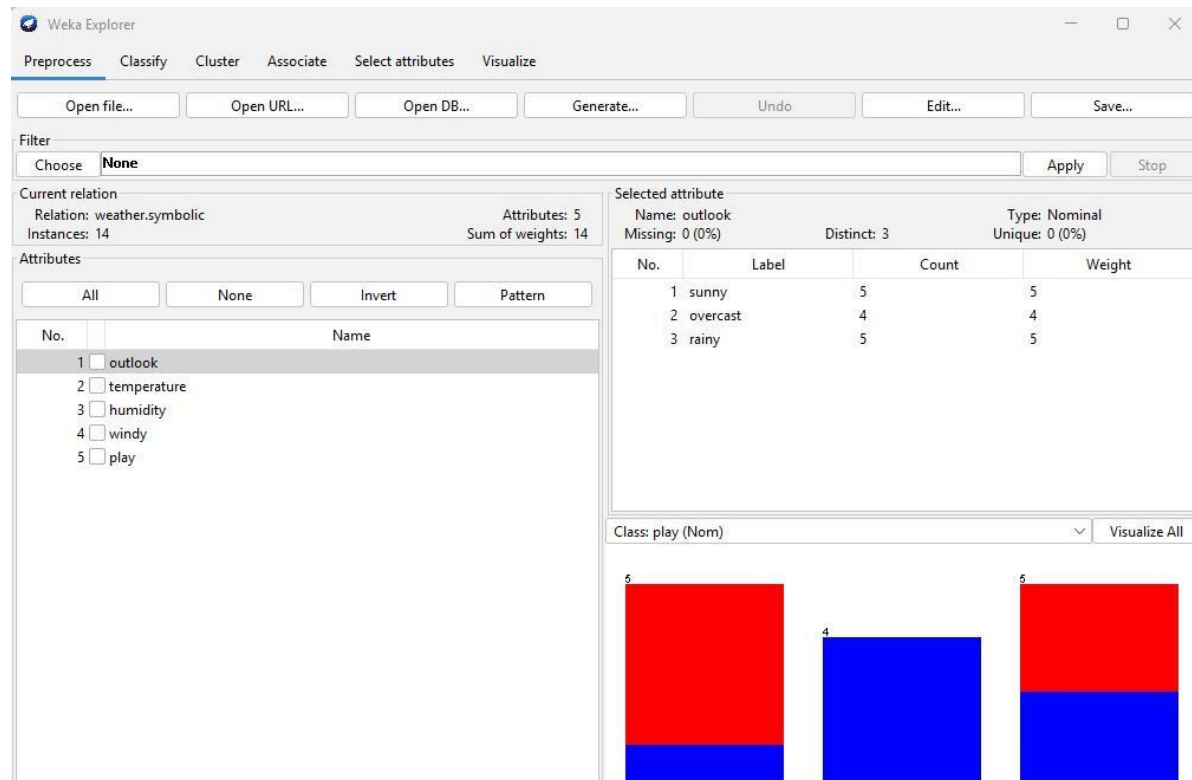
1. To familiarize with the WEKA Interface and load a dataset.

Goto Explorer → Open File → weather.nominal dataset

Observe:

- Number of Instances
- Number of Attributes
- Class Attribute
- Missing Values





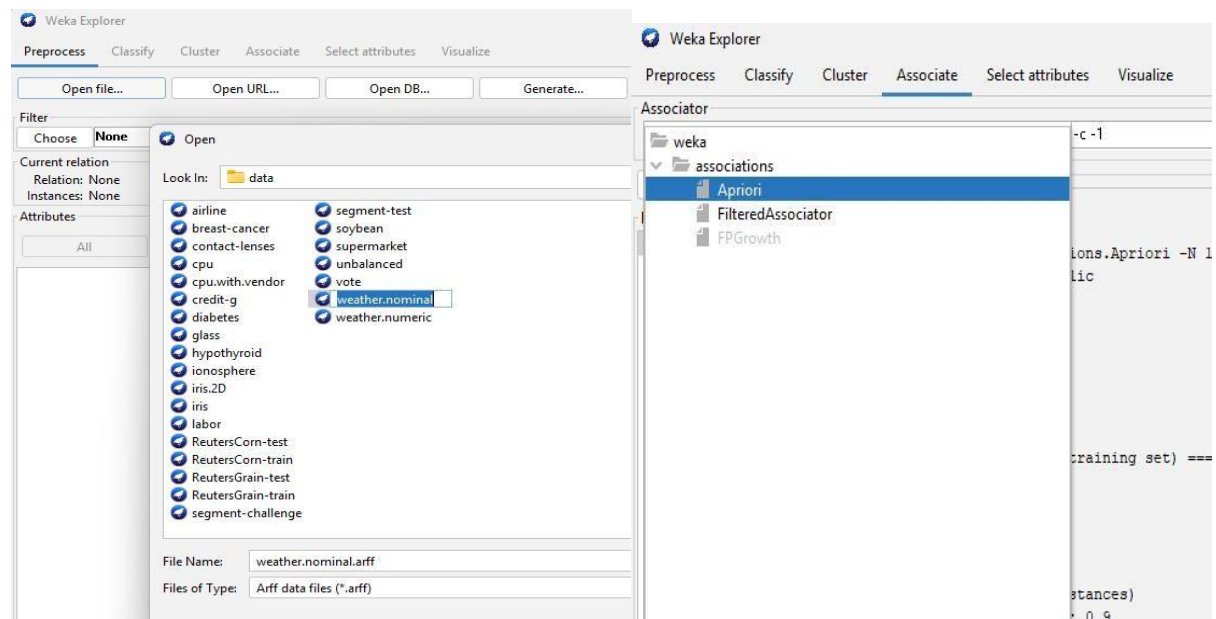
2. Association Rule Mining using Apriori

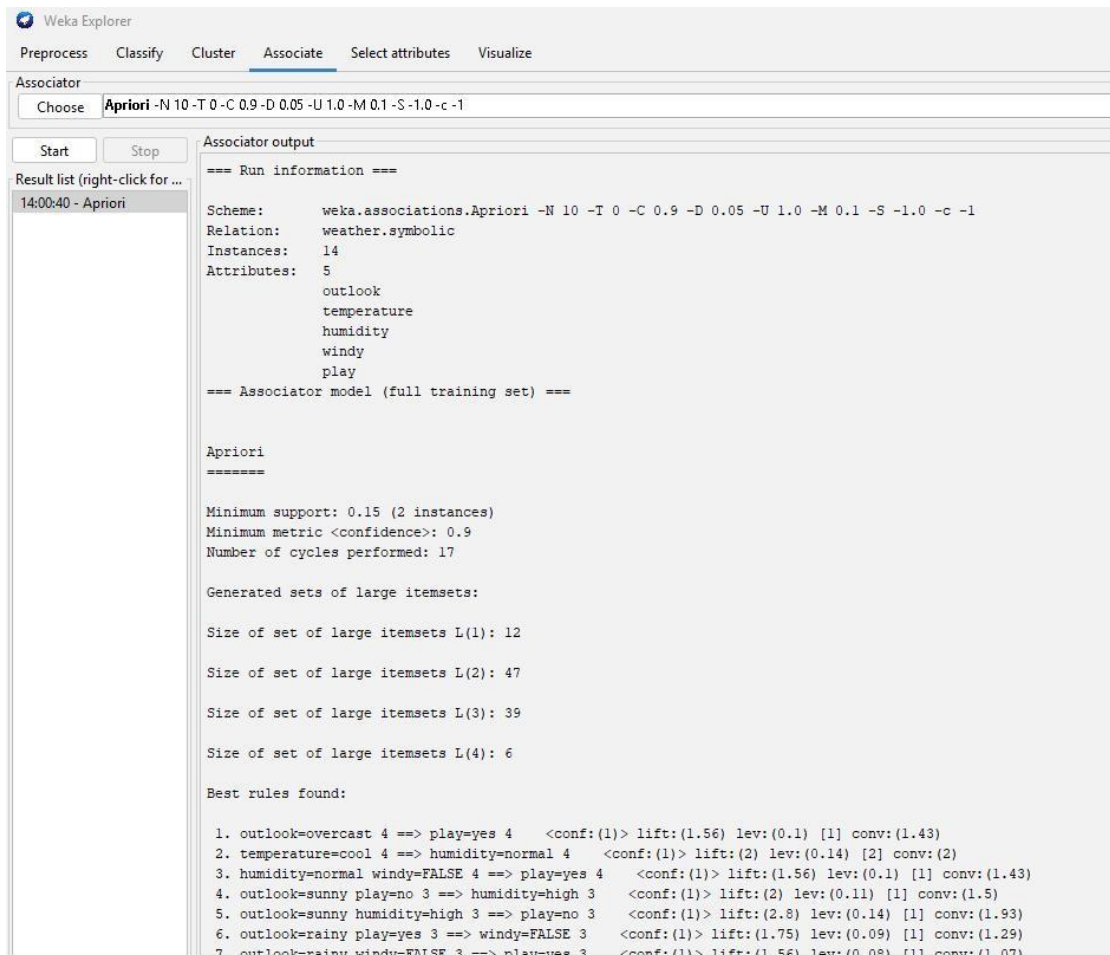
Go to Explorer → Open File → weather.nominal dataset

Choose → Apriori → Start

Record:

- Number of rules generated
- Strongest rule
- Highest confidence
- Highest lift





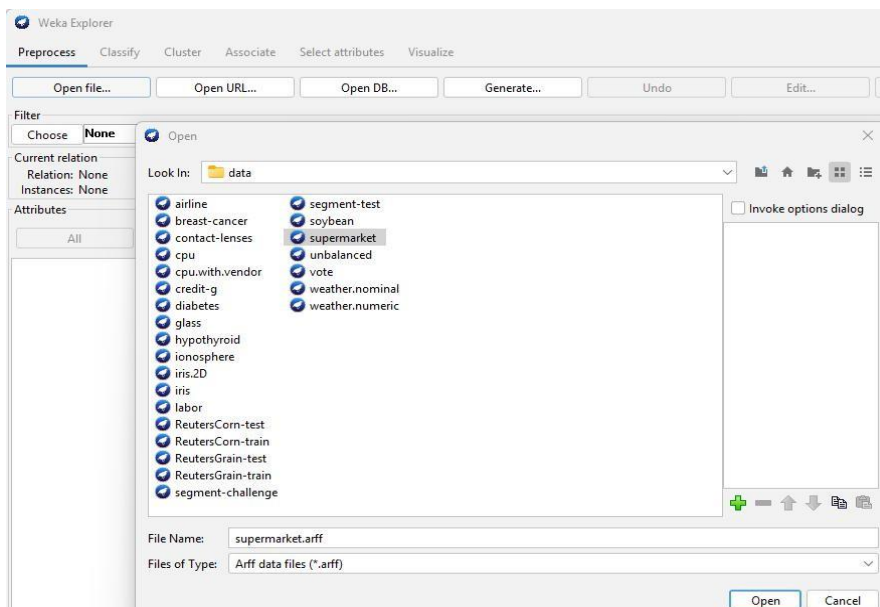
3. Association Rule Mining using FPGrowth.

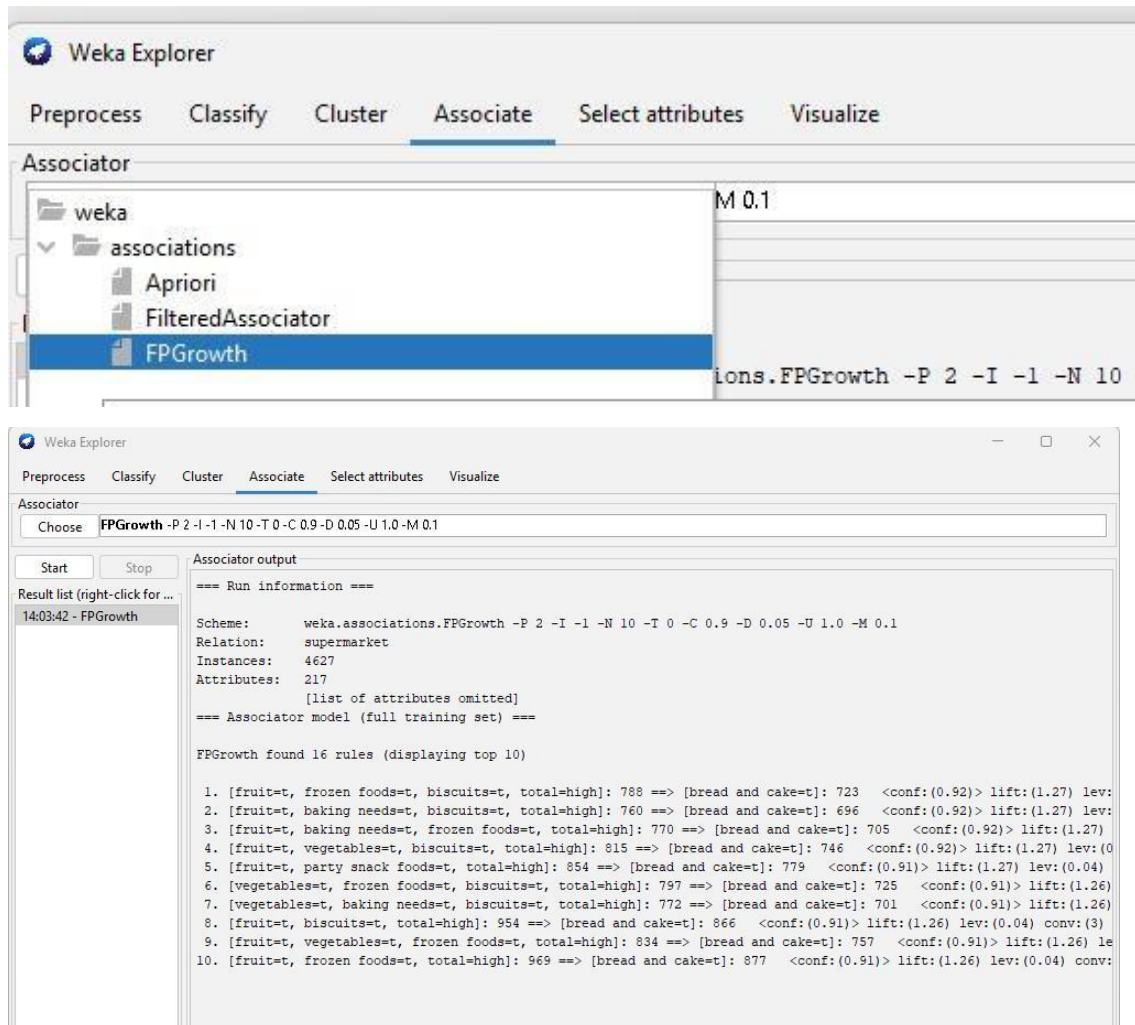
Goto Explorer → Open File → supermarket dataset

Choose → fpgrowth → Start

Record

- Number of rules generated
- Strongest rule
- Highest confidence
- Highest lift

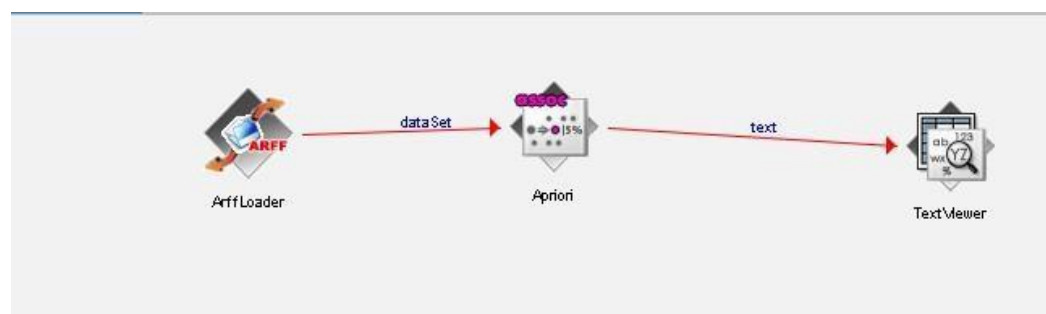


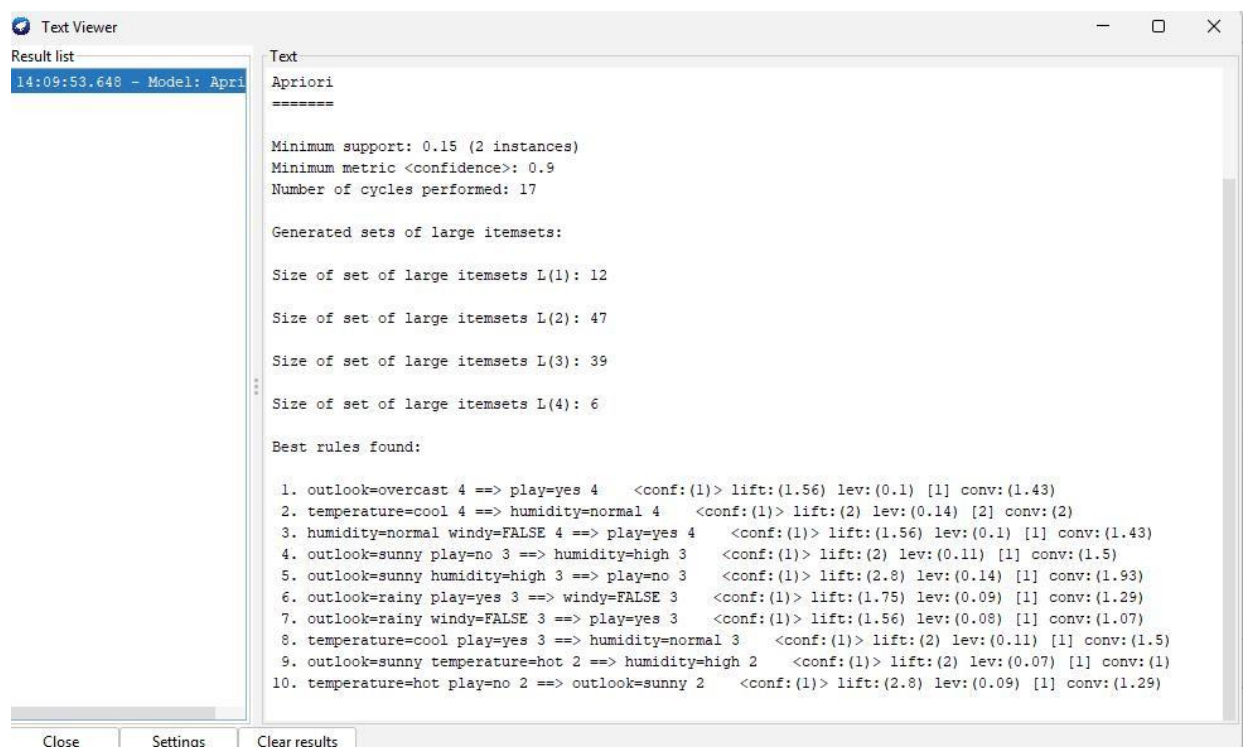
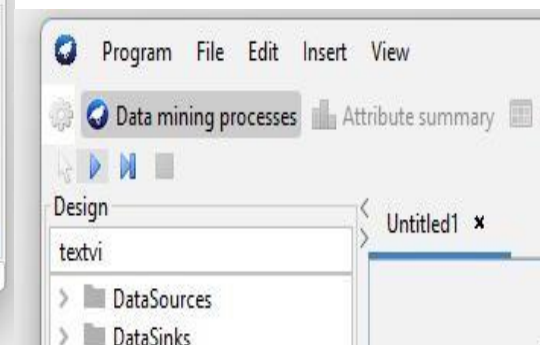
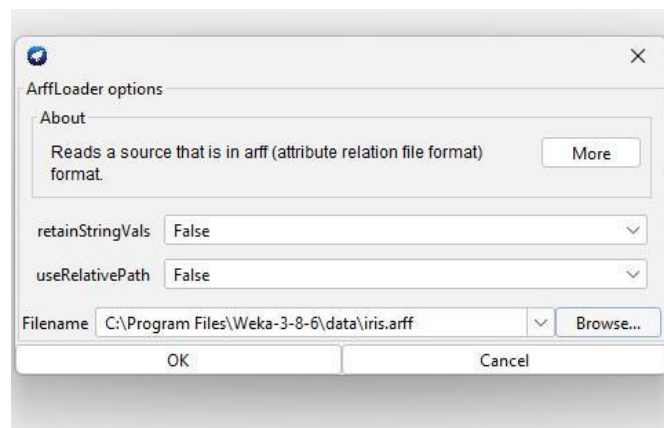
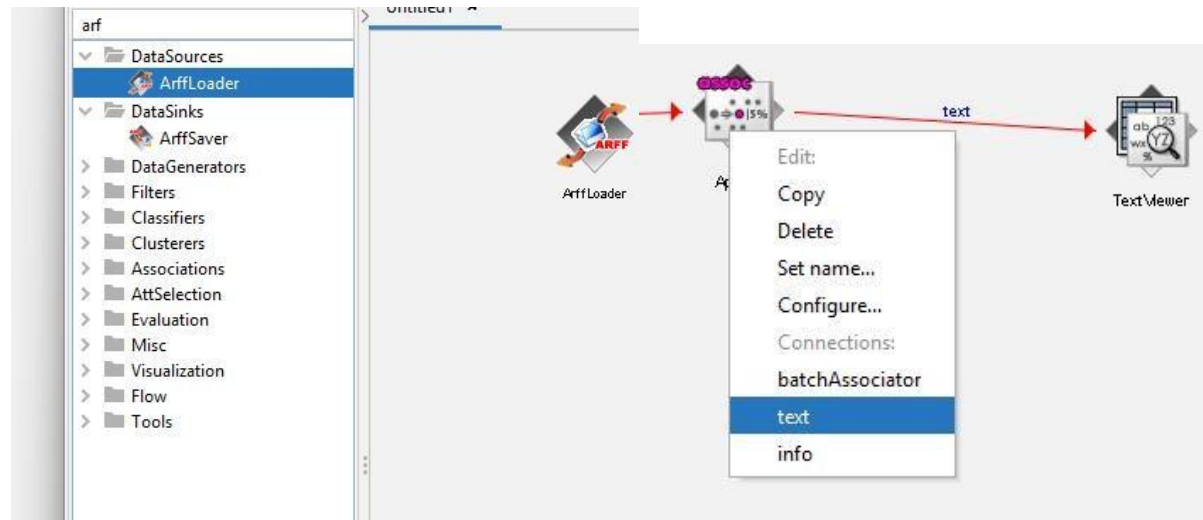


4. Implement Apriori algorithm using Weka Knowledge Flow layout.

Goto KnowledgeFlow

ArffLoader → Apriori → TextViewer



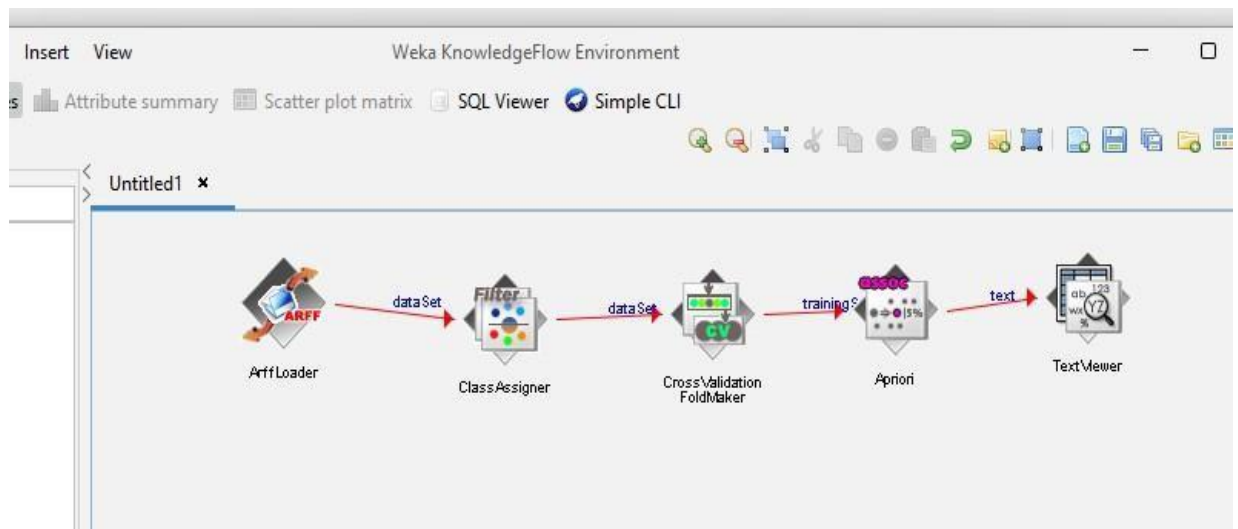
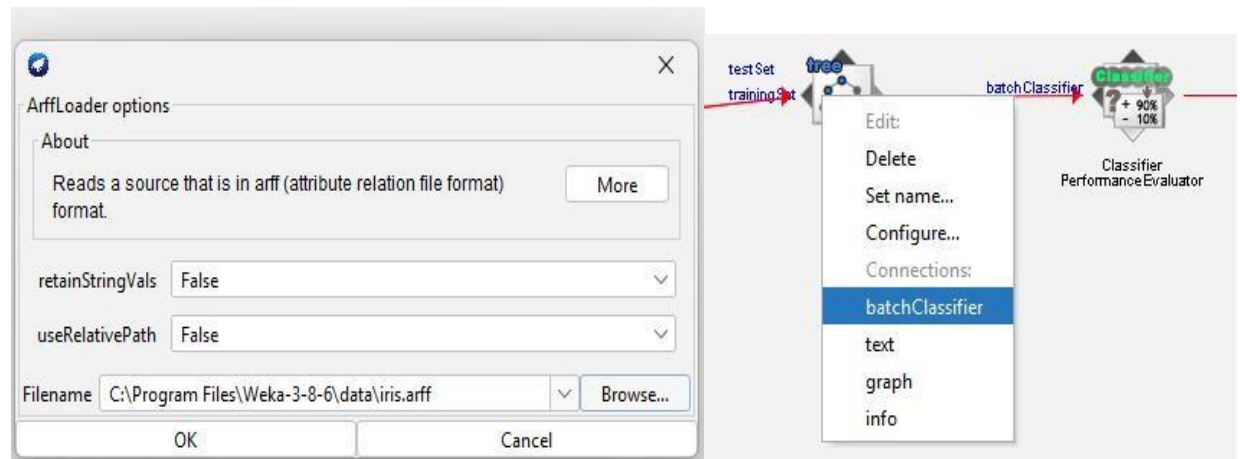


5. Implement Apriori algorithm with classassigner and crossvalidationfoldmaker using Weka Knowledge Flow layout.

Goto KnowledgeFlow

Dataset: weather.nominal

ArffLoader → ClassAssigner → CrossValidationFoldMaker → Apriori → TextViewer



Text Viewer

Result list

14:18:11.984 - Model: Apri
 14:18:11.987 - Model: Apri
 14:18:11.987 - Model: Apri
 14:18:11.988 - Model: Apri
 14:18:11.988 - Model: Apri
 14:18:11.988 - Model: Apri
 14:18:11.989 - Model: Apri
 14:18:11.989 - Model: Apri
 14:18:11.989 - Model: Apri
 14:18:11.989 - Model: Apri
 14:18:11.989 - Model: Apri

Text

=== Associator model ===

Scheme: Apriori
 Relation: weather.symbolic-weka.filters.unsupervised.attribute.ClassAssigner-Clast

Apriori
 =====

Minimum support: 0.15 (2 instances)
 Minimum metric <confidence>: 0.9
 Number of cycles performed: 17

Generated sets of large itemsets:

Size of set of large itemsets L(1): 12
 Size of set of large itemsets L(2): 43
 Size of set of large itemsets L(3): 32
 Size of set of large itemsets L(4): 5

Best rules found:

1. temperature=cool 4 ==> humidity=normal 4 <conf:(1)> lift:(1.86) lev:(0.14) [1] conv:(1.85)
 2. humidity=normal windy=FALSE 4 ==> play=yes 4 <conf:(1)> lift:(1.63) lev:(0.12) [1] conv:(1.54)
 3. outlook=overcast 3 ==> play=yes 3 <conf:(1)> lift:(1.63) lev:(0.09) [1] conv:(1.15)
 4. outlook=sunny play=no 3 ==> humidity=high 3 <conf:(1)> lift:(2.17) lev:(0.12) [1] conv:(1.62)
 5. outlook=sunny humidity=high 3 ==> play=no 3 <conf:(1)> lift:(2.6) lev:(0.14) [1] conv:(1.85)
 6. outlook=rainy play=yes 3 ==> windy=FALSE 3 <conf:(1)> lift:(1.86) lev:(0.11) [1] conv:(1.38)
 7. outlook=rainy windy=FALSE 3 ==> play=yes 3 <conf:(1)> lift:(1.63) lev:(0.09) [1] conv:(1.15)
 8. temperature=cool play=yes 3 ==> humidity=normal 3 <conf:(1)> lift:(1.86) lev:(0.11) [1] conv:(1.38)
 9. temperature=hot humidity=high 2 ==> outlook=sunny 2 <conf:(1)> lift:(2.6) lev:(0.09) [1] conv:(1.23)
 10. outlook=sunny temperature=hot 2 ==> humidity=high 2 <conf:(1)> lift:(2.17) lev:(0.08) [1] conv:(1.08)

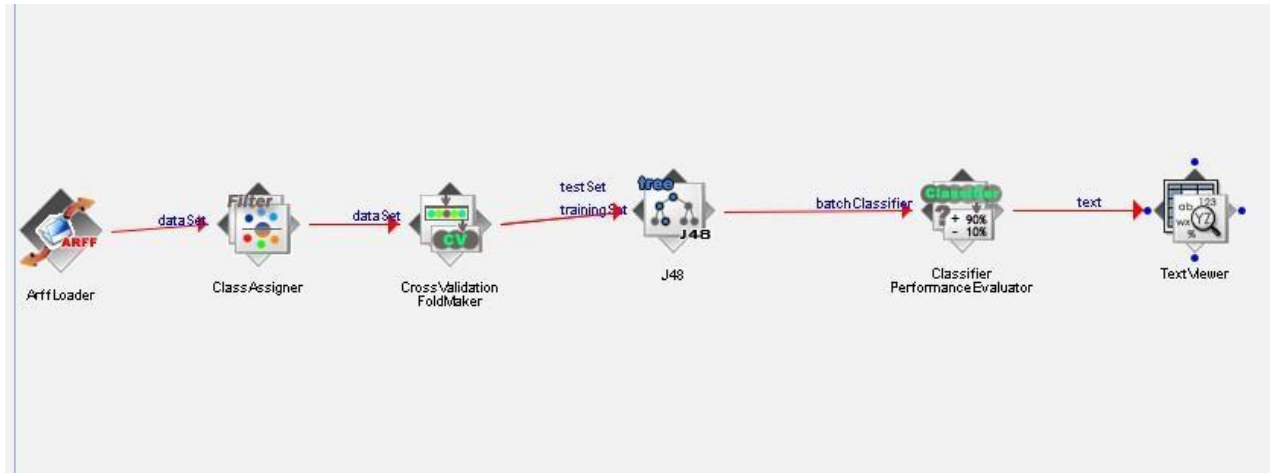
6. Implement J48 algorithm using Weka Knowledge Flow layout.

Goto KnowledgeFlow

Dataset: iris

Workflow: ArffLoader → ClassAssigner → CrossValidationFoldMaker → J48 →

ClassifierPerformanceEvaluator → TextViewer



Text Viewer

Result list
14:23:56.686 - J48

Text

```

=== Evaluation result ===

Scheme: J48
Options: -C 0.25 -M 2
Relation: iris-weka.filters.unsupervised.attribute.ClassAssigner-Clas

=== Summary ===

Correctly Classified Instances      144           96    %
Incorrectly Classified Instances     6            4    %
Kappa statistic                    0.94
Mean absolute error                 0.035
Root mean squared error             0.1586
Relative absolute error             7.8705 %
Root relative squared error        33.6353 %
Total Number of Instances         150

=== Detailed Accuracy By Class ===

      TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
      0.980    0.000    1.000     0.980    0.990     0.985    0.990     0.987    Iris-setosa
      0.940    0.030    0.940     0.940    0.940     0.910    0.952     0.880    Iris-versicolor
      0.960    0.030    0.941     0.960    0.950     0.925    0.961     0.905    Iris-virginica
Weighted Avg.   0.960    0.020    0.960     0.960    0.960     0.940    0.968     0.924

=== Confusion Matrix ===

 a  b  c  <-- classified as
49  1  0 | a = Iris-setosa
 0 47  3 | b = Iris-versicolor
 0  2 48 | c = Iris-virginica
  
```

Close Settings Clear results

RESULT:

Thus, the data mining process was successfully implemented using the WEKA tool.

EXP NO: 9
DATE:

EXECUTION OF PIG LATIN QUERIES FOR DATA ANALYSIS USING APACHE PIG

AIM:

To execute Pig Latin queries on a student dataset using Apache Pig in the Cloudera environment and perform basic data-analysis operations such as loading, filtering, projection, grouping, counting, and aggregation.

QUESTIONS:

Step 1: Prepare the Student Dataset

Create a file named student.csv containing:

```
1,adarsh,int,83
2,aswin,cs,101
3,kumar,mec,105
4,bershey,bio,106
```

Upload the file to HDFS.

OUTPUT:

Name	Size	User	Group	Permissions	Date
.		hdfs	supergroup	drwxr-xr-x	August 06, 2026 12:42 AM
.Trash		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:33 AM
.staging		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:35 AM
input		it23a41	it23a41	drwxr-xr-x	August 06, 2026 12:57 AM
oozie-oozi		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:35 AM
output		it23a41	it23a41	drwxr-xr-x	August 13, 2026 12:33 AM
sample.txt	36 bytes	it23a41	it23a41	-rw-r--r--	August 13, 2026 12:17 AM
student.csv	173 bytes	it23a41	it23a41	-rw-r--r--	August 13, 2026 12:30 AM

Step 2: Start Apache Pig

Open the terminal in the Cloudera VM and select Pig from Query Editors

Q1: Load and Display/Save All Records

```
students = LOAD '/user/john/student.csv'
USING PigStorage(',')
AS (roll:int,name:chararray,dept:chararray,marks:int);
STORE students
INTO '/user/john/output/q1'
USING PigStorage(',');
```

OUTPUT:

```
1,Aishwarya,int,83
2,Priya,cs,101
3,Sneha,mec,105
4,Kavya,bio,106
```

Q2: Filter Students Scoring More Than 85

```
high = FILTER students BY marks > 85;
STORE high
INTO '/user/john/output/q2'
USING PigStorage(',');
```

OUTPUT:

```
2,aswin,cs,101
3,kumar,mec,105
4,bershey,bio,106
```

Q3: Display Only Student Name and Marks

```
result1 = FOREACH students GENERATE name, marks;
STORE result1
INTO '/user/john/output/q3'
USING PigStorage(',');
```

OUTPUT:

```
adarsh,83
aswin,101
kumar,105
bershey,106
```

Q4: Group Students Department-wise

```
grp = GROUP students BY dept;
result2 = FOREACH grp
GENERATE group, COUNT(students);
STORE result2
INTO '/user/john/output/q4'
USING PigStorage(',');
```

OUTPUT:

```
cs,1  
bio,1  
int,1  
mec,1
```

Q5: Find Average Marks Department-wise

```
result3 = FOREACH grp  
GENERATE group, AVG(students.marks);  
STORE result3  
INTO '/user/john/output/q5'  
USING PigStorage(',');  
:
```

RESULT:

Thus, the student dataset was successfully loaded and analysed using Apache Pig in the Cloudera environment. Pig Latin operations such as LOAD, FILTER, FOREACH, GROUP, COUNT, AVG, DUMP, and STORE were performed successfully to filter student records, project selected fields, group students department-wise, count students, and calculate average marks.

EXP NO: 10**DATE:****IMPLEMENTATION OF BASIC HIVEQL DATABASE,
TABLE, DATA INSERTION AND QUERY****AIM:**

To create a Hive database and employee table, insert employee records, and perform basic HiveQL queries on the inserted data.

QUESTIONS:**Step 1: Open Hive Editor**

Open the Cloudera Hue Web UI and select: Query Editors → Hive

Step 2: Create the Database

```
CREATE DATABASE IF NOT EXISTS userdb;  
SHOW DATABASES;  
USE userdb;
```

Q1: Create the Employee Table

```
CREATE TABLE IF NOT EXISTS emp1 (  
    eid INT,  
    name STRING,  
    salary FLOAT,  
    dest STRING  
);
```

```
INSERT INTO TABLE emp1  
VALUES  
(1,"devi",40000.0,"Chennai"),  
(2,"rose",45000.0,"Hydrebad"),  
(3,"bob",50000.0,"Chennai");  
Display all records:  
SELECT * FROM emp1;
```

OUTPUT:

The screenshot shows the Hive Editor interface with a query executed. The query is as follows:

```

1 CREATE DATABASE IF NOT EXISTS userdb;
2 SHOW DATABASES;
3 CREATE TABLE IF NOT EXISTS emp1 (eid int, name string, salary float, dest string);
4 INSERT INTO TABLE emp1
5 VALUES
6 (1,"devi",40000.0,"Chennai"),
7 (2,"rose",45000.0,"Hydrabad"),
8 (3,"bob",50000.0,"Chennai");
9 SELECT * FROM emp1;

```

The result shows a data sample for 'values_tmp_table_1' with the following data:

tmp_values_col1	tmp_values_col2	tmp_values_col3	tmp_values_col4
0 1	devi	40000.0	Chennai
1 2	rose	45000.0	Hydrabad
2 3	bob	50000.0	Chennai

Q2: Display Employees with Salary Greater Than 40,000

```

SELECT *
FROM emp1
WHERE salary > 40000;

```

OUTPUT:

The screenshot shows the Hive Editor interface with a query executed. The query is as follows:

```

SELECT *
FROM emp1
WHERE salary > 40000;

```

The result shows a data sample for 'emp1' with the following data:

eid	name	salary	dest
0 2	rose	45000.0	Hydrabad
1 3	bob	50000.0	Chennai

Q3: Display Employee Name and Destination

```

SELECT name, dest
FROM emp1;

```

OUTPUT:

	name	dest
0	devi	Chennai
1	rose	Hydresbad
2	bob	Chennai

Q4: Find the Highest Salary

```
SELECT MAX(salary) AS highest_salary
FROM emp1;
```

OUTPUT:

The screenshot shows the Hue interface with a query editor containing the SQL query: `SELECT MAX(salary) AS highest_salary FROM emp1;`. Below the query editor, a 'Data sample for highest_salary' window is displayed, showing a single row with the value 50000.0 for the column highest_salary.

	highest_salary
0	50000.0

Q5: Display Employees in Descending Order of Salary

```
SELECT eid, name, salary, dest
FROM emp1
ORDER BY salary DESC;
```

OUTPUT:

Data sample for emp1

	eid	name	salary	dest
0	3	bob	50000.0	Chennai
1	2	rose	45000.0	Hydresbad
2	1	devi	40000.0	Chennai

RESULT:

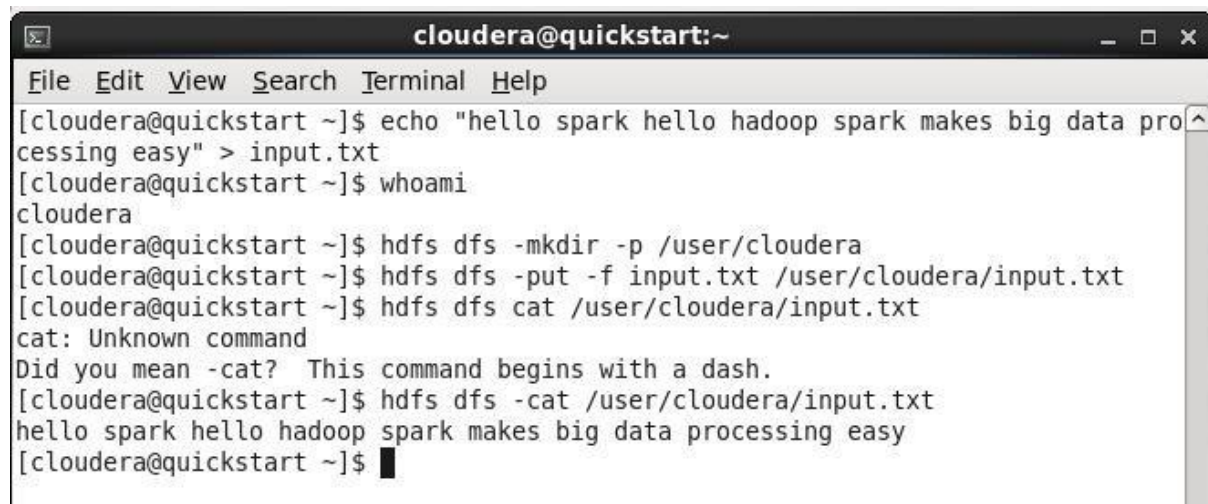
Thus, a Hive database and employee table were successfully created using HiveQL in the Cloudera environment. Employee records were inserted and retrieved successfully, and various HiveQL operations such as filtering, projection, aggregate function MAX(), and sorting using ORDER BY were performed successfully.

EXP NO: 11**WORD COUNT USING SPARK 1.3.0 AND HDFS****DATE:****AIM:**

To implement a simple Word Count program in Apache Spark using RDDs by reading input data from HDFS and writing the output back to HDFS.

STEPS:

1. Open the Cloudera environment and connect through the terminal/Putty.
2. Create an input text file in HDFS using Hue WebUI under the user directory.
Example content: hello spark hello Hadoop spark makes big data processing easy
3. Start the Spark shell: spark-shell
A Scala REPL should open, and Spark should indicate that the Spark context is available as sc and SQL context as sqlContext.
4. Load the input file from HDFS:
`val textFile = sc.textFile("hdfs:///user/john/input.txt")`
To verify that the file is loaded: `textFile.first()`
5. Split each line into individual words:
`val words = textFile.flatMap(line => line.split("\\s+"))`
6. Convert each word into a key-value pair (word, 1):
`val wordPairs = words.map(word => (word, 1))`
7. Count the occurrences of each word using reduceByKey:
`val wordCounts = wordPairs.reduceByKey(_ + _)`
8. Display the result:
`wordCounts.collect().foreach { case (word, count) => println(s"$word: $count") }`
9. Save the result back to HDFS:
`wordCounts.saveAsTextFile("hdfs:///user/john/output_wordcount")`
10. Exit the Scala shell: :quit or: exit

OUTPUT:


```

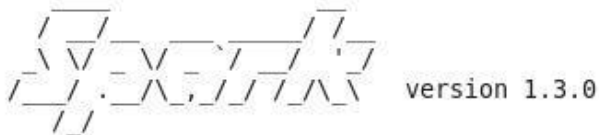
cloudera@quickstart:~
File Edit View Search Terminal Help
[cloudera@quickstart ~]$ echo "hello spark hello hadoop spark makes big data processing easy" > input.txt
[cloudera@quickstart ~]$ whoami
cloudera
[cloudera@quickstart ~]$ hdfs dfs -mkdir -p /user/cloudera
[cloudera@quickstart ~]$ hdfs dfs -put -f input.txt /user/cloudera/input.txt
[cloudera@quickstart ~]$ hdfs dfs cat /user/cloudera/input.txt
cat: Unknown command
Did you mean -cat? This command begins with a dash.
[cloudera@quickstart ~]$ hdfs dfs -cat /user/cloudera/input.txt
hello spark hello hadoop spark makes big data processing easy
[cloudera@quickstart ~]$

```

```

File Edit View Search Terminal Help
[cloudera@quickstart ~]$ spark-shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/lib/zookeeper/lib/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/jars/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/lib/parquet/lib/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
26/08/31 00:18:27 INFO SecurityManager: Changing view acls to: cloudera
26/08/31 00:18:27 INFO SecurityManager: Changing modify acls to: cloudera
26/08/31 00:18:27 INFO SecurityManager: SecurityManager: authentication disabled; ui acls disabled; users with view permissions: Set(cloudera); users with modify permissions: Set(cloudera)
26/08/31 00:18:28 INFO HttpServer: Starting HTTP Server
26/08/31 00:18:28 INFO Server: jetty-8.y.z-SNAPSHOT
26/08/31 00:18:28 INFO AbstractConnector: Started SocketConnector@0.0.0.0:53410
26/08/31 00:18:28 INFO Utils: Successfully started service 'HTTP class server' on port 53410.
Welcome to

```



```

Using Scala version 2.10.4 (Java HotSpot(TM) 64-Bit Server VM, Java 1.7.0_67)
Type in expressions to have them evaluated.
Type :help for more information.
26/08/31 00:18:52 INFO SparkContext: Running Spark version 1.3.0
26/08/31 00:18:52 INFO SecurityManager: Changing view acls to: cloudera
26/08/31 00:18:52 INFO SecurityManager: Changing modify acls to: cloudera

```

```

tracking URL: http://quickstart.cloudera:8080/proxy/application_178582660745_0018/
user: cloudera
26/08/31 00:19:56 INFO YarnClientSchedulerBackend: Application application_178582660745_0018 has started running.
26/08/31 00:19:58 INFO NettyBlockTransferService: Server created on 48388
26/08/31 00:19:58 INFO BlockManagerMaster: Trying to register BlockManager
26/08/31 00:19:59 INFO BlockManagerMasterActor: Registering block manager quickstart.cloudera:48388 with 265.4 MB RAM, BlockManagerId(<driver>, quickstart.cloudera, 48388)
26/08/31 00:20:01 INFO BlockManagerMaster: Registered BlockManager
26/08/31 00:20:06 INFO EventLoggingListener: Logging events to hdfs://quickstart.cloudera:8020/user/spark/applicationHistory/application_178582660745_0018
26/08/31 00:20:07 INFO YarnClientSchedulerBackend: SchedulerBackend is ready for scheduling beginning after waiting maxRegisteredResourcesWaitingTime: 30000(ms)
26/08/31 00:20:07 INFO SparkILoop: Created spark context..
Spark context available as sc.
26/08/31 00:20:11 INFO SparkILoop: Created sql context (with Hive support)..
SQL context available as sqlContext.

```

```

scala> 26/08/31 00:20:49 INFO YarnClientSchedulerBackend: Registered executor: Ac
26/08/31 00:20:52 INFO BlockManagerMasterActor: Registering block manager quickst

```

```
scala> █
```

```

scala> val textfile1 = sc.textFile("hdfs:///user/cloudera/input.txt")
26/08/31 00:23:34 INFO MemoryStore: ensureFreeSpace(283068) called with curMem=305316, maxMem=278302556
26/08/31 00:23:34 INFO MemoryStore: Block broadcast_1 stored as values in memory (estimated size 276.4
26/08/31 00:23:34 INFO MemoryStore: ensureFreeSpace(22296) called with curMem=588384, maxMem=278302556
26/08/31 00:23:34 INFO MemoryStore: Block broadcast_1_piece0 stored as bytes in memory (estimated size
26/08/31 00:23:34 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on quickstart.cloudera:4838
26/08/31 00:23:34 INFO BlockManagerMaster: Updated info of block broadcast_1_piece0
26/08/31 00:23:34 INFO SparkContext: Created broadcast 1 from textFile at <console>:21
textfile1: org.apache.spark.rdd.RDD[String] = hdfs:///user/cloudera/input.txt MapPartitionsRDD[3] at t
scala> textfile1.first()
26/08/31 00:23:47 INFO FileInputFormat: Total input paths to process : 1
26/08/31 00:23:48 INFO SparkContext: Starting job: first at <console>:24
26/08/31 00:23:48 INFO DAGScheduler: Got job 0 (first at <console>:24) with 1 output partitions (allow
26/08/31 00:23:48 INFO DAGScheduler: Final stage: Stage 0(first at <console>:24)
26/08/31 00:23:48 INFO DAGScheduler: Parents of final stage: List()
26/08/31 00:23:48 INFO DAGScheduler: Missing parents: List()
26/08/31 00:23:48 INFO DAGScheduler: Submitting Stage 0 (hdfs:///user/cloudera/input.txt MapPartitions
26/08/31 00:23:48 INFO MemoryStore: ensureFreeSpace(2680) called with curMem=610680, maxMem=278302556
26/08/31 00:23:48 INFO MemoryStore: Block broadcast_2 stored as values in memory (estimated size 2.6 KI
26/08/31 00:23:48 INFO MemoryStore: ensureFreeSpace(1656) called with curMem=613360, maxMem=278302556
26/08/31 00:23:48 INFO MemoryStore: Block broadcast_2_piece0 stored as bytes in memory (estimated size
26/08/31 00:23:48 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on quickstart.cloudera:4838
26/08/31 00:23:48 INFO BlockManagerMaster: Updated info of block broadcast_2_piece0
26/08/31 00:23:48 INFO SparkContext: Created broadcast 2 from broadcast at DAGScheduler.scala:839
26/08/31 00:23:48 INFO DAGScheduler: Submitting 1 missing tasks from Stage 0 (hdfs:///user/cloudera/in
26/08/31 00:23:48 INFO YarnScheduler: Adding task set 0.0 with 1 tasks
26/08/31 00:23:49 INFO TaskSetManager: Starting task 0.0 in stage 0.0 (TID 0, quickstart.cloudera, NODI
26/08/31 00:23:53 INFO BlockManagerInfo: Added broadcast_2_piece0 in memory on quickstart.cloudera:388
26/08/31 00:23:56 INFO BlockManagerInfo: Added broadcast_1_piece0 in memory on quickstart.cloudera:388
26/08/31 00:24:13 INFO TaskSetManager: Finished task 0.0 in stage 0.0 (TID 0) in 24415 ms on quickstart
26/08/31 00:24:13 INFO DAGScheduler: Stage 0 (first at <console>:24) finished in 24.516 s
26/08/31 00:24:13 INFO YarnScheduler: Removed TaskSet 0.0, whose tasks have all completed, from pool
26/08/31 00:24:13 INFO DAGScheduler: Job 0 finished: first at <console>:24, took 25.511302 s
res1: String = hello spark hello hadoop spark makes big data processing easy

```

```

scala> val words = textfile1.flatMap(line => line.split("\\s+"))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[4] at flatMap at <console>:23

scala> val wordpairs = words.map(word => (word,1))
wordpairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[5] at map at <console>:25

scala> val wordcounts = wordpairs.reduceByKey(_+_ )
wordcounts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[6] at reduceByKey at <console>:27

scala> wordcounts.collect().foreach {case (word,count) => println(s"$word: $count")}
26/08/31 00:24:19 INFO SparkContext: Starting job: collect at <console>:30
26/08/31 00:24:19 INFO DAGScheduler: Registering RDD 5 (map at <console>:25)
26/08/31 00:24:19 INFO DAGScheduler: Got job 1 (collect at <console>:30) with 2 output partitions (a
26/08/31 00:24:19 INFO DAGScheduler: Final stage: Stage 2(collect at <console>:30)
26/08/31 00:24:19 INFO DAGScheduler: Parents of final stage: List(Stage 1)
26/08/31 00:24:19 INFO DAGScheduler: Missing parents: List(Stage 1)
26/08/31 00:24:19 INFO DAGScheduler: Submitting Stage 1 (MapPartitionsRDD[5] at map at <console>:25)
26/08/31 00:24:19 INFO MemoryStore: ensureFreeSpace(3752) called with curMem=615016, maxMem=27830255
26/08/31 00:24:19 INFO MemoryStore: Block broadcast_3 stored as values in memory (estimated size 3.7
26/08/31 00:24:19 INFO MemoryStore: ensureFreeSpace(2158) called with curMem=618768, maxMem=27830255
26/08/31 00:24:19 INFO MemoryStore: Block broadcast_3_piece0 stored as bytes in memory (estimated si
26/08/31 00:24:19 INFO BlockManagerInfo: Added broadcast_3_piece0 in memory on quickstart.cloudera:4
26/08/31 00:24:19 INFO BlockManagerMaster: Updated info of block broadcast_3_piece0
26/08/31 00:24:19 INFO SparkContext: Created broadcast 3 from broadcast at DAGScheduler.scala:839
26/08/31 00:24:19 INFO DAGScheduler: Submitting 2 missing tasks from Stage 1 (MapPartitionsRDD[5] at
26/08/31 00:24:23 INFO SparkContext:
26/08/31 00:24:23 INFO DAGScheduler:
26/08/31 00:24:23 INFO YarnScheduler:
26/08/31 00:24:23 INFO TaskSetManager
26/08/31 00:24:23 INFO BlockManagerIn
26/08/31 00:24:23 INFO MapOutputTrack
26/08/31 00:24:23 INFO MapOutputTrack
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO DAGScheduler:
26/08/31 00:24:24 INFO DAGScheduler:
26/08/31 00:24:24 INFO TaskSetManager
26/08/31 00:24:24 INFO YarnScheduler:
big: 1
data: 1
hello: 2
easy: 1
spark: 2
hadoop: 1
processing: 1
makes: 1

```

```

scala> wordcounts.saveAsTextFile("hdfs:///user/cloudera/output_wordcount")
26/08/31 00:26:26 INFO BlockManager: Removing broadcast 4
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_4
26/08/31 00:26:26 INFO MemoryStore: Block broadcast 4 of size 2128 dropped from memory (free 2776803;
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_4_piece0
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_4_piece0 of size 1310 dropped from memory (free ;
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_4_piece0 on quickstart.cloudera:48388 in ;
26/08/31 00:26:26 INFO BlockManagerMaster: Updated info of block broadcast_4_piece0
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_4_piece0 on quickstart.cloudera:38801 in ;
26/08/31 00:26:26 INFO ContextCleaner: Cleaned broadcast 4
26/08/31 00:26:26 INFO BlockManager: Removing broadcast 3
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_3_piece0
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_3_piece0 of size 2158 dropped from memory (free ;
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_3_piece0 on quickstart.cloudera:48388 in ;
26/08/31 00:26:26 INFO BlockManagerMaster: Updated info of block broadcast_3_piece0
26/08/31 00:26:26 INFO BlockManager: Removing block broadcast_3
26/08/31 00:26:26 INFO MemoryStore: Block broadcast_3 of size 3752 dropped from memory (free 2776875;
26/08/31 00:26:26 INFO BlockManagerInfo: Removed broadcast_3_piece0 on quickstart.cloudera:38801 in ;
26/08/31 00:26:26 INFO ContextCleaner: Cleaned broadcast 3
26/08/31 00:26:26 INFO deprecation: mapred.tip.id is deprecated. Instead, use mapreduce.task.id
26/08/31 00:26:26 INFO deprecation: mapred.task.id is deprecated. Instead, use mapreduce.task.attempt
26/08/31 00:26:26 INFO deprecation: mapred.task.is.map is deprecated. Instead, use mapreduce.task.isr
26/08/31 00:26:26 INFO deprecation: mapred.task.partition is deprecated. Instead, use mapreduce.task
26/08/31 00:26:26 INFO deprecation: mapred.job.id is deprecated. Instead, use mapreduce.job.id
26/08/31 00:26:26 INFO FileOutputCommitter: File Output Committer Algorithm version is 1
26/08/31 00:26:27 INFO SparkContext: Starting job: saveAsTextFile at <console>:30
26/08/31 00:26:27 INFO DAGScheduler: Got job 2 (saveAsTextFile at <console>:30) with 2 output partit
26/08/31 00:26:27 INFO DAGScheduler: Final stage: Stage 4(saveAsTextFile at <console>:30)
26/08/31 00:26:27 INFO DAGScheduler: Parents of final stage: List(Stage 3)
26/08/31 00:26:27 INFO DAGScheduler: Missing parents: List()
26/08/31 00:26:27 INFO DAGScheduler: Submitting Stage 4 (MapPartitionsRDD[7] at saveAsTextFile at <co
26/08/31 00:26:27 INFO MemoryStore: ensureFreeSpace(137608) called with curMem=615016, maxMem=278302;
26/08/31 00:26:27 INFO MemoryStore: Block broadcast 5 stored as values in memory (estimated size 134
26/08/31 00:26:27 INFO MemoryStore: ensureFreeSpace(47774) called with curMem=752624, maxMem=2783025;
26/08/31 00:26:27 INFO MemoryStore: Block broadcast_5_piece0 stored as bytes in memory (estimated si
26/08/31 00:26:27 INFO BlockManagerInfo: Added broadcast_5_piece0 in memory on quickstart.cloudera:4
26/08/31 00:26:27 INFO BlockManagerMaster: Updated info of block broadcast_5_piece0
26/08/31 00:26:27 INFO SparkContext: Created broadcast 5 from broadcast at DAGScheduler.scala:839

scala> :quit
Stopping spark context.
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/metrics/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/stages/stage/kill,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/static,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/threadDump/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/threadDump,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/executors,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/environment/json,null}
26/08/31 00:27:41 INFO ContextHandler: stopped o.s.j.s.ServletContextHandler{/environment,null}
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/output_wordcount
Found 3 items
-rw-r--r-- 1 cloudera cloudera 0 2026-08-31 00:26 /user/cloudera/output_wordcount/_SUCCESS
-rw-r--r-- 1 cloudera cloudera 36 2026-08-31 00:26 /user/cloudera/output_wordcount/part-00000
-rw-r--r-- 1 cloudera cloudera 46 2026-08-31 00:26 /user/cloudera/output_wordcount/part-00001
[cloudera@quickstart ~]$ hdfs dfs -cat '/user/cloudera/output_wordcount/part-*'
(big,1)
(data,1)
(hello,2)
(easy,1)
(spark,2)
(hadoop,1)
(processing,1)
(makes,1)
[cloudera@quickstart ~]$

```

RESULT:

Thus, the Word Count program was successfully implemented using Apache Spark RDDs. The input file was read from HDFS, split into words, converted into key-value pairs, reduced to obtain word frequencies, displayed successfully, and the output was stored back into HDFS.

EXP NO: 12
DATE:

WORKING WITH HBASE IN CLOUDERA MANAGER

AIM:

To explore the features of HBase using Cloudera Manager and perform basic operations such as creating a table, inserting data, and retrieving data.

QUESTIONS:

Q1: Explore HBase in Cloudera Manager

1. Start the Cloudera environment.
2. Open Cloudera Manager.
3. Select the cluster.
4. Click the HBase service.
5. Observe whether HBase is running.
6. Open the Charts section.
7. Observe HBase metrics such as:
 - Requests per second
 - Reads and writes
 - Compactions
 - Other available performance indicators
8. Open the Configuration section.
9. Search for:
 - heap
 - hbase.rootdir
10. Record the configured heap size and HDFS storage path.

OUTPUT:

APACHE

HBASE

Home

Table Details

Procedures & Locks

HBCK Report

Operation Details

Process Metrics

Local Logs

Log Level

Debug Dump

Metrics

Profiler

HBase Configuration

Startup Progress

Master

localhost

Region Servers

Base Stats

Memory

Requests

Storefiles

Compactions

Replications

Client Connections

ServerName	State	Start time	Last contact	Version	Requests Per Second	Num. Regions
localhost:16020.1788094380788	Normal	Sun Aug 30 18:23:00 IST 2026	2 s	3.0.0	0	1
Total:1					0	1

Backup Masters

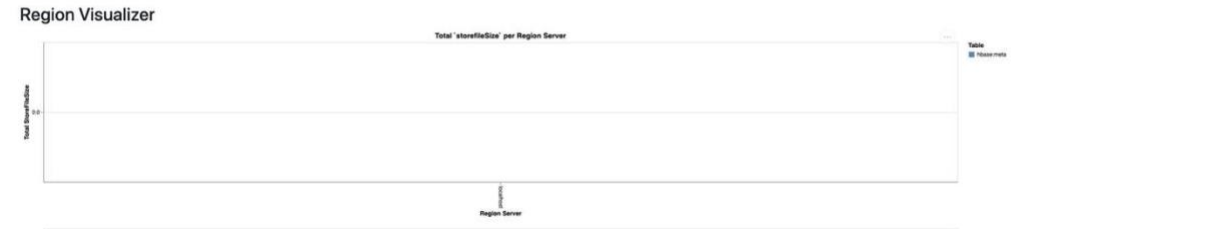
ServerName	Port	Start Time
Total:0		

Tables

User Tables

System Tables

Snapshots



APACHE

HBASE

Home

Table Details

Procedures & Locks

HBCK Report

Operation Details

Process Metrics

Local Logs

Log Level

Debug Dump

Metrics

Profiler

HBase Configuration

Startup Progress

Peers

Peer Id	Cluster Key	Endpoint	State	IsSerial	Remote WAL	Sync Replication State	Bandwidth	ReplicateAll	Namespaces	Exclude Namespaces	Table Cfs	Exclude Table Cfs
Total: 0												

Tasks

Show All Monitored Tasks

Show non-RPC Tasks

Show All RPC Handler Tasks

Show Active RPC Calls

Show Client Operations

View as JSON

Start Time	Description	State	Status	Completion Time
Sun Aug 30 18:23:01 IST 2026	Closing region 1595e783b53d99cd5eeef43b6debb2682 as it is being closed	COMPLETE (since 33sec ago)	Closed (since 33sec ago)	Sun Aug 30 18:23:01 IST 2026 (since 33sec ago)
Sun Aug 30 18:23:01 IST 2026	Initializing region master.store.,1.1595e783b53d99cd5eeef43b6debb2682.	COMPLETE (since 32sec ago)	Region opened successfully (since 32sec ago)	Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)
Sun Aug 30 18:23:01 IST 2026	Initializing region hbase.meta.,1.1588230740	COMPLETE (since 32sec ago)	Region opened successfully (since 32sec ago)	Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)
Sun Aug 30 18:23:01 IST 2026	Closing region 1588230740 as it is being closed	COMPLETE (since 32sec ago)	Closed (since 32sec ago)	Sun Aug 30 18:23:01 IST 2026 (since 32sec ago)
Sun Aug 30 18:23:05 IST 2026	Initializing region hbase.meta.,1.1588230740	COMPLETE (since 29sec ago)	Region opened successfully (since 29sec ago)	Sun Aug 30 18:23:05 IST 2026 (since 29sec ago)
Sun Aug 30 18:23:00 IST 2026	Master startup	COMPLETE (since 24sec ago)	Initialization successful (since 24sec ago)	Sun Aug 30 18:23:09 IST 2026 (since 24sec ago)
Sun Aug 30 18:23:09 IST 2026	Progress after master initialized	COMPLETE (since 24sec ago)	Progress after master initialized complete (since 24sec ago)	Sun Aug 30 18:23:09 IST 2026 (since 24sec ago)

Software Attributes

Attribute Name	Value	Description
JVM Version	Eclipse Adoptium 17.0.15-17.0.15+6	JVM vendor and version
HBase Version	3.0.0, revision=da418afa6973b64222231c1d233410d9c18ac7ec	HBase version and revision
HBase Compiled	Fri Jul 17 15:43:49 UTC 2026, work	When HBase version was compiled and by whom
HBase Source Checksum	63f026b7bbcc4d89a7d710cb962270650549bfddc8cc78e6cfc6a206bd5e7e919be092d5e359ed42798656aebd47b1317bc4a871e584f1c36d8f64c33ed7fe4b	HBase source SHA512 checksum
Hadoop Version	3.4.3, revision=9d50c6884666e794e45102260a4017bb31802e1b	Hadoop version and revision
Hadoop Compiled	2026-02-12T11:31Z, stevel	When Hadoop version was compiled and by whom
Hadoop Source Checksum	2331238c4c2929e66645316a32a8613	Hadoop source MD5 checksum
ZooKeeper Client Version	3.8.6, revision=6df26081269769c160c8c3a24929c60c91cd19c3	ZooKeeper client version and revision hash
ZooKeeper Client Compiled	2026-01-28 20:28 UTC	When ZooKeeper client version was compiled

Q2: Create and Manage a Table in HBase Shell

1. Open a terminal in the Cloudera VM.
2. Start HBase shell: hbase shell
3. Create a table named student with column family info: create 'student', 'info'
4. Insert sample records:
 - put 'student', '101', 'info:name', 'Anu'
 - put 'student', '101', 'info:dept', 'IT'
 - put 'student', '102', 'info:name', 'Bala'
 - put 'student', '102', 'info:dept', 'CSE'
5. Retrieve all records from the table: scan 'student'
6. Retrieve the record corresponding to row key 101: get 'student', '101'

OUTPUT:

```
(base) john@ohns-McBook-Air-2 ~ % hbase shell
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hbase/3.0.0/libexec/lib/client-facing-thirdparty/log4j-slf4j-impl-2.25.4.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hadoop/3.5.0/libexec/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
HBase Shell
Use "help" to get list of supported commands.
Use "exit" to quit this interactive shell.
For Reference, please visit: https://hbase.apache.org/docs/shell
Version 3.0.0, rds418afa6973bf4222231cd233410d8c18ac7ec, Fri Jul 17 15:43:49 UTC 2026
Took 0.0004 seconds
hbase:001:0> status
2026-08-30T18:26:26.690 INFO [main] util.log: Logging initialized @865ms to org.eclipse.jetty.util.log.Slf4jLog
1 active master, 0 backup masters,
1 servers,
0 decommissioned,
0 dead,
1.0000 average load
Took 0.4939 seconds
hbase:002:0> list
TABLE
0 row(s)
Took 0.0153 seconds
=> []
hbase:003:0> create 'student', 'info'
Created table student
Took 0.6996 seconds
=> hbase:table - student
hbase:004:0> put 'student', '101', 'info:name', 'Anu'
hbase:005:0> put 'student', '101', 'info:dept', 'IT'
Took 0.0832 seconds
hbase:006:0> put 'student', '102', 'info:name', 'Bala'
hbase:007:0> put 'student', '102', 'info:dept', 'CSE'
Took 0.0067 seconds
Took 0.0023 seconds
hbase:008:0> scan 'student'
ROW COLUMN+CELL
101 column=info:dept, timestamp=2026-08-30T18:27:11.207, value=IT
101 column=info:name, timestamp=2026-08-30T18:27:11.199, value=Anu
102 column=info:dept, timestamp=2026-08-30T18:27:18.722, value=CSE
102 column=info:name, timestamp=2026-08-30T18:27:18.716, value=Bala
2 row(s)
Took 0.0591 seconds
hbase:009:0> get 'student', '101'
COLUMN CELL
info:dept timestamp=2026-08-30T18:27:11.207, value=IT
info:name timestamp=2026-08-30T18:27:11.199, value=Anu
1 row(s)
Took 0.0172 seconds
hbase:010:0> exit
(base) john@ohns-McBook-Air-2 ~ % hbase org.apache.hadoop.hbase.util.HBaseConfTool hbase.rootdir
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hbase/3.0.0/libexec/lib/client-facing-thirdparty/log4j-slf4j-impl-2.25.4.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/homebrew/Cellar/hadoop/3.5.0/libexec/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
file:///opt/homebrew/var/hbase
(base) john@ohns-McBook-Air-2 ~ %
```

RESULT:

Thus, HBase was successfully explored using Cloudera Manager, including its service status, performance metrics, and configuration parameters. A student table was created using HBase Shell, sample records were inserted successfully, and the stored records were retrieved using scan and get commands.

EXP NO: 13**DATA VISUALIZATION USING TABLEAU****DATE:****AIM:**

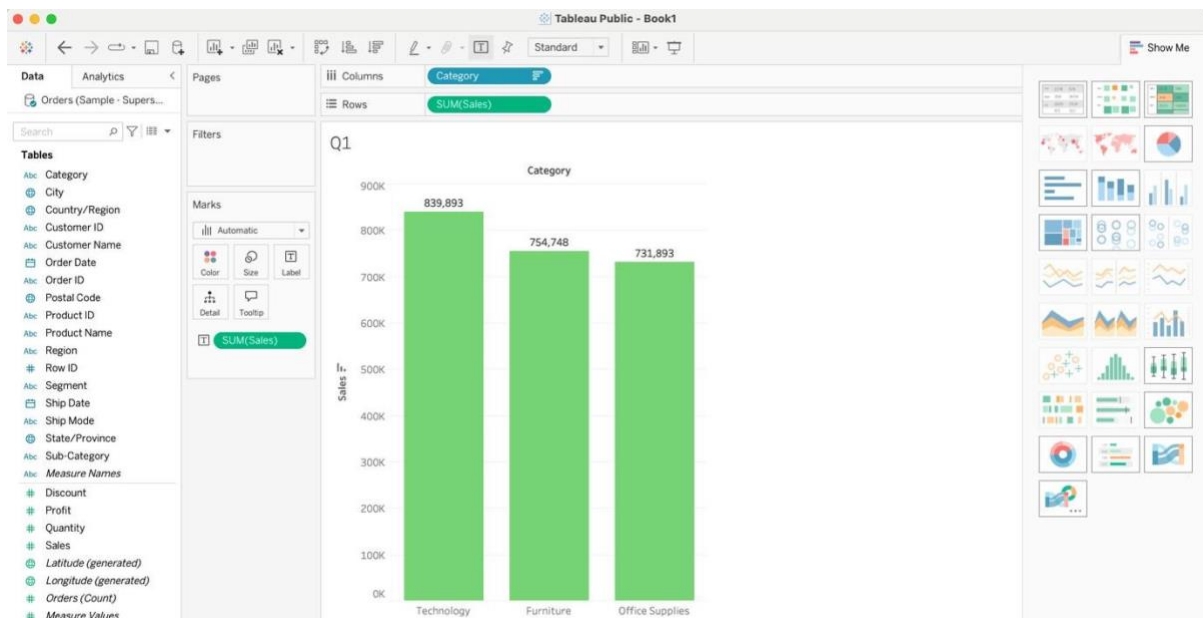
To analyse and visualize the Sample Superstore dataset using Tableau by creating different charts, performing sales and profit analysis, applying filters and time hierarchy drill-down, and designing an interactive business intelligence dashboard.

Q1. Sales Across Product Categories

QUESTION: Create a bar chart to compare sales across different product categories.

PROCEDURE:

1. Open Tableau Public.
2. Connect to the Sample – Superstore dataset.
3. Create a new worksheet and rename it Q1.
4. Drag Category to Columns.
5. Drag Sales to Rows.
6. Select Bar Chart from Show Me.
7. Drag Sales to Label in the Marks card to display the sales values.
8. Compare the total sales of Technology, Furniture, and Office Supplies.

OUTPUT:

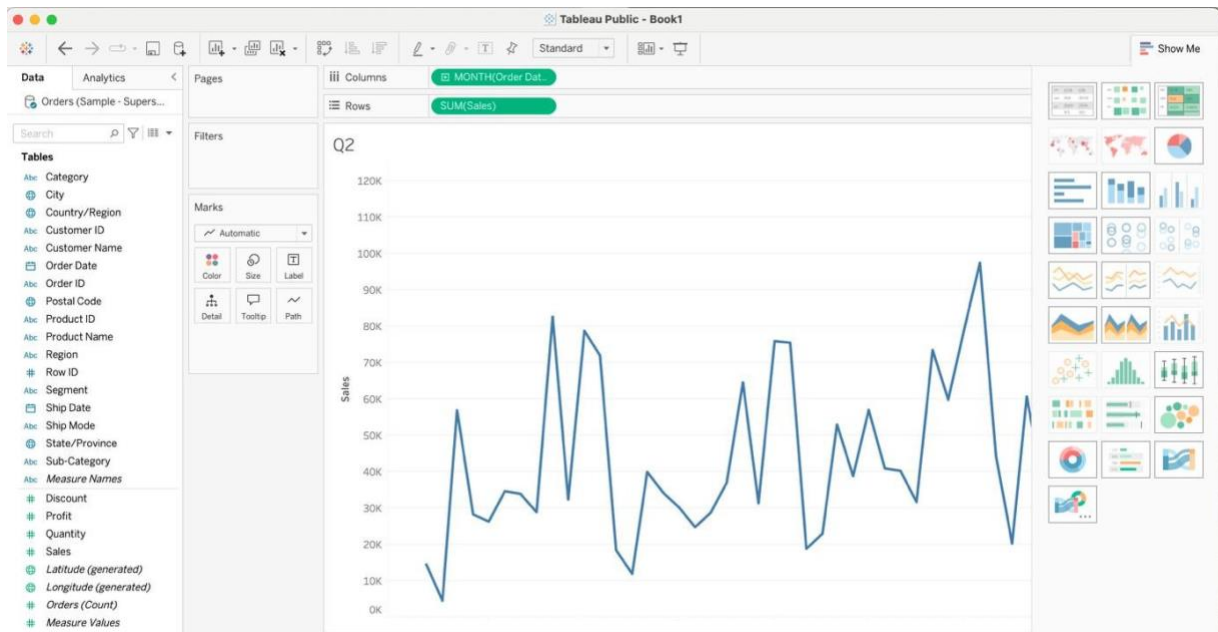
Q2. Monthly Sales Trend

QUESTION: Visualize how sales change over time using a line chart and identify the month with the highest sales.

PROCEDURE:

1. Create a new worksheet named Q2.
2. Drag Order Date to Columns.
3. Change the date level to Month.
4. Drag Sales to Rows.
5. Select Line Chart from Show Me.
6. Observe the variation in sales over different months.
7. Locate the highest point on the line chart to identify the month having maximum sales.

OUTPUT:

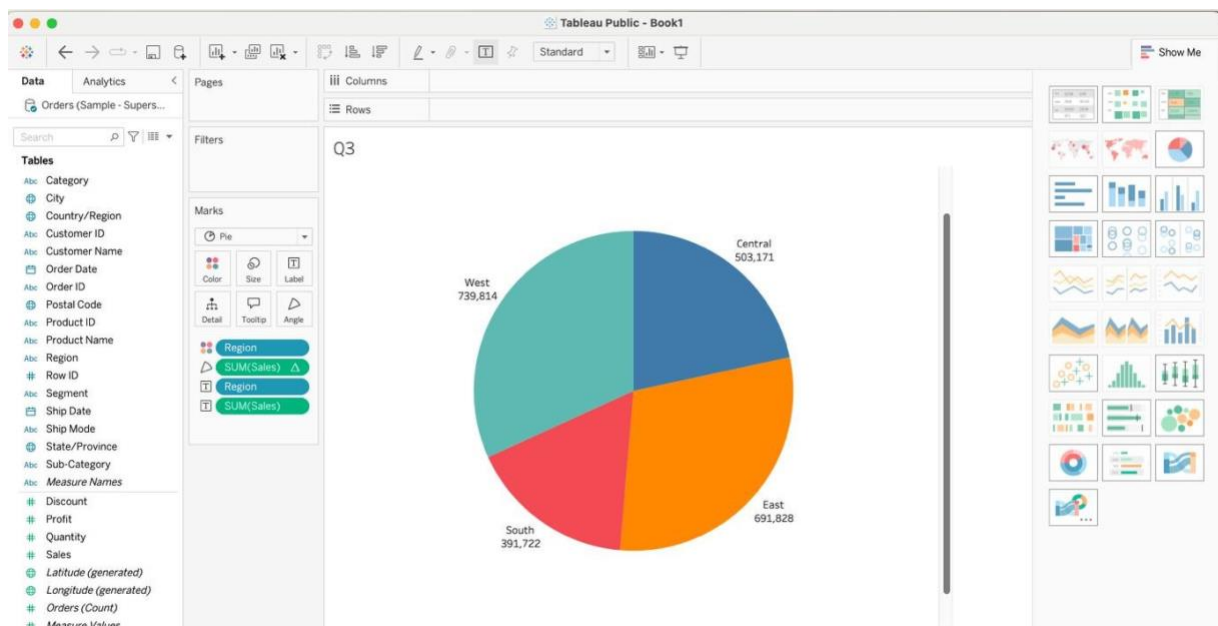


Q3. Regional Contribution to Sales

QUESTION: Show the contribution of different regions to total sales using a pie chart.

PROCEDURE:

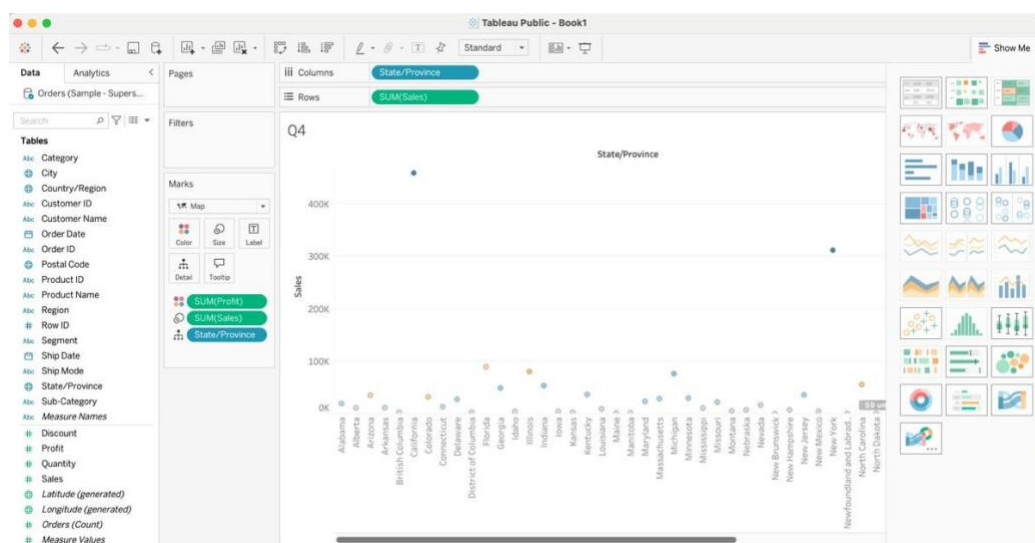
1. Create a new worksheet named Q3.
2. Select Pie as the Marks type.
3. Drag Region to Colour.
4. Drag Sales to Angle.
5. Drag Region and Sales to Label.
6. Adjust the size of the pie chart for clear visualization.
7. Compare the contribution of Central, East, South, and West regions.

OUTPUT:**Q4. Geographical Sales and Profit Analysis**

QUESTION: Visualize geographical differences in sales and profit using a geographic visualization.

PROCEDURE:

1. Create a new worksheet named Q4.
2. Use State/Province as the geographical dimension.
3. Drag Sales and Profit to the appropriate Marks properties.
4. Use Sales to represent the magnitude/size of the marks.
5. Use Profit to distinguish the profitability of different states.
6. Observe variations in sales and profit across different states.

OUTPUT:

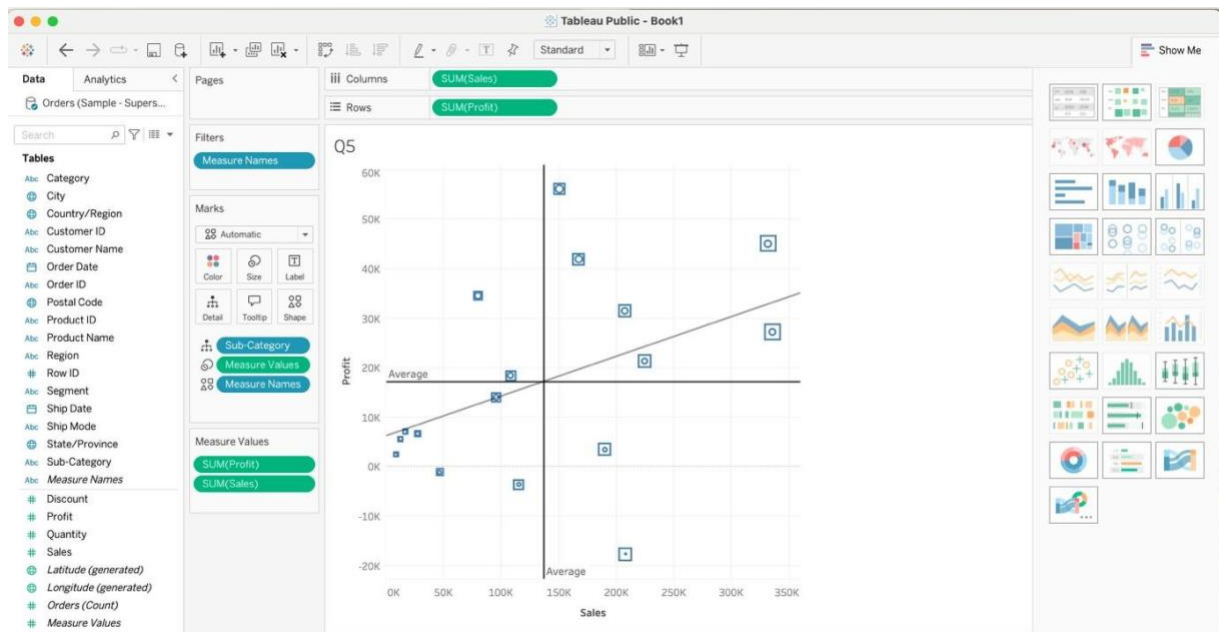
Q5. Relationship Between Sales and Profit

QUESTION: Determine the relationship between Sales and Profit using a scatter plot. Identify sub-categories having high/low sales and profit and add a trend line.

PROCEDURE:

1. Create a worksheet named Q5.
2. Drag Sales to Columns.
3. Drag Profit to Rows.
4. Drag Sub-Category to Detail in the Marks card.
5. Create the resulting Scatter Plot.
6. Add average reference lines for Sales and Profit.
7. The reference lines divide the scatter plot into four regions:
 - High Sales – High Profit
 - High Sales – Low Profit
 - Low Sales – High Profit
 - Low Sales – Low Profit
8. Open the Analytics pane.
9. Add a Trend Line to study the overall relationship between Sales and Profit.

OUTPUT:



Q6. Interactive Business Intelligence Dashboard

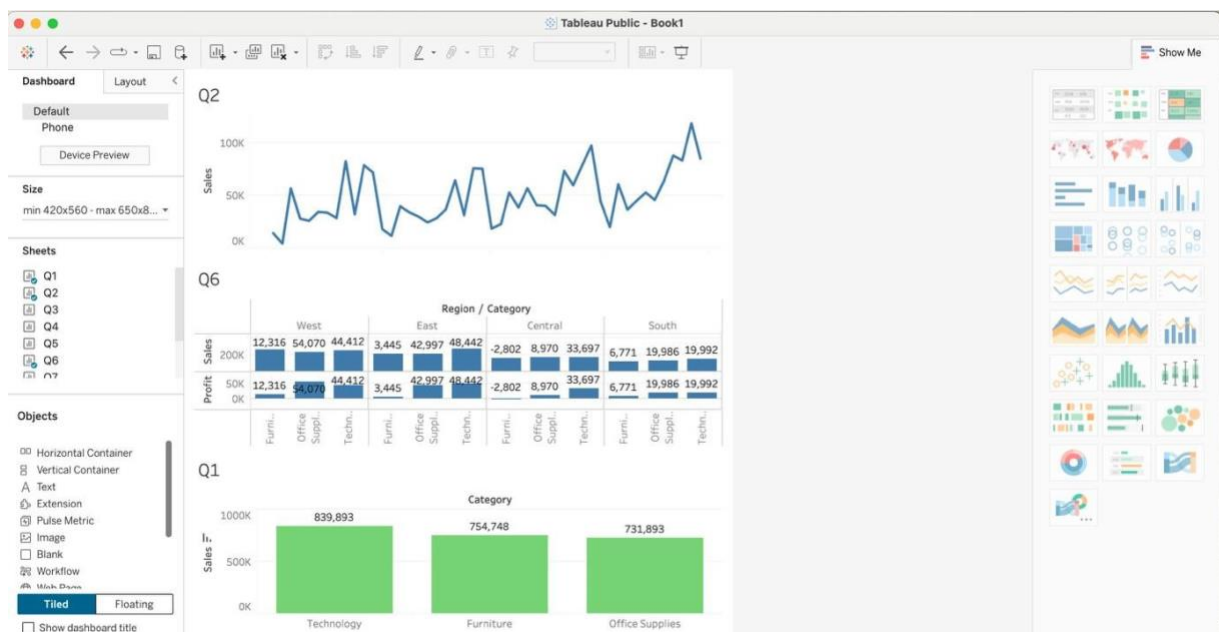
QUESTION: Combine multiple visualizations into a single dashboard and design a complete interactive business intelligence dashboard.

PROCEDURE:

1. Click New Dashboard in Tableau.

2. Select an appropriate dashboard size.
3. Drag the required worksheets from the Sheets panel onto the dashboard.
4. Arrange the visualizations clearly using tiled containers.
5. Resize each visualization for proper readability.
6. Include multiple analyses such as:
 - Category/Region-wise Sales
 - Monthly Sales Trend
 - Profit analysis
 - Sales comparison
7. Arrange them into a single dashboard for consolidated analysis.

OUTPUT:

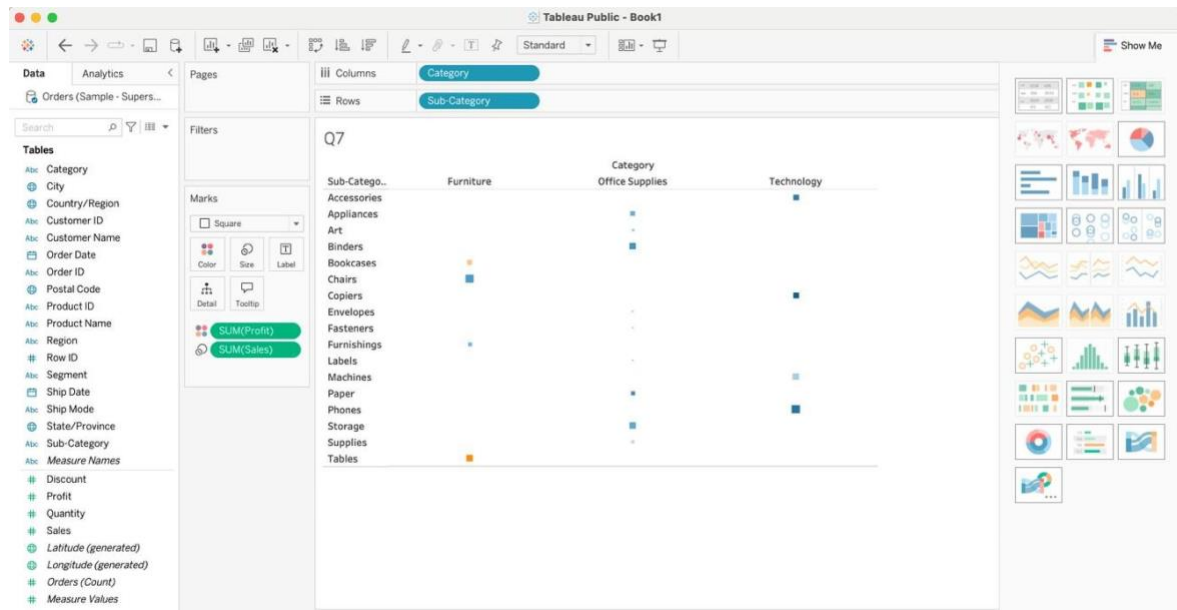


Q7. Profitable and Loss-Making Product Combinations

QUESTION: Identify profitable and loss-making product combinations using a heat map.

PROCEDURE:

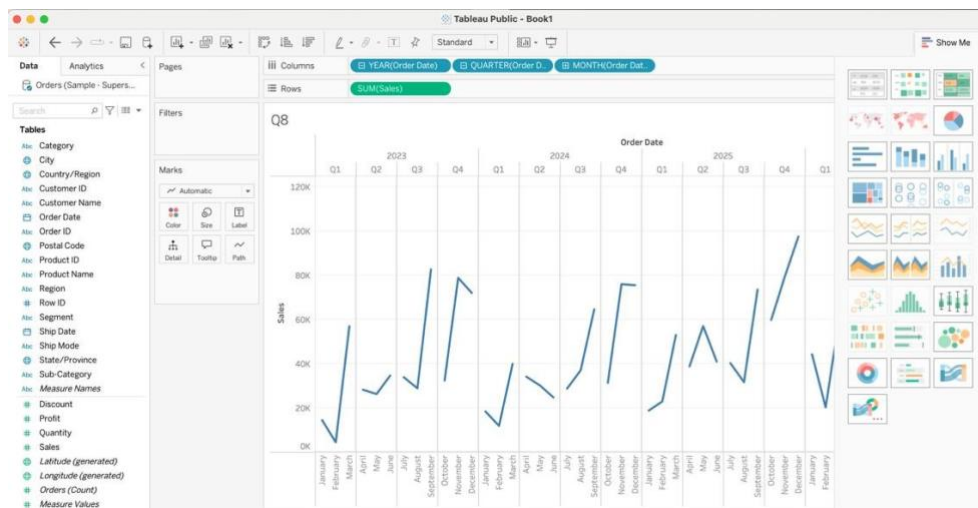
1. Create a worksheet named Q7.
2. Drag Category to Columns.
3. Drag Sub-Category to Rows.
4. Change the Marks type to Square.
5. Drag Profit to Colour.
6. Drag Sales to Size.
7. Observe the different Category–Sub-Category combinations.
8. Use the colour variation to distinguish profitable and loss-making combinations and mark size to compare sales.

OUTPUT:**Q8. Time Hierarchy Drill-Down Analysis**

QUESTION: Analyse sales at different levels of the time hierarchy using drill-down analysis for Year, Quarter and Month.

PROCEDURE:

1. Create a worksheet named Q8.
2. Drag Order Date to Columns.
3. Drag Sales to Rows.
4. Expand the Order Date hierarchy from: Year → Quarter → Month
5. Keep YEAR(Order Date), QUARTER(Order Date) and MONTH(Order Date) visible.
6. Use a line chart to visualize Sales.
7. Observe sales variations at yearly, quarterly, and monthly levels.

OUTPUT:

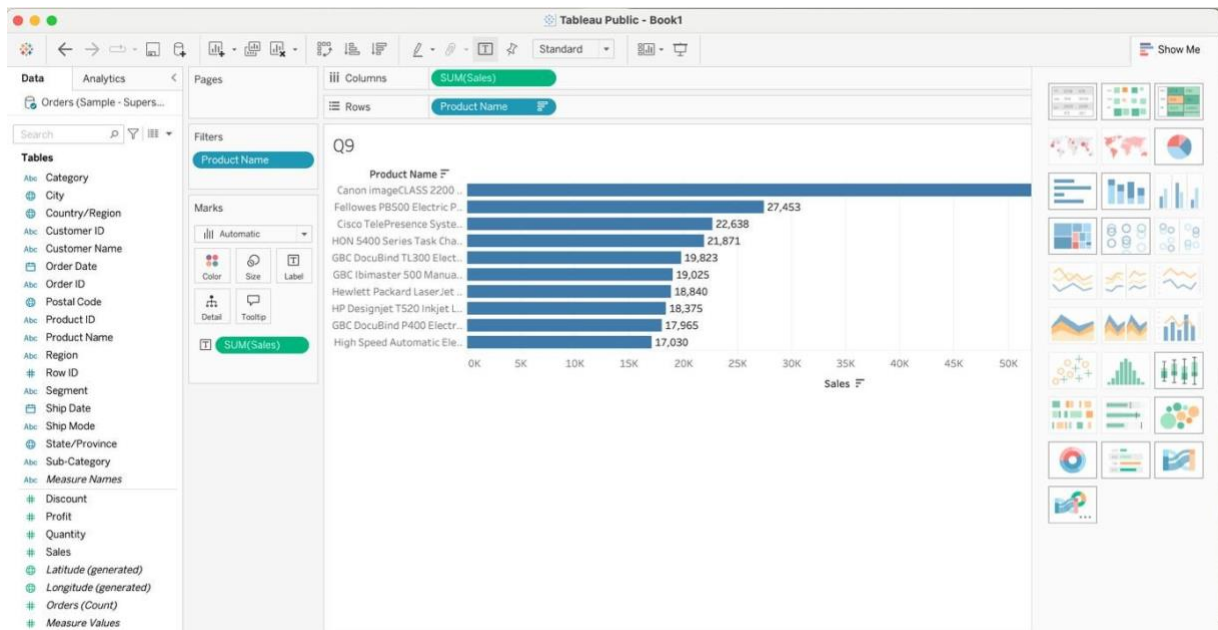
Q9. Top-Performing Products

QUESTION: Identify the top-performing products using a filter and draw a bar chart.

PROCEDURE:

1. Create a worksheet named Q9.
2. Drag Product Name to Rows.
3. Drag Sales to Columns.
4. Select a horizontal Bar Chart.
5. Drag Product Name to the Filters shelf.
6. Apply a Top N filter based on SUM(Sales).
7. Sort the products in descending order of Sales.
8. Drag Sales to Label to display values.
9. Analyse the products generating the highest sales.

OUTPUT:



RESULT:

Thus, the Sample Superstore dataset was successfully analysed and visualized using Tableau. Various visualizations such as bar chart, line chart, pie chart, geographical visualization, scatter plot, heat map, time hierarchy drill-down, and top-product analysis were created. Sales and profit patterns across categories, regions, states, time periods, and products were analysed, and multiple visualizations were combined to form an interactive business intelligence dashboard.